



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Grado en Ingeniería Informática

Trabajo Fin de Grado

**Booster: Una nueva plataforma de
videos de código abierto**
www.boostervideos.net

Autor: Samuel Caraballo Chichiraldi
Tutor: Fernando Pérez Costoya

Madrid, Junio 2026

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado
Grado en Ingeniería Informática

Título: Booster: Una nueva plataforma de videos de código abierto
www.boostervideos.net

Junio 2026

Autor: Samuel Caraballo Chichiraldi

Tutor: Fernando Pérez Costoya

Departamento de Arquitectura y Tecnología de Sistemas Informáticos

Escuela Técnica Superior de Ingenieros Informáticos

Universidad Politécnica de Madrid

Resumen

El trabajo aborda el diseño desarrollo e implementación de una red social orientada a compartir y distribuir contenido audiovisual en línea. El objetivo principal es desarrollar un prototipo funcional de una plataforma de streaming de videos, denominada Booster, que permita a los usuarios publicar, compartir, descubrir e interactuar con contenido digital de manera sencilla e intuitiva centrada en la comunidad.

Booster surge así como respuesta a problemas existentes identificados por múltiples usuarios en las plataformas de videos actuales. En particular la limitada interacción entre creadores y audiencias; los algoritmos de recomendación opacos; la escasa participación de los usuarios en el proceso de descubrimiento de comunidades; la gran cantidad de contenido publicitario; la creciente creación de contenido con base en IA sin la debida advertencia y la ausencia de mecanismos que incentiven la contribución dentro de una comunidad, solo por mencionar algunos. Para resolver estos problemas, Booster introduce elementos de gamificación y mecanismos de interacción social para fomentar la participación, fortalecer el sentido de pertenencia de la comunidad y reducir el contenido comercial.

Adicionalmente, el trabajo realiza un análisis completo de la arquitectura y las tecnologías disponibles. Se evalúan distintos proveedores de infraestructura en la nube, servicios dedicados al procesamiento y distribución de video, sistemas de almacenamiento, bases de datos y herramientas de desarrollo, con el fin de alinear las decisiones de diseño de forma eficiente con los objetivos técnicos y empresariales del proyecto.

El trabajo recopila el proceso completo de ingeniería de software desde la definición de requisitos y el diseño de la arquitectura hasta la implementación de funcionalidades y la puesta en marcha del servicio en un entorno de producción. De ese modo, la solución busca ofrecer una alternativa a las plataformas de streaming de video tradicionales que pretende optimizar el descubrimiento de contenido, aumentar las interacciones sociales y la participación en comunidades.

Finalmente, el trabajo demuestra a través del desarrollo de un Mínimo Producto Viable (MVP) www.boostervideos.net la viabilidad técnica de una plataforma de este tipo que introduce nuevas funcionalidades más allá de la simple distribución de contenido en busca de una experiencia de usuario más dinámica,

orientada a la participación y a la creación de comunidades digitales.

Abstract

This work addresses the design, development, and implementation of a social network for sharing and distributing online audiovisual content. The main objective is to develop a functional prototype of a video streaming platform, called Booster, that allows users to easily and intuitively publish, share, discover, and interact with digital content in a community-centric way.

Booster thus emerges as a response to existing problems identified by many users on current video platforms. These include limited interaction between creators and audiences; opaque recommendation algorithms; low user participation in the community discovery process; the large amount of advertising content; the increasing creation of AI-based content without proper warning; and the lack of mechanisms to incentivize contributions within a community, to name just a few. To solve these problems, Booster introduces gamification elements and social interaction mechanisms to encourage participation, strengthen community sense of belonging, and reduce commercial content.

Additionally, this work includes a comprehensive analysis of the architecture and available technologies. Various cloud infrastructure providers, dedicated video processing and distribution services, storage systems, databases, and development tools are evaluated to efficiently align design decisions with the project's technical and business objectives.

This work compiles the complete software engineering process, from requirements definition and architecture design to functional implementation and service deployment in a production environment. The solution aims to offer an alternative to traditional video streaming platforms, optimizing content discovery, increasing social interactions, and fostering community engagement.

Finally, the work demonstrates, through the development of a Minimum Viable Product (MVP) www.boostervideos.net, the technical feasibility of such a platform, which introduces new functionalities beyond simple content distribution, striving for a more dynamic user experience focused on engagement and the creation of digital communities.

Tabla de contenidos

1. Introducción	1
1.1. Motivación y necesidades del proyecto	1
1.2. Objetivos	3
1.3. Estructura de la memoria	4
2. Estado del Arte	7
2.1. Procesamiento, Transcodificación, Segmentación y Entrega de Video	7
2.1.1. FFmpeg	7
2.1.2. Docker	9
2.1.3. MPEG-DASH	10
2.1.4. HLS	11
2.1.5. Object Storage	12
2.1.6. CDN - Content Delivery Network	13
2.1.7. Colas de Procesamiento	15
2.2. Análisis de Proveedores	16
2.2.1. Amazon Web Services	16
2.2.2. Mux.com	18
2.2.3. Bunny.net	20
2.2.4. Tabla comparativa de proveedores	22
2.3. Modelos de Arquitectura	22
2.3.1. Modelo Centralizado	23
2.3.2. Modelo Federado	25
2.3.3. Modelo Peer-to-Peer (P2P)	27
3. Tecnologías Empleadas	29
3.1. Backend	29
3.1.1. TypeScript como lenguaje de programación	29
3.1.2. Base de Datos	31
3.1.3. ORMs	33
3.1.4. Comunicación Cliente Servidor - tRPC	35
3.1.5. Next.js - backend	37
3.1.6. Autenticación - ClerkAuth	39
3.2. Frontend	40
3.2.1. React.Js	40
3.2.2. Next.js - frontend	41
3.2.3. TailwindCSS y Shadcn	41

TABLA DE CONTENIDOS

4. Innovaciones	43
4.1. Problemas detectados y la solución de Booster	43
4.1.1. Problemas con el algoritmo de recomendación	43
4.1.2. Problemas en encontrar comunidades	44
4.1.3. Problemas con los anuncios	46
4.2. Mejoras e Innovaciones	47
4.2.1. Mejoras en la interfaz de usuario	47
4.2.2. Mejoras en los métodos de moderación	54
4.2.3. Mejoras en los comentarios	56
4.2.4. Mejoras en los canales de usuario	57
4.2.5. Incorporación de gamificación	61
5. Análisis	63
5.1. Requisitos Funcionales	63
5.1.1. Gestión de usuario	63
5.1.2. Carga, gestión y moderación de videos	64
5.1.3. Reproducción y consumo de contenido	64
5.1.4. Exploración y búsqueda	65
5.1.5. Interacción social	65
5.1.6. Comunidades	65
5.1.7. Canales	66
5.1.8. Gamificación	66
5.2. Requisitos No Funcionales	67
5.3. Actores	67
5.4. Casos de uso	68
5.4.1. Gestión de usuarios	68
5.4.2. Carga, gestión y moderación de videos	75
5.4.3. Reproducción y consumo de contenido	82
5.4.4. Exploración y búsqueda	86
5.4.5. Interacción social	90
5.4.6. Comunidades	95
5.4.7. Canales	98
5.4.8. Gamificación	101
5.5. Diagramas de secuencia	106
5.5.1. Carga de videos	106
5.5.2. Registro	107
5.5.3. Inicio de sesión	108
5.5.4. Vista de Video	109
5.6. Diagrama Conceptual de la Base de Datos	110
6. Diseño y Arquitectura	115
6.1. Patrones y mejores prácticas	115
6.1.1. Mono-Repositorio con Next.js	115
6.1.2. Estructura general de archivos	116
6.1.3. Separación de funcionalidades y Arquitectura modular	118
6.1.4. Rutas agrupadas en Next.js	118
6.1.5. Acceso a datos mediante ORM	119
6.1.6. Esquemas de validación	119

6.1.7. Centralización de integraciones externas	119
6.1.8. Seguridad mediante variables de entorno	119
6.1.9. Middleware para control de accesos	119
6.1.10 Reutilización de componentes visuales	120
6.1.11 Prefetch de datos	120
6.1.12 Convención de nombres y estructura	120
6.1.13 Procedimientos públicos y protegidos	120
6.2. Diseño de la Autenticación	121
6.2.1. Middleware	121
6.2.2. Usuario Clerk vs Usuario Interno	122
6.2.3. Creación y Sincronización de Usuarios	122
6.3. Diseño de la Base de Datos	122
6.3.1. Tablas	123
6.4. Arquitecturas de procesamiento de videos	136
6.4.1. Arquitectura de procesamiento propia	136
6.4.2. Arquitectura de procesamiento con un proveedor	137
6.5. Diseño del Algoritmo de Recomendación	140
6.6. Diseño de la búsqueda semántica de videos	141
7. Desarrollo	143
7.1. Precarga de Datos con tRPC	143
7.2. Implementación del procesamiento de videos propio	145
7.2.1. Consumidor	145
7.2.2. Procesador de video	148
7.3. Implementación del procesamiento de videos con Bunny	154
7.4. Implementación de los comentarios	159
7.5. Acceder y modificar datos de usuario	159
7.6. Acceder y modificar datos de video	160
8. Despliegue y Riesgos de Seguridad	163
8.1. Despliegue	163
8.1.1. Vercel	163
8.2. Feedback de Usuarios	165
8.2.1. Opiniones positivas y sugerencias	165
8.2.2. Opiniones negativas y preocupaciones	168
8.3. Vulnerabilidades detectadas y acciones tomadas	170
8.3.1. Protección de accesos y modificación de videos	170
8.3.2. Webhook de Bunny	171
8.3.3. Inyección SQL detectada	172
8.3.4. Limitaciones de uso (Rate Limits)	173
9. Conclusiones	175
10. Análisis de Impacto	177
Bibliografía	179

Capítulo 1

Introducción

1.1. Motivación y necesidades del proyecto

Los servicios de video bajo demanda se encuentran en una fase de crecimiento continuo y altísima intensidad de uso, con YouTube, TikTok, Netflix y los Reels de Meta siendo las principales plataformas dominantes del sector. Esto se confirma con los datos de Google [1] quien reporta que los ingresos de YouTube por publicidad y suscripciones superaron los 60 mil millones de dólares en 2025, teniendo un alcance publicitario mensual de 2.53 mil millones de usuarios a inicios de ese año, según reporta DataReportal [2].

Adicionalmente, en el mes de abril de ese mismo año YouTube informó que la plataforma acumulaba más de 20 mil millones de videos cargados, agregando más de 20 millones de videos nuevos cada día. De igual forma, en 2024 promedió más de 100 millones de comentarios diarios y más de 3.5 mil millones de "likes" por día. Esto evidencia no sólo el tamaño de la escala, sino también la intensidad de uso de las plataformas de este estilo a nivel global.

En 2025, TikTok [3] reportó superar los 200 millones de usuarios mensuales en Europa. Por otro lado, estimaciones de eMarketer señalaban que el tiempo diario destinado por un usuario en la plataforma rondaba los 52 minutos.

También Netflix, informó [4] que recibió más de 325 millones de membresías de pago y 96 mil millones de horas vistas durante el segundo semestre de 2025. Esto equivale a aproximadamente 522 millones de horas reproducidas por día durante ese semestre, lo cual es una muestra de la persistencia de consumo y la alta inversión en tiempo que dedican los usuarios.

En Meta, la situación es similar. La empresa [5] reportó 3.58 mil millones de cuentas activas por día en su familia de aplicaciones en diciembre de 2025 y comunicó, adicionalmente, que en Estados Unidos el tiempo de visualización de videos en Facebook creció más de 20 % interanual, lo que refleja el consumo masivo de audiovisuales en línea y el dominio del video de formato corto basado en algoritmos que priorizan el contenido dinámico.

Es importante destacar, el impacto medioambiental que tiene este consumo ma-

Capítulo 1. Introducción

sivo de contenido en línea. Un estudio realizado por La Agencia Internacional de la Energía [6] estimó que los centros de datos consumieron alrededor de 415 TWh en 2024, equivalentes a cerca de 47.4 GW de demanda media, lo cual se aproxima a 1.5% del consumo mundial de electricidad. Aunque esa cifra no se puede atribuir exclusivamente al contenido de videos en línea, un estudio de Global Internet Phenomena de AppLogic confirmó que el streaming de video continúa siendo la principal fuente de tráfico en Internet, con 38% del total global, y que las aplicaciones de redes sociales de videos en línea y entretenimiento dominan el volumen de red por internet.

Estos datos, demuestran el estado actual de las plataformas de streaming de video bajo demanda, el alcance masivo que tienen y su elevadísima frecuencia de uso. Además de aplicaciones de entretenimiento, son servicios digitales con infraestructuras extremadamente complejas capaces de concentrar a miles de millones de personas, acumular millones de horas de video y moldear los hábitos de entretenimiento de la sociedad.

No obstante, cuanto más dominantes se han vuelto estas plataformas, más visibles se han hecho las observaciones, quejas y descontentos de sus usuarios.

En cuanto al sesgo, las cámaras de eco y la sensación de encierro algorítmico, los datos son significativos. Ofcom [7] encontró que alrededor de un tercio de los usuarios manifiesta incomodidad explícita con los algoritmos de recomendación y que, a su vez, estos no suelen favorecer las opiniones con las que discrepa el usuario.

Con respecto al contenido producido por IA, contenido inapropiado y desinformación, la escala del malestar es incluso más evidente. Ofcom [8] reportó que 41% de los adultos usuarios de Internet en el Reino Unido encontró desinformación, 22% vio imágenes o videos falsos o engañosos, y 25% dijo haber quedado realmente molesto u ofendido por el contenido al cual fue expuesto. De igual forma, el 47% de los entrevistados manifestó nunca haber visto herramientas, etiquetas o íconos que le ayudaran a entender si un contenido había sido creado o editado con IA, aunque el 85% consideró importante que las plataformas lo informen.

Asimismo, un estudio académico publicado en European Journal of Operational Research en 2025 [9], demostró un malestar significativo de los usuarios frente a la sobrecarga publicitaria recibida a través de las redes sociales de video. Tomando datos globales sobre uso de bloqueadores de anuncios, el mismo estudio señala que las razones más frecuentes para bloquear contenido es la cantidad excesiva de publicidad (62.1%) y el carácter intrusivo (54.2%). Esto constituye una señal clara de fatiga publicitaria en entornos digitales de streaming de video. A ello se suma que, el estudio global Media Reactions 2025 de Kantar mostró que la percepción de una plataforma mejora cuando sus anuncios son vistos como menos intrusivos, como ocurre con Snapchat. Esto confirma que la intrusividad publicitaria es un criterio central de evaluación para los usuarios de plataformas sociales de video y un motivo de malestar muy frecuente para los mismos.

Inspirado en la necesidad de resolver estos problemas y entendiendo que existe una brecha en el mercado que define una oportunidad, nace la idea de *Booster* como plataforma de video alternativa, con un algoritmo original y enfocado en la experiencia de usuario para minimizar los problemas presentados. En definitiva, *Booster* no se plantea como una réplica de las plataformas existentes, sino como una respuesta tecnológica a necesidades no resueltas en el mercado, con el fin de ofrecer un entorno más equilibrado y confiable tanto para quienes consumen contenido como para quienes lo producen.

1.2. Objetivos

El objetivo de *Booster* es diseñar y desarrollar una plataforma de video web sustentada en un algoritmo de recomendación impulsado por la participación activa de los usuarios en la plataforma, nuevos métodos de moderación, una disminución de la cantidad de anuncios publicitarios y la posibilidad de elegir voluntariamente exposiciones a la publicidad, una experiencia de usuario basada en métodos de gamificación y la habilidad de crear y encontrar comunidades de usuarios y contenido más fácilmente.

De esta forma, *Booster* está orientado a responder a una necesidad creciente del servicio de video bajo demanda, con el propósito de ofrecer a creadores y usuarios un entorno más transparente y eficiente que las plataformas dominantes actuales.

Esta necesidad surge de la acumulación de problemas que afectan tanto la experiencia de consumo como la de la creación de contenidos. Entre estos, las dificultades de monetización para los creadores, la opacidad algorítmica en la distribución de contenidos, la formación de cámaras de eco, la reproducción de sesgos en los sistemas de recomendación, la abundancia de contenido generado por inteligencia artificial sin mecanismos suficientes de identificación y la percepción de censura injustificada derivada de procesos de moderación poco claros. En este contexto, *Booster* se propone como una solución tecnológica capaz de abordar dichos problemas mediante un algoritmo original que optimice la personalización y el descubrimiento de contenido.

En consecuencia, *Booster* se plantea como una nueva plataforma de distribución audiovisual y también como una propuesta de arquitectura digital orientada a resolver necesidades reales del mercado.

Para desarrollar esta plataforma, se implementará una página web para llegar a un público más amplio mediante el uso de tecnologías de vanguardia que faciliten el desarrollo. Además, se implementará un sistema de procesamiento y transcodificación de videos que permitirá a los usuarios subir su propio contenido audiovisual y que estos puedan ser reproducidos en la plataforma.

1.3. Estructura de la memoria

El trabajo de fin de grado se organiza en una secuencia de capítulos que conduce al lector desde la comprensión del problema hasta la formulación, diseño y validación de la solución propuesta, de la siguiente forma:

- **Capítulo 1 - Introducción:** En este capítulo se presenta el contexto del trabajo, el estado actual del mercado de la distribución de contenido audiovisual mediante plataformas bajo demanda, los problemas que enfrentan los usuarios y también los creadores, así también como los objetivos que se persiguen con el desarrollo de *Booster*.
- **Capítulo 2 - Estado del Arte:** En este capítulo se presentan las tecnologías esenciales para desarrollar un sistema de procesamiento de video. Se realiza un análisis de 3 proveedores de servicios de infraestructura de contenido multimedia comparando sus fortalezas y debilidades. Finalmente, se presentan los 3 modelos de arquitectura posibles para implementar una plataforma de videos y la seleccionada para este trabajo.
- **Capítulo 3 - Tecnologías Estudiadas:** En este capítulo se detalla el análisis tecnológico que fundamenta al proyecto *Booster*. Se describen las principales tecnologías web seleccionadas y las descartadas para el objetivo del proyecto. La finalidad es mostrar que las decisiones técnicas adoptadas responden a criterios de escalabilidad y usabilidad.
- **Capítulo 4 - Innovaciones:** Se describen las innovaciones tecnológicas, de diseño y de experiencia de usuario que se han implementado para solventar los problemas identificados. Se detallan los aspectos novedosos de la plataforma y su factor diferencial frente a las alternativas existentes.
- **Capítulo 5 - Requisitos del Sistema:** En este capítulo se indican los requisitos funcionales y no funcionales de *Booster*. Se especifican las capacidades mínimas que ofrece la plataforma, las condiciones de rendimiento esperadas, las restricciones del sistema y las necesidades de usuarios y creadores de contenido así como también el diseño Entidad-Relación de la base de datos junto con los diagramas de secuencia y casos de uso más relevantes.
- **Capítulo 6 - Diseño y Arquitectura:** En este capítulo se presenta la arquitectura general de *Booster*; el procesamiento y transcodificación de videos; los componentes principales del sistema, su interacción y el diseño de la interfaz de usuario. Aquí se explican, entre otros elementos, el algoritmo de recomendación, los sistemas de interacción como comentarios y puntos de experiencia (XP), las interfaces de usuario, la búsqueda semántica de videos, el diseño de la autenticación y de la base de datos.
- **Capítulo 7 - Desarrollo:** Se detalla la implementación de ciertos componentes de la aplicación y su integración, los retos técnicos enfrentados y las soluciones adoptadas.
- **Capítulo 8 - Pruebas, Validación y Riesgos de Seguridad:** Este capítulo

recoge las pruebas realizadas sobre la plataforma, tanto desde el punto de vista funcional como de seguridad, desempeño y experiencia de usuario. Se detallan medidas de seguridad tomadas, ataques reales detectados y mitigados, y se presentan los resultados de las pruebas.

- **Capítulo 9 - Conclusiones:** Se sintetizan los principales resultados del trabajo, evaluando qué se ha conseguido con el desarrollo de *Booster* según los objetivos planteados. Asimismo, se valoran las aportaciones del proyecto, sus limitaciones y las posibles mejoras o ampliaciones de la plataforma.
- **Capítulo 10 - Análisis de Impacto:** Se realiza un análisis del impacto del proyecto, considerando sus posibles implicaciones tecnológicas, económicas y medioambientales.

Capítulo 2

Estado del Arte

Este capítulo constituye un estudio técnico exhaustivo para identificar fortalezas y oportunidades de mejora en las tecnologías más conocidas para implementar la infraestructura de procesamiento de audiovisuales. Asimismo, integra un análisis de los principales proveedores disponibles en el mercado teniendo en cuenta aspectos técnicos y financieros. Finalmente, se exploran distintos modelos de arquitectura empleados por las aplicaciones que brindan este servicio.

2.1. Procesamiento, Transcodificación, Segmentación y Entrega de Video

En esta sección se hace un análisis de las herramientas más utilizadas para procesar, almacenar, transcodificar y entregar el contenido multimedia en la plataforma.

2.1.1. FFmpeg

FFmpeg es una herramienta de código abierto utilizada para el procesamiento de contenido multimedia, en particular audio y video. Es empleada en una amplia variedad de tareas y aplicaciones como:

- **Transcodificación:** FFmpeg puede convertir videos entre distintos formatos, códecs ¹ y resoluciones, lo que es esencial para garantizar la compatibilidad con diferentes dispositivos y plataformas. Por ejemplo, se puede convertir un video en formato `.mov` a `.mp4`, o extraer el audio de un archivo `.mp4` a `.mp3`. También se emplea para generar distintas calidades de reproducción de un video, 360p, 720p, 1080p, entre otros.
- **Compresión y Optimización:** Permite reducir el tamaño de archivos sin perder calidad significativa ajustando la resolución, códec y el *bitrate* ². Es-

¹Se entiende por códec a aquella tecnología software o hardware que permite comprimir o descomprimir archivos multimedia.

²Se entiende por bitrate (tasa de bits) como la cantidad de información que se transmite por unidad de tiempo.

to es esencial para mejorar la experiencia de usuario. Por ejemplo, en este tipo de servicios se suele utilizar **Adaptive Bitrate**. Esto consiste en guardar varias copias del mismo contenido con diferentes calidades y tamaños y, de esta forma, el reproductor puede seleccionar la versión más adecuada según la velocidad de conexión del usuario.

- **Edición de video:** Permite realizar tareas básicas de edición como recortar la duración, unir varios videos, rotar, etc.
- **Streaming:** FFmpeg puede ser utilizado para transmitir video en tiempo real a través de protocolos como RTP o UDP.
- **Segmentación:** FFmpeg permite segmentar un video para dividirlo en trozos más pequeños. Esto resulta útil para implementar protocolos de streaming de videos.



Figura 2.1: Logo de FFmpeg

Ventajas

La principal ventaja de FFmpeg es su versatilidad y flexibilidad. Puede aplicar filtros complejos, producir salidas en distintos formatos, ajustar la resolución, bitrate, códecs, frame rate y parámetros de la codificación dentro del mismo flujo de trabajo. Eso lo vuelve muy útil para transcodificación, remuxing, extracción de audio, recortes, subtítulo, entre otros casos de uso en el procesamiento de video.

Es una herramienta *open source* lo que implica que reduce la dependencia de un tercero. Está disponible gratuitamente y está bien documentada. Todo esto facilita su integración en cualquier tipo de proyecto, evitando así el *vendor lock-in*.

La herramienta destaca por su velocidad y eficiencia, implementando algoritmos avanzados de procesamiento de video. Además, es compatible con una amplia variedad de formatos y códecs y tiene soporte para diversos sistemas operativos. Por estas razones, FFmpeg es una opción popular y altamente utilizada en la industria para el procesamiento de videos.

Desventajas

La principal desventaja de la herramienta es su complejidad. Requiere un conocimiento técnico profundo para su uso efectivo. A esto, se le suma su sintaxis compleja y la gran cantidad de opciones disponibles, lo que puede resultar abrumador para los usuarios sin experiencia.

2.1. Procesamiento, Transcodificación, Segmentación y Entrega de Video

FFmpeg es el ganador en términos de flexibilidad y control. Sin embargo, cuando el objetivo es la velocidad extrema o el ahorro masivo de energía, el hardware dedicado o las soluciones implementadas desde cero pueden superar a FFmpeg. Por ejemplo, el chip personalizado de Google Argos VCU (Video Coding Unit) supera a FFmpeg en velocidad, eficiencia de cómputo y ahorro energético optimizando la capacidad de los centros de datos de YouTube. [10]

Servicios que emplean FFmpeg

- Netflix, que utiliza esta herramienta para la transcodificación de sus videos.
- Youtube, que incorpora FFmpeg y tecnologías personalizadas basadas en FFmpeg para procesar contenido multimedia.
- Meta, que también emplea FFmpeg para la transcodificación en plataformas como Facebook e Instagram.
- TikTok, que también utiliza FFmpeg para procesar los videos junto con tecnologías personalizadas.
- NASA, utiliza FFmpeg en algunos de sus rovers marcianos para comprimir los videos capturados por las cámaras de estos robots antes de ser enviados a la Tierra.

2.1.2. Docker

Docker es una herramienta que permite empaquetar una aplicación junto con sus dependencias, variables de entorno, librerías y configuraciones en contenedores o microservicios. Una aplicación contenedora es un paquete independiente y aislado que ejecuta un programa. De esta forma, el software se puede desplegar de manera portátil y compatible en distintos equipos o servidores. En definitiva, funciona como una máquina virtual que se despliega según unas instrucciones pero que presentan un menor peso [11].

Para conseguir esto, se utiliza normalmente un archivo llamado *Dockerfile*, donde se definen las instrucciones, librerías y configuraciones necesarias para construir la imagen del servicio. Aquí, se instalan los paquetes y las dependencias necesarias para poner en marcha el programa, los archivos que deben copiarse al contenedor, el comando que se debe ejecutar al iniciar el servicio, entre otros.

Ventajas

La principal ventaja de *Docker* es su portabilidad. Un mismo contenedor puede construirse una vez y ejecutarse de forma consistente en un gran número de equipos. Esto reduce los errores por entorno, dependencias, configuraciones inconsistentes, entre otros.

Además, cada contenedor se ejecuta de forma aislada sin interferir con otros sistemas. Esto resulta en *Booster* ya que se puede implementar un microservicio que se encargue del procesamiento, transcodificación y segmentación de video sin alterar a otros servicios o funcionalidades.

Por otra parte, a diferencia de las máquinas virtuales tradicionales, *Docker* es más ligero. Al compartir el kernel con el sistema que lo despliega (*host*), se consiguen lanzar múltiples workers con menos infraestructura.

Desventajas

A pesar de todos sus beneficios, *Docker* añade complejidad adicional en la arquitectura de los servicios. Los contenedores se deben gestionar adecuadamente, orquestrar y configurar de forma correcta para garantizar que cumplen con sus objetivos funcionales. Además, requieren invertir esfuerzo en optimizar la carga de trabajo. Si no se diseña con cuidado, se pueden estar utilizando más recursos de los necesarios, afectando así los costos operativos.

Esto lo convierte en una tecnología con una curva de aprendizaje elevada, que requiere un buen conocimiento para aprovechar al máximo sus capacidades.

Uso de Docker en Booster

En *Booster*, *Docker* se utiliza para lanzar un microservicio que se encarga del procesamiento, transcodificación y segmentación de video. Este utiliza, herramientas como FFmpeg 2.1.1 para generar distintas calidades y segmentar el contenido preparándolo para el streaming. Además, se utiliza otro microservicio llamado "consumidor" que recibe las peticiones de procesamiento de un video y se encarga de generar la tarea correspondiente.



Figura 2.2: Logo de Docker

2.1.3. MPEG-DASH

MPEG-DASH [12] (Dynamic Adaptive Streaming over HTTP) es un estándar internacional de transmisión de video sobre HTTP, enfocado en una distribución eficiente definido en la familia **ISO/IEC 23009**. Para esto, implementa técnicas de **Adaptive Bitrate** las cuales permiten ajustar automáticamente la calidad de reproducción, cambiando parámetros como el bitrate y la resolución del video según las condiciones de red y las capacidades del dispositivo del usuario. De esta forma, se garantiza una experiencia de visualización fluida y sin interrupciones, incluso en conexiones lentas e inestables.

Ventajas

La principal ventaja de MPEG-DASH es su escalabilidad y eficiencia. Al hacer uso de HTTP, se puede aprovechar la infraestructura existente de servidores web y CDNs para distribuir el contenido eficazmente a una audiencia global. Además, al implementar Adaptive Bitrate el video se divide en segmentos pequeños y se reduce la latencia.

Asimismo, MPEG-DASH es ampliamente compatible con una variedad de códecs, formatos de videos y dispositivos, lo que lo hace una opción versátil para la

2.1. Procesamiento, Transcodificación, Segmentación y Entrega de Video

distribución de contenido multimedia.

Desventajas

MPEG-DASH, puede ser complejo de implementar y configurar. DASH, suele exigir una configuración cuidadosa de los servidores y los reproductores de video.

Aunque DASH funciona bien para la transmisión de video, no es la mejor opción para escenarios donde se requiere una latencia sumamente baja. En tales casos, se puede optar por utilizar Low-Latency DASH (LL-DASH) u otros protocolos alternativos como WebRTC.

Otra desventaja importante es que este protocolo no tiene soporte nativo en Apple, lo que limita su incorporación en este tipo de dispositivos.

Servicios que implementan DASH

DASH es un estándar abierto y empleado en gran cantidad de servicios de streaming. Algunos de estos son:

- YouTube es uno de los casos más importantes. Google informa a los desarrolladores que YouTube utiliza MPEG-DASH en su plataforma HTML5 para implementar Adaptive Bitrate.
- Brightcove, una empresa proveedora de servicios de software en la nube especializada en el alojamiento, distribución y gestión de video online, también ha informado que utiliza MPEG-DASH para la entrega de contenido multimedia.
- Bitmovin, otro proveedor de infraestructura de streaming, también ha confirmado que emplea MPEG-DASH en su plataforma para la entrega de video.
- Vimeo, una plataforma de alojamiento de videos similar a YouTube, también ha confirmado que utiliza MPEG-DASH e implementa Adaptive Bitrate.
- Odysee y *PeerTube*, plataformas de videos descentralizadas y basadas en tecnologías blockchain, adoptan también MPEG-DASH para la entrega de contenido multimedia.

2.1.4. HLS

De forma similar a MPEG-DASH, **HLS** [13] (HTTP Live Streaming) también es un protocolo de transmisión de video desarrollado por Apple. HLS implementa técnicas de **Adaptive Bitrate** y es utilizado para la distribución de contenido audiovisual bajo demanda y en directo fundamentalmente en dispositivos Apple, aunque es compatible con otros sistemas operativos.

Ventajas

Usualmente, este protocolo divide el video en segmentos pequeños y en formato de archivo `.m3u8` que permite entregar el contenido de forma eficiente por HTTP usando CDNs y servidores web. El reproductor de video solicita los segmentos

más adecuados teniendo en cuenta la calidad de conexión del usuario mejorando así la experiencia de visualización.

Además, HLS es altamente escalable. Esto lo hace ideal para la distribución de contenido a gran escala siendo una opción popular para los servicios de streaming en vivo y bajo demanda durante momentos de elevado tráfico.

HLS, es compatible con una amplia variedad de dispositivos especialmente con los dispositivos Apple, lo que lo convierte en una opción preferida para la distribución de contenido a usuarios de iOS y macOS.

Desventajas

Una de las desventajas más notorias de este protocolo es la falta de soporte en algunos dispositivos y navegadores, especialmente en aquellos que no responden a las configuraciones Apple. Esto puede limitar su adopción en ciertos entornos y requerir la implementación de soluciones alternativas para garantizar la compatibilidad con una audiencia más amplia. [14]

Adicionalmente, HLS puede ser inadecuado para circunstancias en las que se requiera una latencia extremadamente baja, como en llamadas de video en tiempo real. Debido a su arquitectura, para este tipo de aplicaciones, otros protocolos como WebRTC podrían ser más apropiados. [15]

Servicios que implementan HLS

La gran mayoría de actores principales en el mercado de streaming implementan este protocolo. Este es el caso de:

- Apple, siendo el creador del protocolo e implementándolo en sus dispositivos y servicios de streaming.
- AWS, en su servicio de live streaming
- Mux, un proveedor de infraestructura de streaming que ofrece soporte para HLS.
- Cloudflare, que también ofrece servicios de streaming con soporte para HLS.

Cabe destacar que, en ambos protocolos HLS y MPEG-DASH, el video no se descarga por completo en una única acción, sino que el reproductor va solicitando pequeños segmentos o fragmentos al servidor conforme avanza la reproducción. En otras palabras, el video está almacenado en partes pequeñas en el servidor, cada una con diferentes calidades, y el reproductor le solicita la parte que mejor se adapte a las condiciones de red del usuario en cada momento. Así, se evitan transmitir grandes cantidades de datos innecesarios por la red y se puede implementar un Adaptive Bitrate eficiente.

2.1.5. Object Storage

El contenido multimedia, en particular los videos, suelen ocupar un gran espacio de almacenamiento. Además, con las técnicas de **Adaptive Bitrate** es necesario

2.1. Procesamiento, Transcodificación, Segmentación y Entrega de Video

almacenar varias copias del mismo video con diferentes calidades y tamaños, lo cual incrementa el uso de espacio de forma considerable. Por ello, es fundamental contar con un sistema de almacenamiento eficiente y escalable. Por esa razón, se suelen emplear sistemas de almacenamiento de objetos **Object Storage** [16] como Google Cloud Storage, Amazon S3 Buckets o Azure Blob Storage que permiten almacenar grandes cantidades de datos de forma escalable y con alta disponibilidad. Esto es esencial para garantizar un acceso rápido y confiable a los videos por parte de los usuarios.

Ventajas

La mayor ventaja es su escalabilidad. Amazon S3 o Google Cloud Storage, son servicios de almacenamiento en la nube que permiten guardar y recuperar cantidades masivas de datos solucionándole al usuario complicaciones con la gestión de particiones, discos, entre otros.

Estos servicios suelen ofrecer una alta disponibilidad y durabilidad, lo que prácticamente garantiza que los datos estarán siempre accesibles y protegidos contra pérdidas.

Desventajas

Aunque son capaces de almacenar grandes cantidades de datos, el acceso a los mismos no es tan rápido. Los Object Storage no están pensados para accesos con latencias extremadamente bajas. Por esa razón, suelen ser utilizados en conjunto con CDNs que permiten entregar el contenido de forma más eficiente a los usuarios finales.

Servicios de Object Storage

Todos las grandes plataformas de streaming emplean Object Storage para almacenar sus videos. Los servicios más comunes para implementar esta tecnología son:

- Amazon S3 Buckets, es el ejemplo más conocido y es extensivamente usado por empresas y servicios como Netflix, Disney+, Twitch, Hulu y Prime Video para almacenar su contenido.
- Google Cloud Storage, es otro servicios de Object Storage empleado por YouTube, Spotify, Vimeo, entre otros.
- Azure Blob Storage, otro servicio de Object Storage utilizado por BBC Player, NBA, Real Madrid, entre otros.

2.1.6. CDN - Content Delivery Network

Un Content Delivery Network (CDN) [17] es una red de servidores distribuidos geográficamente que trabajan juntos para entregar contenido de manera rápida y eficiente a los usuarios finales. Un usuario, al solicitar un video, es atendido por el servidor más cercano a su ubicación, con el objetivo de reducir la latencia, aumentar la escalabilidad y mejorar el rendimiento del servicio. Su uso, se vuelve necesario en plataformas con alta demanda y con usuarios distribuidos

Capítulo 2. Estado del Arte

geográficamente, donde depender exclusivamente del servidor de origen generaría mayores tiempos de respuesta, mayores costos y una sobrecarga innecesaria sobre la infraestructura principal.

De este modo, las CDN funcionan como una caché de contenido de los Object Storage, replicando o almacenando temporalmente los archivos en distintos nodos de la red para servirlos de manera más eficiente a los usuarios finales. Por tanto, los Object Storage cumplen la función de almacenamiento central del contenido, mientras que las CDN asumen la función de distribución acelerada.

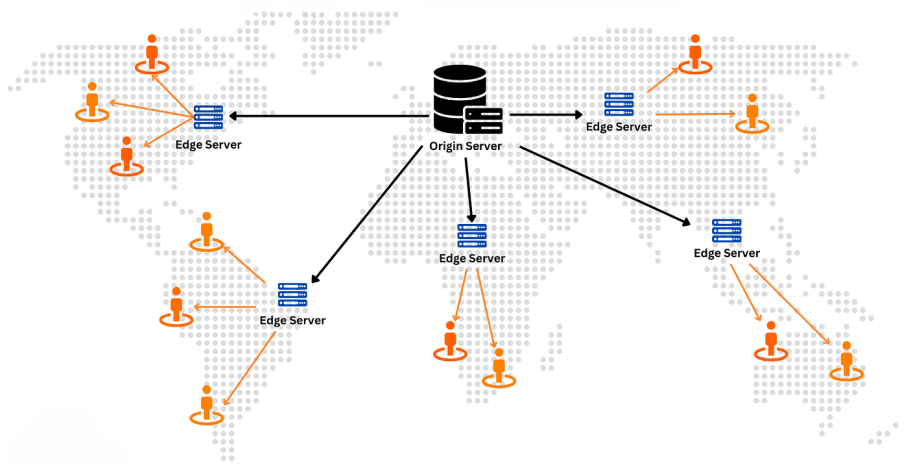


Figura 2.3: Diagrama de un CDN

Ventajas

Claramente, la principal ventaja de los CDN es que reducen la latencia y mejoran el rendimiento del servicio. Como los servidores están distribuidos geográficamente, el usuario accede al servidor más cercano y esto, reduce el tiempo de carga, mejorando la experiencia de usuario.

Además, el uso de CDN reduce la carga en los servidores de origen y en los Object Storage. Al servir contenido desde los nodos de la CDN, el servidor principal recibe menos solicitudes directas, cumpliendo así una función de caché que alivia la infraestructura principal.

Los CDN también facilitan la escalabilidad del servicio. En momentos de alta demanda, distintos CDN pueden absorber el tráfico adicional permitiendo que el servicio se mantenga estable y sin depender exclusivamente del servidor de origen.

Por otra parte, un gran número de CDN suelen ofrecer capacidades adicionales de seguridad. Esto incluye protección contra ataques DDoS, firewalls (WAF) y otras medidas de seguridad que ayudan a proteger el contenido y la infraestructura del servicio.

Desventajas

La desventaja más evidente es el costo. Aunque mejora la experiencia de usua-

2.1. Procesamiento, Transcodificación, Segmentación y Entrega de Video

rio, facilita la escalabilidad y ofrece seguridad adicional, los CDNs suelen tener precios por transferencia, almacenamiento y solicitudes elevados, lo que puede resultar costoso para servicios con grandes volúmenes de tráfico y contenido.

Asimismo, los CDNs introducen complejidad adicional en la arquitectura del servicio. Se deben definir políticas de caché, gestionar la invalidación de contenido y configurar adecuadamente la distribución del contenido entre los distintos nodos. Incluso, puede existir la posibilidad de servir contenido obsoleto o incorrecto si no se gestiona adecuadamente la sincronización entre el servidor de origen y los nodos de la CDN. Por tanto, el uso de CDNs requiera una gestión constante y una planificación rigurosa añadiendo complejidad a la arquitectura y a la operación.

Servicios que emplean CDNs

A pesar de sus desventajas, el uso de CDNs es muy común en la industria, especialmente en el servicio de streaming. Normalmente, cada servicio utiliza sus propios CDNs para evitar la dependencia de un proveedor y ahorrar costos. Sin embargo, algunos de los servicios de CDNs más populares son:

- Cloudflare CDN, es un servicio de CDN utilizado por empresas como HyperConnect para el streaming de video y comunicaciones en tiempo real a escala global.
- Amazon CloudFront, es otra opción popular para entregar sitios web, APIs, archivos y streaming de videos globalmente.

2.1.7. Colas de Procesamiento

El procesamiento y la transcodificación de video son tareas que suelen tomar un largo tiempo en completarse. Por esa razón, cuando un usuario sube un video, en lugar de forzarlo a esperar, se utilizan colas de procesamiento. En vez de hacer las operaciones lentas en la misma petición HTTP, el sistema registra un trabajo pendiente en una cola y este espera a ser atendido por un consumer. A continuación, se realiza este trabajo y se reportan los resultados al finalizar. Así, se consigue desacoplar procesos lentos de otros más rápidos que de lo contrario causarían una mala experiencia de usuario.

En el contexto de procesamientos de videos el esquema es el siguiente:

- 1. El usuario sube el video
- 2. El sistema guarda el archivo en un Object Store
- 3. Se crea un job en la cola de procesamiento
- 4. Un consumer toma el trabajo pendiente
- 5. Se ejecuta la transcodificación de video, se generan copias del video, se comprime, etc.
- 6. Al terminar se notifica al sistema

Existen diversas formas de implementar una cola de procesamiento. Por ejemplo, RabbitMQ, Kafka, Amazon SQS (Simple Queue Service) suelen ser opciones comunes.

Ventajas

La ventaja principal es que mejoran la experiencia de usuario. La subida de video suele ser un proceso rápido mientras que la transcodificación puede tardar minutos e incluso horas en finalizar.

Además, las colas de procesamiento facilitan la escalabilidad. Este modelo permite responder de forma efectiva ante picos de alta demanda sin colapsar el *backend* porque los trabajos permanecen en un estado de espera en la cola. Para evitar que haya saturaciones o una gran cantidad de trabajos pendientes en ser atendidos, una solución común es añadir más consumers según sea necesario.

Por otra parte, también favorecen la priorización de trabajos. Por ejemplo, dando paso a videos Premium o urgentes antes que al resto.

Desventajas

La desventaja más clara es que las colas de prioridad añaden complejidad a la arquitectura. Esto quiere decir que además de operar con las APIs y las bases de datos, ahora es necesario gestionar consumers, reintentos, resolver duplicados, timeouts, entre otros.

Servicios de Colas de Procesamiento

Cada servicio de streaming suele implementar su propia solución de colas de procesamiento adaptada a sus necesidades. Sin embargo, existen soluciones comunes como:

- Amazon SQS, es una cola de procesamiento general ofrecida por Amazon y utilizada por servicios como Fox Sports u otros clientes.
- RabbitMQ, es otra opción popular utilizada por empresas de broadcasting de contenido como Norwegian Broadcasting Corporation (NRK).
- Apache Kafka, es una plataforma de streaming de eventos ampliamente utilizada por empresas como Netflix, LinkedIn, entre otros.

2.2. Análisis de Proveedores

En esta sección se hace un estudio de los principales proveedores de infraestructura de streaming que se han considerado e implementado en el proyecto. Se analizan sus características, ventajas, desventajas, costos, facilidad de implementación e integración, entre otros aspectos relevantes.

2.2.1. Amazon Web Services

Amazon Web Services es una proveedor de infraestructura en la nube que permite al cliente implementar soluciones propias utilizando servicios modulares

según la necesidad. Esto le aporta al cliente la posibilidad de crear desde cero la lógica para el almacenamiento, entrega de video, transcodificación y segmentación, generación de miniaturas y carga de videos al servidor. Esta solución, aunque es más compleja, permite un control total sobre el funcionamiento del servicio.

Las funcionalidades de este proveedor serán desarrolladas en detalle en el capítulo de arquitectura donde se describe el diseño de una pipeline propio mediante los servicios de AWS y con las tecnologías expuestas en el capítulo anterior.



Figura 2.4: Logo de AWS

Ventajas

La principal ventaja que tiene Amazon Web Services es que permite diseñar un pipeline propio que facilita tener un control total sobre el funcionamiento del servicio. Esto permite ajustar cada etapa a las necesidades específicas del proyecto. Por ejemplo, se pueden ajustar los servicios de AWS para que se adapten a la escala, realizar el procesamiento de video de forma personalizada, entre otros.

A su vez, se evita la dependencia de un proveedor de infraestructura de streaming separado. Con esto se evita el *vendor lock-in* y se tiene la libertad de cambiar o ajustar los servicios de AWS según sea necesario sin depender de las limitaciones de un proveedor externo.

Asimismo, AWS es un servicio usado extensivamente en la industria, con una disponibilidad y escalabilidad probada.

Desventajas

Evidentemente, la principal desventaja es la complejidad que añade. Implementar esta solución implica gestionar y configurar cada uno de los servicios de AWS. Además, requiere saber sobre cada una de las etapas del proceso en detalle y su integración. Un proveedor externo suele resolver todos los problemas de mantenimiento, almacenamiento, colas, procesamiento y seguridad, mientras que con esta solución, el equipo de desarrollo es el responsable de gestionar cada uno de estos aspectos. En consecuencia, los recursos y el tiempo de implementación son mayores.

Uno tendería a pensar que esta solución es más eficiente en términos de costos, pero no siempre es así, como se demostrará más adelante.

Servicios que emplean AWS

Capítulo 2. Estado del Arte

Algunos Servicios de streaming que emplean AWS o utilizan servicios de AWS para su infraestructura son:

- Netflix, para su infraestructura de streaming, utiliza AWS para almacenar y entregar su contenido multimedia.
- Amazon Prime Video, que es el servicio de streaming de Amazon, también utiliza AWS para su infraestructura de streaming.
- Twitch, una plataforma de streaming de videojuegos, también utiliza AWS para su infraestructura de streaming.
- Vision+, un servicio de streaming de Indonesia, usa servicios de AWS.

Análisis de Costos

Para calcular el costo aproximado de esta solución, se deben considerar cada uno de los componentes del proceso por separado. Aunque más tarde, se detallará el funcionamiento y la razón de uso de cada uno de estos servicios, a continuación se hace un desglose de los costos aproximados para cada uno de los servicios:

- Amazon S3 Buckets: Suponiendo 500 GB de datos almacenados cada mes y tomando el precio para la región de España de un S3 Bucket Standard (\$ 0,023 por GB) el costo total es: $500 \text{ GB} \cdot 0,023\$ \text{ por GB} = \$11,5$ al mes.
- Amazon EC2: Suponiendo un *t3.medium* que trabaja 24 horas por día, y tomando el precio por hora de \$ 0,0416 el costo total es: $0,0416\$ \text{ por hora} \cdot 24 \text{ horas por día} \cdot 30 \text{ días por mes} = \$29,95$ al mes.
- Amazon SQS: Suponiendo 2 millones de solicitudes al mes, y tomando \$ 0,40 como el precio estándar por cada millón de solicitudes, el costo total es: $2 \text{ millones de solicitudes} \cdot 0,40\$ \text{ por millón de solicitudes} = \$0,80$ al mes.
- Amazon CloudFront: Si se entregan 2TB de datos al mes y tomando el precio para la región de Europa de \$0,09 por GB, el costo total es: $2 \text{ TB} \cdot 1024 \text{ GB por TB} \cdot 0,09\$ \text{ por GB} = \$184,32$ al mes.

Sumando cada uno de estos costos, el costo total aproximado con este proveedor es: $\$11,5 + \$29,95 + \$0,80 + \$184,32 = \$226,57$ al mes.

2.2.2. Mux.com

Mux es un proveedor de infraestructura de streaming de video orientado a desarrolladores. Permite subir videos, procesarlos, entregarlos, almacenarlos, gestionarlos, acceder a estadísticas, entre otras funcionalidades. Mux, facilita el proceso de desarrollo ya que se encarga de gestionar y mantener la infraestructura de streaming. Por esta razón, es una opción común para aquellos equipos que no disponen de los recursos o el tiempo necesario para implementar una solución propia.

Ventajas



Figura 2.5: Logo de Mux

Reduce el tiempo de desarrollo, los recursos necesarios y la complejidad de la implementación. Mux, se encarga de resolver los problemas de almacenamiento, entrega y procesamiento de video, lo que permite a los desarrolladores centrarse en otras áreas del proyecto.

Mux, ofrece una documentación clara y una API flexible y fácil de usar con una gran variedad de funcionalidades. Tiene soporte para una amplia variedad de tecnologías web, lenguajes de programación y frameworks, facilitando su integración en cualquier tipo de proyecto. Esto lo hace ideal para equipos pequeños o para demostrar un producto mínimo viable (MVP) de forma rápida y barata.

A su vez, Mux ofrece características adicionales como un contador de visitas de los videos, análisis del tiempo de retención, geolocalización de las visualizaciones, entre otras estadísticas de video.

Desventajas

Al depender de un proveedor externo, se pierde el control sobre la arquitectura de procesamiento de los videos o posibles optimizaciones personalizadas que se deseen implementar.

Puede resultar una opción costosa a largo plazo, en especial para proyectos de gran escala o aquellos que esperan un crecimiento importante en el tráfico, volumen de datos, o videos subidos. Además, al ser un servicio de terceros, se está sujeto a las políticas de precios o cambios en la plataforma.

Servicios que emplean Mux

Mux resulta un proveedor de infraestructura de streaming popular en el sector, en particular para proyectos pequeños, startups o empresas medianas. Algunos de los servicios que emplean Mux son:

- Patreon, una plataforma de membresías, lo utiliza para integrar videos nativos en su plataforma
- Typeform, una plataforma diseñada para crear encuestas, emplea este servicio para incluir preguntas en video u otras funciones de videos en sus cuestionarios.
- Shopify, una empresa de *e-commerce*, utiliza en ciertas circunstancias la infraestructura de Mux para mostrar videos sobre los productos que vende.

Análisis de Costos

Según la página oficial de precios de Mux [18] y tomando como supuestos las cantidades de almacenamiento y transferencia anteriores:

- El tipo de recurso a transmitir es *On Demand Video*

- La resolución máxima es 1080p
- La calidad de video es *Basic*
- La cantidad de GB almacenados por mes son 500 GB .
- La cantidad de datos transferidos por mes son 2 TB.
- La cantidad de GB procesados por mes son 500 GB.

Con estas estimaciones, el costo aproximado por mes es \$20,00. Sin embargo, cabe destacar que si se considera usar una calidad de video superior como *Plus*, el precio ascendería hasta \$321,64 por mes.

2.2.3. Bunny.net

Bunny.net es una plataforma que ofrece varios servicios. Entre estos, se encuentra la entrega de contenido mediante CDNs, almacenamiento de objetos, streaming de videos, optimización de imágenes y seguridad contra ataques DDoS. El principal punto fuerte de Bunny.net es su precio competitivo en comparación con otros proveedores de infraestructura de streaming. Sin embargo, su integración requiere un mayor esfuerzo de desarrollo.



Figura 2.6: Logo de Bunny.net

Ventajas

La principal ventaja de Bunny.net son sus precios reducidos. El almacenamiento, procesamiento y distribución de contenido multimedia es costoso, pero sorprendentemente, Bunny.net ofrece una solución barata en comparación con una solución propia u otros proveedores de infraestructura de streaming. Esto lo hace ideal para proyectos con una escala pequeña o para entregar un producto mínimo viable (MVP) de forma rápida y barata.

BunnyStream, el servicio de streaming de Bunny.net, integra CDN, transcodificación, procesamiento de video, almacenamiento, estadísticas avanzadas, subtítulos automáticos, personalización del reproductor, entre otras funcionalidades. Esto lo convierte en una opción para startups que busquen una solución completa y económica para su servicio de streaming.

Desventajas

De forma similar a Mux, al depender de un proveedor externo, se pierde el control sobre la arquitectura de procesamiento de los videos, incluyendo optimizaciones personalizadas. Para una escala más grande, puede resultar ineficiente y en este caso, sería más recomendable implementar una solución propia con servicios de AWS o similares.

Asimismo, en comparación con Mux, Bunny.net tiene una documentación más limitada y no tan exhaustiva como en el caso de Mux. Esto dificulta la integración y el uso de algunas de sus funcionalidades. Emplear BunnyStream, requiere más tiempo, recursos y esfuerzos de desarrollo que Mux, lo que puede no ser ideal para equipos pequeños o con recursos limitados.

Otro aspecto importante a destacar es que Bunny.net ofrece menos personalización que Mux. BunnyStream incorpora un reproductor de video con diversas opciones, pero no siempre se puede adaptar el estilo del reproductor de forma completa y sencilla a las necesidades de una empresa.

Servicios que emplean Bunny.net

Bunny.net es una alternativa menos conocida que otras, siendo una opción más común para proyectos pequeños o medianos. Algunos de los servicios que emplean Bunny.net son:

- Younity, una aplicación de desarrollo personal, integra BunnyStream para streaming de sus cursos en vivo y bajo demanda.
- STIRR, un servicio americano de streaming de TV y películas, emplea Bunny.net para la entrega de su contenido multimedia.
- Circle.so, una plataforma de comunidades online, utiliza Bunny.net para la entrega de videos.

Análisis de Costos

Tomando los mismos supuestos que en el caso de Mux, AWS y los datos publicados en la web oficial de Bunny.net [19], el desglose de los costos aproximados es el siguiente.

- Almacenando 500 GB por mes, con un precio de \$ 0,01 por GB, el costo total es: $500 \text{ GB} \cdot 0,01\$ \text{ por GB} = \$5,00$ al mes.
- Para un procesamiento básico de video ³ Bunny.net no cobra. Por tanto para el supuesto de 500GB procesados por mes el costo es \$ 0,00 al mes.
- Los costos de transferencia varían según la cantidad y topología de CDNs utilizados. Utilizando 3 CDNs se obtiene 0,005 \$ por GB. Por tanto, el precio por mes para 2 TB de entrega de datos es: $2 \text{ TB} \cdot 1024 \text{ GB por TB} \cdot 0,005\$ \text{ por GB} = \$10,24$

El costo total aproximado para esta alternativa es: $\$5,00 + \$0,00 + \$10,24 = \$15,24$ al mes, lo que la convierte en una opción económica en comparación con otras alternativas. Asimismo, si se elige una calidad de video superior, el precio no se vería afectado como en el caso de Mux.

³Un procesamiento básico incluye funcionalidades como transcodificación, Adaptive Bitrate, salidas con códecs x264 y resoluciones configurables desde 240p hasta 2160p. El plan básico no tiene códecs más avanzados ni prioridades en las colas de procesamiento.

2.2.4. Tabla comparativa de proveedores

Criterio	Amazon Web Services	Mux.com	Bunny.net
Facilidad de implementación	Baja	Alta	Media
Flexibilidad e integración con distintas tecnologías	Alta	Alta	Media
Escalabilidad	Alta	Media	Media
Costos	Medios	Altos	Bajos
Soporte	Alto	Alto	Alto
Documentación	Alta	Alta	Media
Personalización	Alta	Media	Baja
Popularidad	Alta	Alta	Baja
Funcionalidades y características adicionales	No incluye características directas adicionales para streaming, por lo que su implementación debe realizarse desde cero.	Ofrece una amplia variedad de funcionalidades, como reproductor de video, subtítulos, estadísticas y moderación de contenido.	Ofrece diversas funcionalidades, incluyendo subtítulos, reproductor de video con opciones avanzadas y estadísticas.
Principal ventaja	Personalización extrema y control absoluto sobre la arquitectura.	Resuelve la complejidad del desarrollo.	Resuelve la complejidad del desarrollo con precios bajos.
Principal desventaja	Mayor complejidad, ya que requiere más recursos, tiempo de desarrollo, un equipo de DevOps y mantenimiento continuo.	Costos elevados y pérdida de control sobre aspectos de la arquitectura.	Pérdida de control en la arquitectura y dependencia de un tercero.

Cuadro 2.1: Tabla comparativa de los proveedores y servicios considerados durante el desarrollo del proyecto

2.3. Modelos de Arquitectura

En esta sección se describen y analizan los tres principales modelos de arquitectura que se pueden emplear en un servicio de streaming de video. Estos incluyen el modelo centralizado, el modelo federado y el modelo peer-to-peer (P2P). Se explican las características de cada uno, sus ventajas, desventajas o debilidades, tecnologías típicas asociadas a cada modelo y ejemplos de servicios que integran alguno de estos en su arquitectura.

2.3.1. Modelo Centralizado

La arquitectura centralizada es el modelo más común y tradicional para plataformas de streaming de video, redes sociales y otros servicios web. En este modelo, una sola organización controla y gestiona los componentes fundamentales del sistema. Esto incluye la autenticación de usuarios, las bases de datos, la transcodificación de videos, el almacenamiento, la entrega de contenido, la moderación de contenido y los algoritmos de recomendación.

En otras palabras, esto quiere decir que la plataforma centralizada recibe los videos en servidores propios o contratados, los procesa con su infraestructura y finalmente, los entrega a la audiencia a través de su propia red de distribución.

Desde un punto de vista más técnico, la arquitectura centralizada se caracteriza por tener un *backend* monolítico ⁴ o basado en microservicios, donde cada componente del sistema está gestionado por la misma organización. Además, no siempre es un único servidor central, sino que puede haber copias del mismo distribuidas geográficamente, pero el control y la gestión de los servicios está siempre bajo la misma entidad.

Normalmente, este tipo de plataformas se compone de una cadena de servicios que trabajan juntos para controlar de extremo a extremo el almacenamiento, procesamiento y la entrega de contenido.

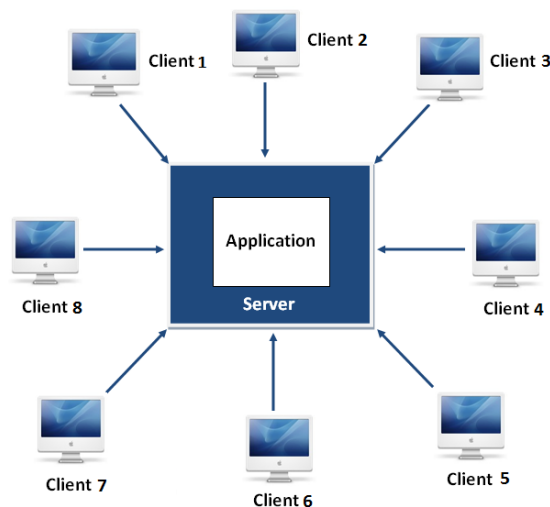


Figura 2.7: Diagrama de un modelo centralizado

Flujo de operación del procesamiento de video

El flujo comienza cuando un usuario sube un video a la plataforma. Se carga un archivo desde el *frontend* de la aplicación web o móvil hacia un servidor *backend*. A continuación, el servidor registra los metadatos del video (identificador, título, descripción, tags, visibilidad, duración, etc.) en una base de datos e intenta almacenar el video en un sistema de Object Storage.

⁴Se entiende por *backend* monolítico a aquella arquitectura software donde toda la aplicación se desarrolla y despliega como una única unidad.

Capítulo 2. Estado del Arte

Posteriormente, el archivo subido pasa a una cola de procesamiento, donde espera a ser atendido por un consumer para ejecutar tareas de transcodificación. Herramientas como FFmpeg juegan un papel fundamental en esta etapa permitiendo convertir el video a diferentes formatos, resoluciones y bitrates, y segmentarlo para adaptarlo a HLS o MPEG-DASH. En paralelo, se suelen generar miniaturas, extraer metadatos adicionales, crear subtítulos, entre otras tareas.

Una vez, el video fue procesado, se distribuyen los segmentos a través de un CDN. De este modo, el video se entrega de forma eficiente a los usuarios finales.

Con los metadatos del video recién creado, un algoritmo de recomendación se encarga de sugerir el video a usuarios que podrían estar interesados en este. No obstante, en una plataforma de videos convencional, el sistema de moderación se asegura de revisar que el contenido cumpla con las políticas de la plataforma antes de ser mostrado a usuarios finales.

Ventajas

La principal ventaja es su alto nivel de control. La organización responsable es la encargada de definir cómo se almacenan los datos, qué políticas de moderación se utilizan, cómo funciona el algoritmo de recomendación, cómo se optimiza el rendimiento, las políticas de seguridad, la monetización.

Asimismo, comparado con otros modelos, la arquitectura centralizada es más fácil de diseñar, implementar, escalar y mantener.

Desventajas

Uno de los puntos débiles de este modelo es su fuerte dependencia de la infraestructura centralizada, que suele crear "puntos únicos de fallo". Es decir, si el servidor central o la infraestructura de almacenamiento experimenta problemas, el servicio puede verse degradado.

Por otra parte, en servicios de streaming de video, una arquitectura de este tipo puede generar altos costos de infraestructura y mantenimiento, especialmente a medida que la plataforma crece y atrae a más usuarios. Esto es debido principalmente a que el almacenamiento, procesamiento y la distribución tienen un alto consumo de recursos.

Servicios que implementan este modelo

La mayor parte de los servicios de streaming y redes sociales utilizan este tipo de modelo, aunque cada una con sus propias variantes. Entre estos destacan YouTube, Rumble, Vimeo, Twitch, Instagram, Netflix, entre otros. También suele ser una opción popular para startups ya que es más fácil de implementar e integrar *frontend*, *backend*, procesamiento y distribución de video.

2.3.2. Modelo Federado

La arquitectura federada no suele ser un modelo común en servicios de streaming de video, aunque es una opción a considerar. Aquí, múltiples servidores independientes llamados nodos o instancias, operan de forma autónoma aunque pueden comunicarse entre ellos mediante protocolos. Al contrario del modelo centralizado, en este caso no existe una única entidad que controle todo el sistema. Cada nodo es responsable de gestionar sus propios usuarios, almacenamiento, procesamiento, base de datos y políticas de moderación. Sin embargo, esto no significa que los nodos estén aislados. Un usuario registrado en una instancia puede acceder y descubrir contenido que se encuentre en otro servidor autónomo si ambas instancias hablan el mismo protocolo de federación.

Desde un punto de vista técnico, cuando un usuario sube un video a una instancia, este se procesa y almacena en ese mismo nodo. Este, también se encarga de gestionar la autenticación, moderación de contenido y el procesamiento de video. Por lo tanto, se podría considerar a cada nodo como una pequeña arquitectura centralizada e independiente.

Lo especial de este modelo es su protocolo de comunicación o federación. Esto permite que un nodo anuncie de su existencia a otros y que intercambie información. Un protocolo común es **ActivityPub**, diseñado para redes sociales federadas y que forma el **Fediverse** [20].

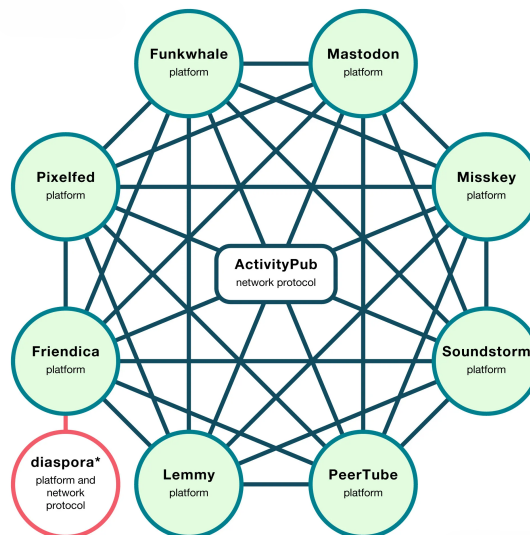


Figura 2.8: Diagrama de un modelo federado

Flujo de Operación del procesamiento de video

El flujo comienza cuando un usuario sube un video a una instancia de la red. Se carga un archivo desde el *frontend* de la aplicación hacia el *backend* de esa instancia. Esta se debió encargar de la autenticación del usuario primero y luego, el registro de los metadatos del video que se desea cargar. Además, esta misma capa se encarga de almacenar, procesar y entregar el contenido.

Capítulo 2. Estado del Arte

Una vez el video fue cargado, se sigue un flujo similar al del modelo centralizado, utilizando herramientas como FFmpeg y formatos de streaming como HLS o MPEG-DASH.

Una vez procesado el contenido, este queda disponible en la instancia donde fue subido. Gracias al protocolo de federación, este nodo anuncia sobre la existencia del nuevo video a otros nodos de la red **Fediverse**. Un protocolo usado con frecuencia es **ActivityPub**.

Cuando un usuario de otra instancia quiere acceder a ese video, se puede recuperar la referencia con el protocolo de federación y solicitar el contenido al nodo origen. Se pueden utilizar tecnologías como WebTorrent para la distribución del contenido o simplemente, crear un enlace al recurso original.

Ventajas

La ventaja más significativa es la distribución del control. No existe una única entidad que controle todo el sistema y que tenga poder absoluto sobre los datos, usuarios y contenido. Cada instancia es independiente y decide su gestión.

Otro aspecto importante a resaltar es la flexibilidad. Cada nodo puede adaptar el sistema a sus necesidades, utilizar sus propios criterios de moderación, decidir su arquitectura interna y mantener un mayor control sobre privacidad, gobernanza y gestión de datos.

Desventajas

Evidentemente, su principal desventaja es la complejidad técnica que añade. Además de gestionar el funcionamiento interno de cada instancia, se deben solucionar problemas de comunicación, sincronización y la interoperabilidad. Esto implica un esfuerzo de desarrollo adicional.

Los problemas de consistencia y disponibilidad también pueden ser un desafío. Si una instancia deja de responder, ciertos contenidos o interacciones pueden volverse inaccesibles.

Como cada instancia define sus políticas y recursos, la experiencia de usuario puede verse comprometida, a diferencia de un modelo centralizado.

Servicios que implementan este modelo

Este modelo no es común en servicios de streaming de videos. Sin embargo, existen aplicaciones que implementan este enfoque. El ejemplo más reconocido es *PeerTube*. En general, este enfoque suele ser adoptado por comunidades que priorizan descentralización, soberanía digital, independencia tecnológica y control local sobre la infraestructura.



Figura 2.9: Logo de *PeerTube*

2.3.3. Modelo Peer-to-Peer (P2P)

Esta arquitectura, aunque menos convencional, es otra alternativa para implementar una red social o un servicio de streaming de videos. Este es un modelo distribuido en el que cada cliente o nodo de la red puede actuar como consumidor o proveedor de contenido. Es distinto a la arquitectura centralizada en el modo de operación. La entrega de contenido se distribuye entre varias instancias independientes y no hay una dependencia con una organización que controle toda la infraestructura principal. Por otra parte, se diferencia del modelo federado debido a los protagonistas que forman parte de la red. En el federado, la red se compone de servidores independientes y autónomos que se comunican entre sí, mientras que en el P2P, son los propios pares de la red o clientes los que asumen un rol más activo.

En otras palabras en P2P, los usuarios comparten fragmentos de video o archivos directamente entre ellos. Esto contrasta con el modelo centralizado donde se recibe el contenido desde la infraestructura principal controlada por la organización y además, con el federado, donde el contenido depende del nodo que lo almacena.

Normalmente, los archivos de videos en este tipo de arquitecturas se dividen en bloques o fragmentos que pueden descargarse desde varios nodos a la vez. Es decir, partes de un mismo video pueden estar distribuidas en distintos nodos de la red.

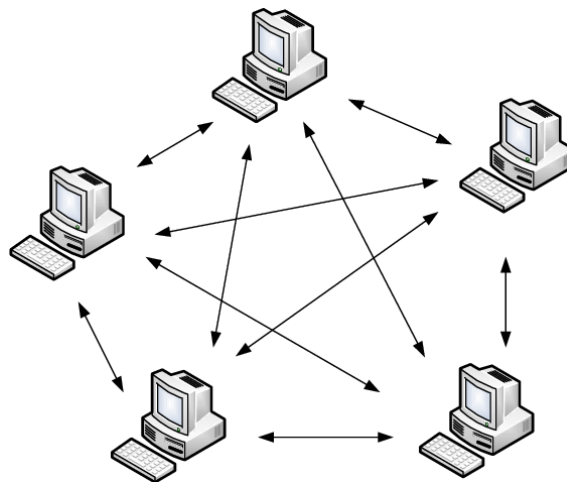


Figura 2.10: Diagrama de una red Peer-To-Peer

Flujo de procesamiento de video

El flujo comienza cuando un usuario publica un video dentro de la red. Dependiendo de las implementaciones, este archivo puede almacenarse en uno o varios nodos a la vez y dividirse en fragmentos para facilitar su distribución. A continuación, la red utiliza mecanismos de descubrimiento para que otros pares puedan localizar el contenido y los nodos que lo tienen.

Cuando un usuario intenta acceder al video, no siempre lo descarga desde una única fuente, ya que este puede estar fragmentado en varios nodos al mismo

tiempo. Por tanto, el cliente se encarga de reconstruir el contenido a medida que recibe la información. Esto claramente se diferencia de los dos métodos anteriores en donde se tiene un origen claramente identificable.

Las tecnologías más populares para este modelos son BitTorrent, WebTorrent e IPFS. Estas, intentan distribuir los datos entre los pares de la red (clientes) mientras que el federado lo hace entre instancias (servidores independientes y autónomos) de la red.

Ventajas

Se reduce la dependencia de una infraestructura central para distribuir el contenido, lo cual puede significar una disminución en costos e incrementar la tolerancia a fallos.

Además, este modelo tiene una escalabilidad elevada. Cuantos más clientes accedan a un mismo contenido, más pares lo tendrán y lo distribuirán. A diferencia del modelo centralizado, en el cual un aumento de la demanda incrementa la carga del servidor central, en esta arquitectura de P2P la carga se distribuye entre los nodos de la red.

Desventajas

La disponibilidad del contenido depende de la cantidad de pares conectados. Si pocos usuarios comparten un archivo, el acceso se volverá lento e inestable perjudicando la experiencia de usuario.

Asimismo, la moderación y eliminación de contenido resulta más compleja que en los otros modelos. En los otros casos, la moderación depende de las políticas establecidas por un servidor mientras que en este caso no.

Implementar una solución P2P añade más complejidad y esfuerzo de desarrollo que un modelo centralizado. Además del procesamiento, se deben sincronizar datos entre pares, distribuir fragmentos de archivos, recopilar información, y localizar el contenido distribuido por varias partes de la red, entre otros problemas de sincronización y comunicación.

Servicios que utilizan P2P

En el proceso de investigación no se han identificado servicios de streaming de videos que utilicen P2P puro. Sin embargo, una plataforma ampliamente reconocida que usa una base P2P es *Odysee*, una alternativa a YouTube descentralizada.



Figura 2.11: Logo de *Odysee*

Capítulo 3

Tecnologías Empleadas

Debido a la facilidad de implementación y a sus ventajas, se ha empleado una arquitectura centralizada para el desarrollo del proyecto según se describe en 2.3.1. No obstante, como posible ampliación se podrían explorar soluciones distribuidas.

En cuanto al procesamiento, transcodificación y entrega de video se experimentó con las tres alternativas mencionadas en 2.2. Para la solución propia se utilizaron los servicios de Amazon Web Service con las tecnologías descritas en 2.1 pero finalmente, se optó por integrar los servicios de un proveedor debido a los costos elevados de las otras alternativas.

Para la creación de un Producto Mínimo Viable (MVP), se ha decidido implementar una página web debido a su facilidad de acceso desde cualquier dispositivo y a la adecuación de los *frameworks* y tecnologías web a los objetivos del proyecto. Por ese motivo, a continuación se presentan las tecnologías utilizadas.

3.1. Backend

El backend es el conjunto de componentes que gestionan la lógica de negocio de una aplicación software, procesan datos, interactúan con la base de datos y administran la seguridad. Existe una amplia gama de tecnologías disponibles para seleccionar, especialmente en un entorno web. En esta sección, se analizarán las tecnologías preferidas y empleadas para la implementación de este servicio de streaming de videos en formato web.

3.1.1. TypeScript como lenguaje de programación

TypeScript es un lenguaje de programación desarrollado por Microsoft y es una extensión de JavaScript. Lo especial es que este lenguaje incorpora tipado estático y mejores herramientas para detectar errores durante el desarrollo y no en producción. Esto ayuda a estructurar y crear proyectos software de forma más efectiva, permitiendo un mantenimiento del código más sencillo.

Capítulo 3. Tecnologías Empleadas

En el caso del proyecto Booster, TypeScript resulta una herramienta útil debido a que el sistema integra múltiples capas de lógica como gestión de usuario, publicación y edición de metadatos de videos, autenticación, creación de comunidades y relación con la base de datos y el servidor. Por tanto, en este tipo de aplicaciones con un gran número de objetos de datos estructurados y bien definidos (usuario, videos, comunidades), el uso de tipado fuerte ayuda a reducir inconsistencias, detectar errores más fácilmente y mejorar la mantenibilidad del código.

Más técnicamente, a diferencia de JavaScript, en TypeScript se pueden definir estructuras complejas y detectar errores de forma más clara. A modo de ejemplo, se pueden declarar variables de la siguiente forma:

```
1 // con ':' se indica el tipo de la variable
2 //a diferencia de JavaScript donde solo se incluye un let.
3 saludos: string = "Hola";
4 contador: number = 42;
5 isFun: boolean = true;
6 notas: number[] = [10,10,9.8];
7
8 const user = {
9     firstName: "Samuel",
10    lastName: "Caraballo",
11    role: "Autor",
12 }
13
14
15 console.log(user.name) // Property 'name' does not exist on type
16                        // '{ firstName: string; lastName: string;
17                        //role: string; }'.
```

De esta forma se pueden detectar errores durante el desarrollo que, de otro modo, serían más complejos de identificar. Es común olvidarse del tipo de una variable e intentar hacer operaciones no permitidas para ese tipo. TypeScript lo detecta automáticamente, mientras que JavaScript no es consciente de esto y, en consecuencia, se generan errores difíciles de detectar.



Figura 3.1: Logo de TypeScript

Ventajas

Su principal ventaja es la detección temprana de errores. Esto es fundamental en proyectos con una base de código grande, donde un pequeño fallo en la estructura de datos puede provocar errores más grandes y arruinar el servicio.

Asimismo, el uso de TypeScript mejora la escalabilidad del proyecto permitiendo mantener un código más consistente y legible.

El uso de TypeScript es importante en la industria y por esa razón, tiene una gran integración con herramientas modernas de desarrollo de aplicaciones web como Next.js, Drizzle, tRPC, entre otros.

Desventajas

Emplear TypeScript, requiere destinar más tiempo a la definición de tipos, lo cual incrementa el tiempo de desarrollo porque requiere mantener precisión y consistencia en todo el proyecto. Por tanto, esto añade complejidad adicional en comparación con JavaScript.

Uso de TypeScript

TypeScript se ha consolidado como una de las tecnologías más utilizadas en el desarrollo web actual. Es el lenguaje por excelencia en la capa *backend* en plataformas SaaS, redes sociales y otras aplicaciones. La necesidad de compartir modelos de datos es un requerimiento frecuente en proyectos grandes que utilizan tecnologías modernas como React.js, Next.js, Node.js y Express.

En Booster, al ser un proyecto de código abierto, TypeScript se emplea como lenguaje principal de programación para la integración de las distintas tecnologías; garantizar precisión y consistencia; detectar errores tempranos; mejorar la mantenibilidad; y mantener la coherencia entre *backend* y *frontend*.

3.1.2. Base de Datos

La base de datos es uno de los componentes más importantes en las aplicaciones software. Esto se debe a que constituye el sistema encargado de almacenar, consultar y actualizar la información del servicio. Para los objetivos de Booster, la base de datos debe permitir realizar consultas complejas de forma eficiente, integrarse con tecnologías modernas y soportar funcionalidades avanzadas como embeddings para implementar la búsqueda semántica de videos.

Por estas razones, Booster integra **PostgreSQL** como sistema principal. Es un gestor de bases de datos relacionales ampliamente utilizado en aplicaciones de todo tipo. Su funcionamiento está basado en el modelo relacional, donde la información se ordena y almacena en tablas, filas y columnas. Las relaciones se modelan mediante claves primarias y foráneas. Este enfoque es el más adecuado para Booster, ya que la información de usuarios, videos y comunidades es claramente estructurada. Además, este gestor ofrece una gran cantidad de funcionalidades avanzadas, integridad referencial, alto rendimiento en producción y buena escalabilidad.



Figura 3.2: Logo de PostgreSQL

Uso de PostgreSQL - NeonDB

En Booster, PostgreSQL se integra a través de una plataforma de base de datos moderna y serverless diseñada para la nube conocida como **NeonDB**.

NeonDB presenta una arquitectura que separa almacenamiento y cómputo para ofrecer un escalado más flexible y administrar mejor la base de datos. Además, está especialmente diseñada para soportar funcionalidades de **Inteligencia Artificial**. En el caso de Booster, se han empleado extensiones como *pgvector* para almacenar embeddings y realizar búsquedas semánticas de los títulos y descripciones de los videos [21]. Esta capacidad, permite realizar búsquedas más inteligentes en comparación con las búsquedas literales. Con esto, se puede encontrar contenido relacionado por significado y mejorar el sistema de recomendación ante una búsqueda en lenguaje natural.



Figura 3.3: Logo de NeonDB

Ventajas

Implementar una base de datos PostgreSQL tiene diversas ventajas. Entre ellas, destaca la facilidad de administración. NeonDB ofrece un entorno gráfico para realizar consultas, ver los datos, tener un diagrama de la base de datos, entre otras funcionalidades que mejoran la gestión. Asimismo, PostgreSQL posee una documentación extensa y una gran compatibilidad con distintos ORMs y frameworks *backend*.

Desventajas

Aunque es muy versátil, cuando hay un volumen alto de datos se requiere un diseño cuidadoso y un mantenimiento constante, incrementando el esfuerzo de desarrollo y la complejidad. Con un flujo de datos grande, es necesario incluir índices y destinar más tiempo a realizar optimizaciones, consumiendo más recursos técnicos.

Supabase

De forma complementaria a NeonDB, se utiliza **Supabase**, otra plataforma de *backend* que implementa una base de datos PostgreSQL. Más concretamente, en Booster se hace uso de esta herramienta para implementar funcionalidades de chat en tiempo real. Supabase Realtime permite transmitir eventos mediante WebSockets y monitorizar cambios en la base de datos en tiempo real, haciéndolo ideal para implementar chats en los canales de usuario para que sus seguidores puedan interactuar entre ellos y con los creadores de contenido.



Figura 3.4: Logo de Supabase

3.1.3. ORMs

Los ORMs (Object-Relational Mappers) son herramientas que permiten la interacción con bases de datos relacionales haciendo uso de estructuras del lenguaje de programación, en el caso de este proyecto, TypeScript. En lugar de depender exclusivamente de consultas SQL manuales, se implementan funciones que actúan como capa de abstracción y además mejoran la seguridad. Por tanto, el programador puede definir relaciones y transacciones directamente desde el código de la aplicación sin trabajar con filas y tablas SQL en cada operación.

El uso de ORMs es muy común en aplicaciones modernas y existen un gran número de tecnologías a elegir. En este proyecto se han considerado dos: Drizzle y Prisma. Siendo la primera, la opción elegida para el desarrollo.

Drizzle

Drizzle ORM [22] es un ORM orientado al desarrollo con TypeScript caracterizado por ofrecer una experiencia similar a SQL y tipada.

Drizzle permite definir el esquema de la base de datos con código TypeScript mediante un archivo llamado `schema.ts`. A partir del mismo, un desarrollador es capaz de establecer relaciones entre tablas y definir los tipos de datos de las columnas para realizar consultas tipadas. Además, permite combinar consultas SQL en sus funciones, lo que lo hace útil para consultas complejas y con varios pasos.

La herramienta es ideal para el proyecto Booster. Esto se debe a que Drizzle permite realizar consultas avanzadas y específicas sobre bases de datos PostgreSQL.

Ventajas

Su principal ventaja, es su tipado fuerte. Junto a TypeScript, ayuda a detectar errores durante el desarrollo y mantener la coherencia y precisión entre el esquema de base de datos y el *backend*. A su vez, ayuda a prevenir inyecciones SQL mediante su capa de seguridad y mejores prácticas.

Por otra parte, según la documentación, tiene un diseño ligero y eficiente, lo cual es favorable para el rendimiento de la aplicación.

Desventajas

Drizzle es una herramienta nueva y además tiene una curva de aprendizaje elevada. Esto se debe a que requiere conocer bien el lenguaje SQL y tener experiencia con bases de datos relacionales antes de entender el funcionamiento del nivel de abstracción ofrecido por esta herramienta. Ofrece más control, pero demanda un conocimiento técnico y sólido.

Uso de Drizzle

En Booster se utiliza Drizzle como el ORM principal para interactuar con la base de datos. Una de las razones principales de esta elección es debido a que con un solo archivo *schema.ts* se puede definir el esquema de la base de datos. Esto facilita la creación de tablas y relaciones y permite mantener la coherencia en todo el proyecto. Además, otro punto fuerte es su compatibilidad con las tecnologías empleadas y su capa extra de seguridad incorporada.



Figura 3.5: Logo de Drizzle-ORM

Prisma

Prisma es otro ORM popular en el desarrollo de aplicaciones TypeScript. El desarrollador define un esquema propio a partir del cual se genera un cliente TypeScript que se emplea para interactuar con la base de datos. En consecuencia, se consiguen realizar consultas con un gran soporte de auto-completado y con una sintaxis orientada al programador. Prisma destaca por su facilidad para modelar la base de datos incluyendo relaciones, entidades y operaciones comunes, así como también, su experiencia de uso.

Es una buena opción cuando se necesita entregar un producto rápido y se busca legibilidad y tener una amplia variedad de herramientas adicionales. No obstante, para el proyecto se ha optado por Drizzle ya que esta tiene una relación más directa con SQL y es más ligera, mejorando así el rendimiento de la aplicación.

Esta alternativa se menciona como una herramienta importante que se podría haber incluido y se evaluó antes de comenzar el desarrollo pero, no forma parte del *backend* del proyecto final construido.

3.1.4. Comunicación Cliente Servidor - tRPC

La comunicación cliente-servidor es esencial dentro de cualquier aplicación web moderna. En la plataforma Booster, esto permite que el *frontend* se comunique con el *backend* para recuperar datos, actualizarlos y ejecutar funciones como registro de usuario e inicio de sesión. Existen diversos enfoques, siendo APIs Rest o GraphQL las más conocidas. Sin embargo, para los efectos del proyecto se ha optado por una tecnología llamada **tRPC**. Esta herramienta está orientada al desarrollo de RPCs (Remote Procedure Calls) con TypeScript, lo que permite construir procedimientos de comunicación entre cliente y servidor respetando los tipos y garantizando coherencia y consistencia.

tRPC es una de las tecnologías fundamentales en este proyecto facilitando el intercambio de datos entre cliente y servidor. A diferencia de un enfoque REST, con tRPC se definen procedimientos en el servidor y el cliente se encarga de consumirlos sin necesidad de especificar un esquema de endpoints o capas adicionales de generación de tipos. Además, la utilización de esta biblioteca mejora la seguridad y la coherencia de tipos de extremo a extremo. Es decir, un tipo definido en el *backend* se puede compartir fácilmente hacia el *frontend*, convirtiéndola en una herramienta ideal para el desarrollo *full-stack*.

Al trabajar con tRPC es conveniente distinguir los siguientes conceptos. [23]

- **Procedure:** un *endpoint* de una API. Representa una acción concreta clasificada en: consultas, mutaciones o suscripciones.
- **Query:** Un procedimiento que obtiene datos.
- **Mutation:** Un procedimiento que modifica datos (create, update, delete).
- **Suscripción:** Un procedimiento que crea una conexión persistente y escucha cambios.
- **Router:** Un conjunto de procedimientos agrupados por un nombre. Por ejemplo la acción de modificar un video (updateVideo) está dentro de un router de videos que contiene más operaciones relacionadas con esa entidad. Por lo general, se suele tener un router por cada entidad en la base de datos.
- **Contexto:** Todo aquello accesible por un procedimiento. Usado normalmente para sesiones de estado de usuarios o conexiones a bases de datos.
- **Middleware:** Una función que puede ejecutar código antes que se realice un procedimiento.
- **Validación:** Verifica que los datos de entrada a un procedimiento cumplen con ciertas normas y son correctos.



Figura 3.6: Logo de tRPC (typescript Remote Procedure Calls)

Lo particularmente interesante de tRPC es su integración con **TanStack Query** en el cliente lo cual añade mejoras de optimización, caché, revalidación, manejo de estados de carga y error, entre otras herramientas para mejorar la experiencia de uso. Esta es una de las principales razones por las que se emplea esta tecnología, además de su integración con typescript y otras tecnologías web.

prefetch con tRPC

Este es el concepto (o método) más poderoso que tiene tRPC. Gracias a él, la experiencia de usuario se ve mejorada y los tiempos de carga reducidos.

El *prefetch* consiste en cargar datos por adelantados antes de que el cliente los necesite. En la figura, se observa el flujo de esta operación implementada en el proyecto.

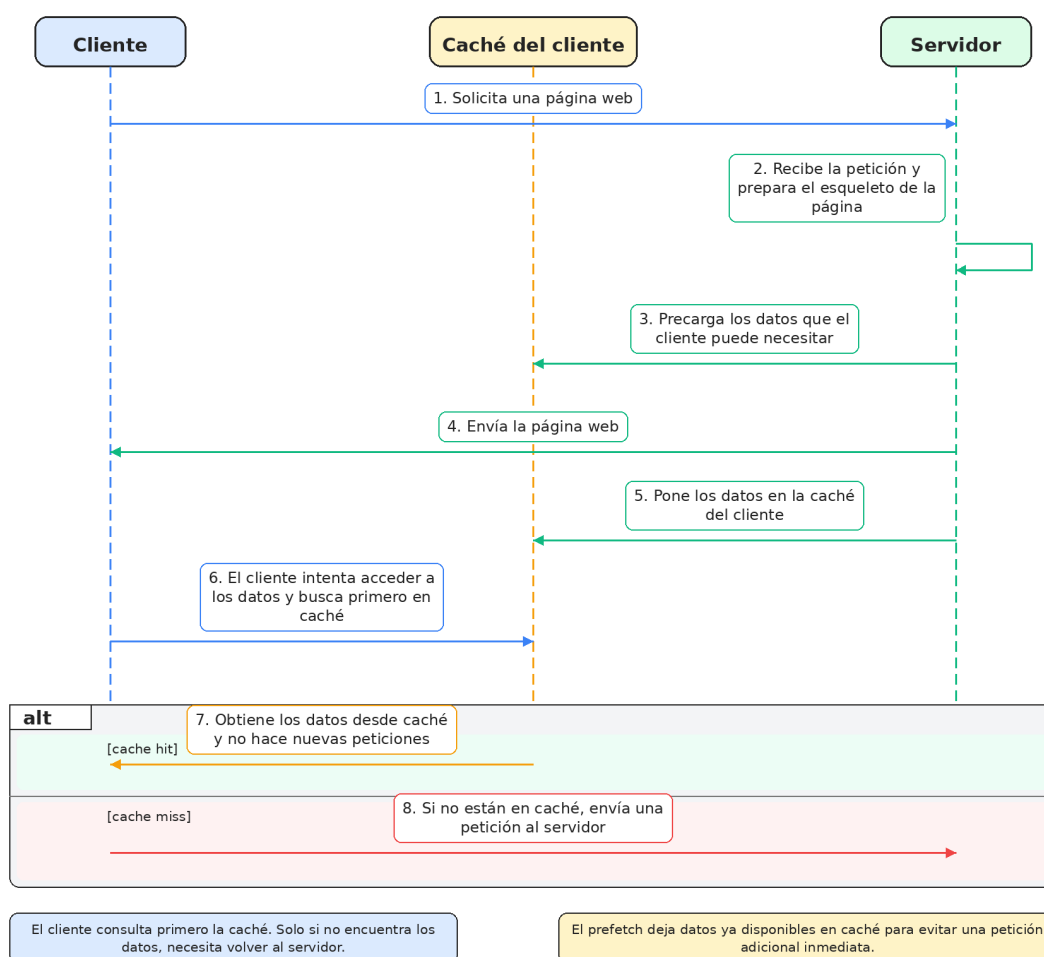


Figura 3.7: Diagrama de secuencia de un flujo de *prefetch*

Existen varias formas de aplicar el *prefetch*. En el desarrollo de esta plataforma se ejecuta una pre-carga en el servidor antes de renderizar la página para que los datos estén disponibles cuando llegue al cliente. En Booster, esta técnica tiene un gran valor. Los usuarios están constantemente navegando entre listados de

video, usuarios o comentarios por tanto, si los datos se cargan anticipadamente, el tiempo de espera disminuye creando la apariencia de una interfaz reactiva e inmediata. De esta forma tRPC tiene un doble uso: capa tipada de comunicación y un 'acelerador' de la interfaz de usuario. Por estas razones, se ha decidido utilizar tRPC para la comunicación cliente-servidor.

3.1.5. Next.js - backend

Next.js es, junto con tRPC, otra de las tecnologías fundamentales de este proyecto. Este es un framework de desarrollo web basado en 3.2.1 que permite construir aplicaciones *full-stack* modernas, gestionando la interfaz de usuario y la lógica de negocio dentro de un mismo repositorio. En Booster, Next.js se emplea para cumplir ambos de estos propósitos. Por un lado, se aprovechan sus habilidades de *frontend* pero también se hace uso de sus capacidades de backend, facilitando la experiencia de desarrollo.

Desde un punto de vista más técnico, para este proyecto se está empleando la versión moderna de Next.js que está basada en un **App Router**. Este es un sistema de enrutamiento basado en el directorio `/app`, en donde se gestionan las rutas de forma más eficiente. Es importante distinguir los siguientes conceptos:

- **Server Component:** Es un componente de 3.2.1 que se ejecuta y se renderiza **únicamente** en el servidor. Por tanto, puede acceder directamente a recursos del *backend*.
- **Client Component:** Es un componente que se ejecuta solamente en el cliente (navegador). Normalmente, se suele reservar para interacciones con la interfaz de usuario donde la disposición de la página web puede variar.

Durante el desarrollo de Booster se combinan las habilidades de los Server Components y los Client Components para mejorar la eficiencia y optimizar el rendimiento de la aplicación. Por otra parte, gracias al *App Router* [24] se logran crear manejadores personalizados de peticiones HTTP, rutas de APIs estáticas y dinámicas, y consultas. Todo esto se puede conseguir sin la necesidad de un servidor *backend* separado, lo cual facilita el desarrollo y ayuda a mejorar la coherencia en la aplicación.

En Booster, esto es extremadamente útil ya que permite concentrar en un mismo entorno diversas responsabilidades. Next.js se encarga de mostrar la aplicación, gestionar la obtención de datos, la comunicación cliente-servidor junto con tRPC y definir rutas de API y direcciones URL.

Además, Next.js contiene métodos de optimización como incorporación de cachés, revalidación, pre-carga de enlaces, optimizaciones de imágenes, optimizaciones de código CSS y optimizaciones de navegación.

Ventajas

Su principal ventaja es su integración *full-stack*. Se puede construir el cliente y el servidor en un mismo repositorio facilitando el proceso de desarrollo y despliegue. Además, emplea reglas, formatos y mejores prácticas que simplifican la

Capítulo 3. Tecnologías Empleadas

creación de aplicaciones web. Su integración con TypeScript y otras herramientas modernas lo convierte en una opción ideal para Booster.

Contiene una gran variedad de elementos de optimización para mejorar el rendimiento de la aplicación. En una plataforma como Booster, esto es importante ya que mejora la experiencia de usuario de forma significativa.

Otro aspecto importante a destacar es que, como Next.js utiliza pre-renderizado estático (Static Site Generation SSG) ¹, los motores de búsqueda pueden indexar más fácilmente el contenido de la página web. Así, se consigue mejorar su descubrimiento en internet e incrementar el tráfico: se mejora el Search Engine Optimization (SEO).

Desventajas

Al concentrar *frontend* y *backend* en un mismo lugar, la complejidad de los proyectos suele aumentar de forma considerable. Por el contrario, si se separa la lógica de negocio con la interfaz de usuario podría ser más fácil gestionar el proyecto por separado.

Otro aspecto importante a mencionar es su elevada curva de aprendizaje. Con los años se han añadido un gran número de operaciones y optimizaciones, incrementando la cantidad de conceptos y técnicas. Esto puede significar que dominar Next.js sea una tarea difícil.



Figura 3.8: Logo de Next.js

¹Se entiende por pre-renderizado estático (SSG) a aquel proceso de generación de páginas HTML en el momento de construcción (build time). Se ejecutan comandos como `npm run build` para generar archivos estáticos (.html, .css, .js) y de esta forma, estas páginas cargan de forma instantánea al no necesitar un servidor para solicitar datos

3.1.6. Autenticación - ClerkAuth

La autenticación de usuarios es también un proceso fundamental en cualquier aplicación software. Para este proyecto se han evaluado distintas formas de gestionarlo. Entre ellas destacan la utilización de librerías como **BetterAuth** y **NextAuth**. Sin embargo, para facilitar el desarrollo, se ha optado por incluir un proveedor de autenticación y gestión de usuario llamado **Clerk** [25].

Clerk es una plataforma de autenticación y gestión de usuarios que cuenta con funcionalidades como: registro de usuario e inicio de sesión con distintas opciones, controladores de estados de sesión, perfiles de usuario, modificación de fotos de perfil, un dashboard de administración, códigos de seguridad y *multi-factor authentication*. Es una solución completa y bien preparada para poner en marcha la autenticación en una aplicación software de forma rápida y con poco esfuerzo de desarrollo.



Figura 3.9: Logo de Clerk

Ventajas

Su principal ventaja es su facilidad de incorporación en proyectos software. Permite añadir una autenticación completa y profesional en poco tiempo y es ideal para proyectos construidos con Next.js.

Gracias a su panel de control, la experiencia de administración es buena y fácil de realizar. Clerk permite revisar usuarios, eliminarlos, gestionar sus datos y aplicarles sanciones. Por otro lado, también cuenta con una gran variedad de opciones distintas y admite personalizar los componentes de inicio o registro de sesión que vienen por defecto, ahorrando así tiempo de desarrollo.

En definitiva, Clerk incluye una amplia gama de funcionalidades y opciones lo que reduce el tiempo de desarrollo de forma significativa y evita tener que construir desde cero los procedimientos relacionados con la gestión de usuarios, sesiones y seguridad.

Desventajas

A pesar de todas las fortalezas de Clerk, su principal debilidad es su elevado costo. Aunque ofrece un plan básico gratis de hasta 10.000 MAU (Monthly Active Users), a partir de ese momento esta opción se vuelve cara. Por esa razón, suele ser utilizado para demostrar un MVP, donde no hay un número elevado de usuarios. De lo contrario, se vuelve inviable para equipos pequeños o con recursos limitados y otras opciones como *Firebase* son más razonables.

Uso de Clerk

En Booster, Clerk se emplea para ofrecer el servicio de autenticación. Esto es debido a su buena integración con el *stack* tecnológico seleccionado y por su

dashboard de administración que facilita la gestión de usuarios y configuraciones. Sin embargo, a medida que el proyecto escale puede resultar necesario migrar a otra alternativa para ahorrar costos.

3.2. Frontend

Se define al *Frontend* como la capa software encargada de la interacción directa con los usuarios. Esto puede incluir el diseño de la interfaz de usuario, botones, menús y navegación programados con HTML, CSS y JavaScript. Estos módulos, se suelen ejecutar en el lado del cliente, para el caso del proyecto, en el navegador.

En esta sección se presentan las tecnologías seleccionadas para diseñar esta capa software recopilando herramientas modernas y compatibles con lo seleccionado en la sección anterior.

3.2.1. React.Js

React.js es una biblioteca de JavaScript desarrollada por Facebook en 2013, utilizada en gran medida para la construcción de interfaces de usuario (UI) mediante componentes reutilizables. Esencialmente se basa en dividir la interfaz en pequeñas partes independientes, cada una con su lógica y representación visual y, por tanto, se crean páginas a partir de componentes separados en lugar de hacerlo todo al mismo tiempo. Además, estos elementos pueden ser reutilizados en otras partes según sea necesario. Por ejemplo, es común construir componentes para barras de navegación, botones o formularios, los cuales se combinan para formar la interfaz.

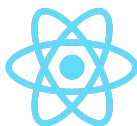


Figura 3.10: Logo de React.Js

Ventajas

La principal ventaja de React.Js es la reutilización de componentes. Esto ayuda a mantener una estructura escalable, limpia y fácil de mantener de la interfaz de usuario y el proyecto. Con esto, se logran abstraer las distintas secciones de la página web y la construcción del proyecto se vuelve más ágil y flexible.

Desventajas

React.Js no es una solución completa por sí misma. Esto quiere decir que necesita integrarse con otras herramientas para que sea útil en un proyecto.

Presenta una curva de aprendizaje significativa lo que puede resultar en un mayor tiempo de desarrollo para equipos inexpertos con esta tecnología. Además,

requiere el conocimiento y dominio de JavaScript, HTML, CSS, Node.js e incluso TypeScript.

Uso de React.Js

En Booster, se emplea React.Js para el desarrollo de las interfaces de usuario. Gracias a esta herramienta, se pueden reutilizar por toda la aplicación elementos como el reproductor de video, comentarios y barras de navegación mejorando así, la organización del código y su mantenibilidad.

3.2.2. Next.js - frontend

Next.js es una tecnología muy poderosa que ya se analizó en secciones anteriores (3.1.5). Aquí, se estudió su utilidad para tareas de *backend* pero también se integra bien con tareas de *frontend*.

Su funcionamiento consiste en apoyarse en componentes de React pero además, añade herramientas nuevas para el enrutamiento, generación de páginas, distintas formas de renderizado, más optimizaciones, entre otros. En cuanto al *frontend*, facilita algunas tareas y mejora la organización del proyecto ya que dispone de un sistema de rutas y páginas bien definidas con patrones claros y estructurados. Así, se consigue un código más legible, limpio y mantenible.

Booster utiliza Next.js en el frontend para organizar los componentes y la aplicación de manera profesional y escalable. Con esto se consigue gestionar la navegación entre vistas, mejorar el rendimiento general del sitio y su presencia en la web. Además, cuenta con una documentación extensa y clara. Otro aspecto positivo es que se integra bien con una amplia gama de tecnologías diferentes, haciendo que el proyecto tenga mayor flexibilidad.

3.2.3. TailwindCSS y Shaden

Normalmente, las aplicaciones web suelen mejorar su interfaz de usuario mediante CSS (Cascading Style Sheets), un lenguaje de diseño gráfico para definir la apariencia y el estilo de los documentos HTML. Sin embargo, para este proyecto se ha optado por utilizar **TailwindCSS**, una herramienta más moderna basado en utilidades de CSS para aplicar estilos a componentes HTML.

Su funcionamiento consiste en aplicar directamente en un atributo `class` o `className` en HTML propiedades CSS (*utility classes*) predefinidas que representan características visuales específicas. Por ejemplo, el siguiente código cambia el color del texto a rojo:

```
1   ...  
2   <p class="text-red-200">Texto rojo. </p>  
3   ...
```

Ventajas

La principal fortaleza de Tailwind es la rapidez con la que se pueden construir interfaces. Además, las utilidades están bien definidas y con ellas se consiguen

desarrollar interfaces modernas, coherentes y visualmente atractivas.

A su vez, como los estilos se aplican directamente como atributo de los componentes HTML, esto evita tener archivos de estilos específicos para distintas partes de la aplicación. Esto mejora la organización del código en el proyecto.

Con este framework es posible además mantener una paleta de colores consistente con la marca a lo largo de toda la aplicación de forma clara y sencilla. Con estas clases también se puede construir una aplicación visualmente responsiva para distintos tamaños de dispositivo, mejorando así la experiencia de usuario en diversas plataformas.

Desventajas

TailwindCSS suele ser una gran opción para el desarrollo moderno de interfaces. No obstante, puede hacer que el código de los componentes HTML esté más sobrecargado. Como las clases se escriben directamente en los elementos HTML, es común tener una gran variedad de atributos distintos para un único elemento. Esto puede dificultar la lectura y la comprensión de la apariencia del componente.

Uso de TailwindCSS

Booster hace uso de las capacidades de TailwindCSS para el diseño de la plataforma. Se utiliza principalmente para mejorar y modernizar la apariencia visual de los elementos en pantalla. Asimismo, se definen colores, tamaños, tipografías y comportamientos responsivos.

Complementariamente a Tailwind, Booster incorpora **shadcn/ui** [26], un sistema de componentes reutilizables y personalizables como botones, barras laterales, avatares, menús, diálogos, toasts, formularios, tablas, entre otros. Gracias a esto, se utilizan elementos de interfaz ya elaborados y con una apariencia muy moderna y atractiva. Esto permite, ofrecer una experiencia de usuario más profesional y mejora la **usabilidad** de la aplicación. El uso de esta librería es importante en Booster ya que se consigue una interfaz limpia, moderna y funcional sin dedicar demasiado tiempo al desarrollo de cada componente desde cero.



Figura 3.11: Logotipos de TailwindCSS y Shadcn/ui

Capítulo 4

Innovaciones

En este capítulo se analizan con más detalle los problemas más significativos que tienen las plataformas de streaming de videos actuales y las soluciones que plantea Booster. En particular, se examinan las dificultades que experimentan algunos usuarios para encontrar comunidades de videos destinadas a intereses específicos; la dependencia de los algoritmos de recomendación que suelen premiar contenido para maximizar la retención; y la gran cantidad de anuncios que arruinan la experiencia de los usuarios.

Asimismo, se explican algunas de las mejoras y nuevas incorporaciones a este tipo de aplicaciones, tomando en cuenta nuevos enfoques de moderación, la interacción en la plataforma e incorporando gamificación.

4.1. Problemas detectados y la solución de Booster

En esta sección se detallan algunos de los problemas más relevantes que presentan las plataformas de streaming de videos actuales. Se analizan y se proponen medidas que implementa Booster para solventar estos inconvenientes. Los problemas más notorios están relacionados con los algoritmos de recomendación, las sugerencias de videos y la gran cantidad de anuncios mostrados. Booster intenta darle un nuevo enfoque a las plataformas de este estilo para mejorar la experiencia de los usuarios.

4.1.1. Problemas con el algoritmo de recomendación

Uno de los problemas más grandes que presentan las plataformas de videos actuales, y motivo de discusiones y quejas por parte de los usuarios, es el funcionamiento de los algoritmos de recomendación. En muchos servicios, estos están diseñados para priorizar el contenido con mayor capacidad para generar clics y tiempo de retención. Esto es objeto de debate porque puede favorecer la difusión de contenido adictivo o de baja calidad y ocultar otros que merecen más reconocimiento.

En 2024, la Comisión Europea [27] mencionó que las plataformas deben evaluar

y mitigar los riesgos que surgen de sus sistemas de recomendación, incluyendo riesgos para la salud mental y la difusión de contenido dañino debido al diseño de los algoritmos que promueven la retención. Además, indica que muchas plataformas usan estas técnicas sin ofrecer controles claros a los usuarios y que se puede priorizar el contenido perjudicial, generando efectos negativos especialmente en menores.

Desde el punto de vista del usuario, esto tiene implicaciones negativas. En primer lugar, puede encerrar al usuario en un bucle de consumo, limitando así el descubrimiento de nuevos temas. Por otra parte, reduce la transparencia dado que no se demuestra por qué se recomienda un tipo de contenido y no otro.

Desde el punto de vista del creador de contenido, esto también puede generar varios problemas. Los creadores nuevos y pequeños son los más afectados por el algoritmo de recomendación, ya que el contenido más popular o con mayores niveles de retención suele ser el que se muestra a los usuarios preferentemente.

Soluciones

Booster plantea un modelo de recomendación más simple, transparente y comprensible. Para abordar estos problemas se introduce un nuevo concepto en la plataforma denominado **puntos de Boost** para darle otro enfoque a las sugerencias de contenido. Los **puntos de Boost** permiten reflejar la valoración y el apoyo que reciben los videos de un creador de contenido. Esencialmente, este es el mecanismo con el que los usuarios apoyan activamente a creadores, canales y comunidades dentro de Booster. A diferencia de otros métodos que se basan en variables difíciles de interpretar, esta mejora permite que la participación de los espectadores tenga un rol más importante en la distribución de contenido. Estos puntos, determinados por los usuarios que han contribuido a un creador, definen su **Nivel de canal**. A medida que sube, se desbloquean beneficios para los seguidores y además, los videos de esos canales serán recomendados con mayor frecuencia en la plataforma y de esta forma, el usuario tiene un rol más importante en el descubrimiento de contenido.

Además de los puntos de Boost y el nivel de canal, el algoritmo también tiene en cuenta otros factores como visualizaciones, valoraciones promedio del video según los usuarios, número de comentarios, categorías del video, antigüedad y si el usuario es seguidor del canal. Todo esto se modela mediante una fórmula matemática que se adapta al espectador y que se publica en la plataforma. En capítulos posteriores se analizará más en detalle su funcionamiento 6.5. De esta forma se consigue una mayor transparencia en los algoritmos de recomendación y se premia el contenido favorito del usuario.

4.1.2. Problemas en encontrar comunidades

Otro problema importante y ligado al anterior es la dificultad para encontrar comunidades específicas. Aunque la mayoría de estos sistemas permiten descubrir contenido mediante búsquedas o sugerencias, no siempre facilitan la localización de espacios centrados en intereses concretos. En muchos casos, estos videos aparecen dispersos y no existe una estructura comunitaria clara.

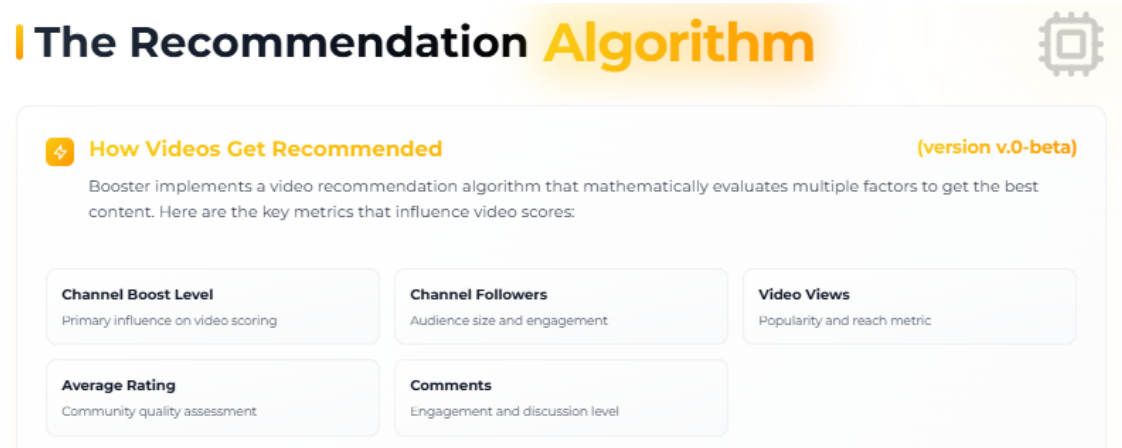


Figura 4.1: Fragmento de la plataforma donde se explica el funcionamiento del algoritmo de recomendación

Este problema es significativo para creadores pequeños, comunidades emergentes o áreas de interés muy específicas, porque los algoritmos de recomendación priorizan el contenido con mayor facilidad para atraer a un gran número de audiencia. En consecuencia, ciertos usuarios pueden verse en la dificultad para encontrar contenido de nicho en la plataforma. Igualmente, los creadores también sufren, ya que les puede resultar más complejo encontrar a su público objetivo.

Soluciones

Booster propone una solución a este problema mediante la incorporación de un **sistema de comunidades**. De esta manera, cualquier usuario es libre de crear su propia comunidad en torno a un tema o interés concreto, generando así un área dedicada a este tipo de contenido. Otros usuarios pueden enlazar videos relacionados a la comunidad, agrupando y organizando el material audiovisual, lo cual facilita su descubrimiento. El creador de la comunidad tiene la habilidad de eliminar contenido que no esté relacionado y dotar a otros usuarios de esta capacidad, convirtiéndolos en moderadores del contenido en la comunidad.

Discover Communities

Find your people. Explore communities that match your interests.

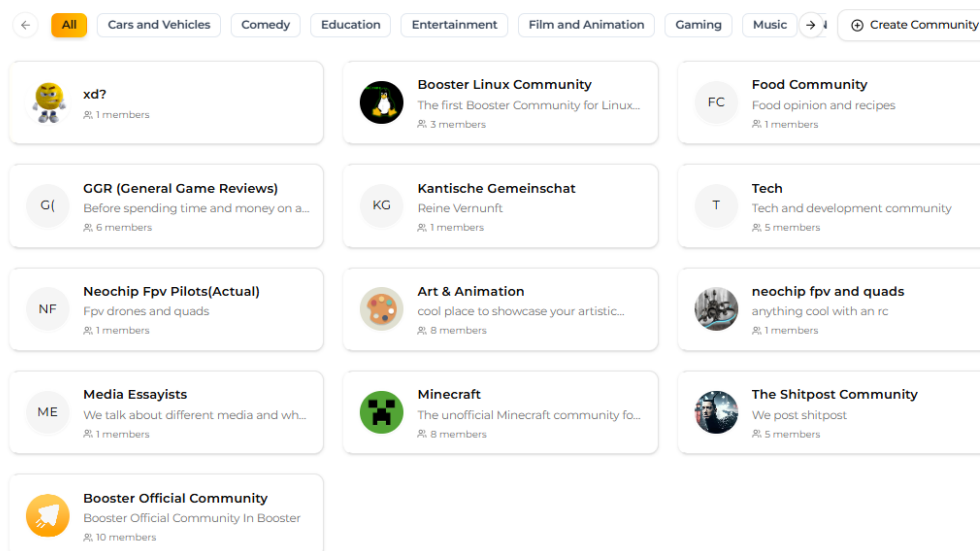


Figura 4.2: Sección de comunidades en la plataforma

Esta innovación añade una nueva forma de organizar los videos en la plataforma según un tema específico. En lugar de depender únicamente del algoritmo de recomendación, Booster permite estructurar el contenido por temas y comunidades. Por consiguiente, se mejora el acceso a algunos videos y se reúnen a usuarios con intereses comunes.

4.1.3. Problemas con los anuncios

Uno de los problemas más grandes desde el punto de vista de los consumidores de contenido es la gran cantidad de anuncios con los que se ven bombardeados cada vez que utilizan la aplicación. Un estudio publicado por *Raman Media Network* [28] señala que en YouTube, la sobre-exposición de anuncios aumenta la fatiga de los usuarios y daña significativamente la confianza en la empresa. Aquí se demuestra que los anuncios repetitivos causan descontento en los consumidores. Según se indica, ver un anuncio repetido 6 veces en una hora produce un 92 % de reconocimiento de la marca y un aumento del 48 % en sentimientos de molestia. En general, aunque los usuarios aceptan ver anuncios a cambio de un costo menor, la frecuencia excesiva de los mismos incrementa las sensaciones negativas hacia la plataforma e incluso el abandono y el uso de bloqueadores de anuncios.

Como se puede observar, el problema no solo reside en la existencia de la publicidad, sino que también, en la poca capacidad de elección que presenta el espectador aparte de su carácter intrusivo y forzado. Esto provoca una experiencia menos fluida y degrada la calidad del servicio percibida por los usuarios derivando en tasas de abandono o en la utilización de bloqueadores de anuncios.

Soluciones

Aunque los anuncios no se pueden eliminar, ya que son la fuente de ingresos principal de una plataforma de este estilo, Booster plantea una alternativa para mejorar la experiencia de los espectadores. Se trata de la incorporación de un sistema de anuncios voluntarios con recompensa. En lugar de basarse únicamente en anuncios forzados, la plataforma tiene un mecanismo por el cual los usuarios pueden decidir ver determinados anuncios de forma opcional y recibir un premio dentro de la plataforma.

Esto se conecta y da lugar al sistema de **puntos de experiencia (XP)** de la plataforma. La **XP** es una moneda virtual que se emplea en la plataforma y que cada usuario puede obtener mediante la visualización de anuncios voluntarios o integrados. Después, la puede utilizar para adquirir elementos de personalización, apoyar a creadores y comunidades, obtener recompensas, adquirir preferencias de uso, entre otros. Por tanto, la solución consiste en convertir la experiencia forzada e intrusiva de los anuncios en un sistema más participativo, donde el usuario recibe una compensación que puede intercambiar a cambio de premios dentro de la plataforma.

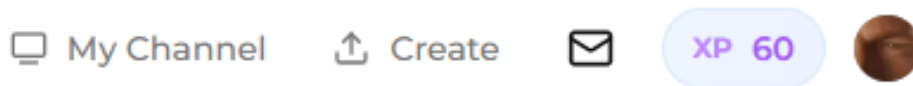


Figura 4.3: XP de un Usuario

4.2. Mejoras e Innovaciones

En esta sección se presentan las innovaciones y mejoras que incorpora Booster con respecto a las plataformas de videos actuales. Se detallan algunas de las decisiones más importantes tomadas para diferenciar Booster de otras alternativas y mejorar la experiencia de usuario.

4.2.1. Mejoras en la interfaz de usuario

Uno de los factores diferenciales de Booster es su interfaz moderna y consistente. Como se estudió en capítulos anteriores se están empleando tecnologías de interfaz de usuario muy completas y con una capacidad para construir apariencias atractivas en la aplicación. Booster se divide por distintas secciones y en cada una de ellas se utilizan técnicas y componentes claros para mejorar la experiencia de usuario.

Página principal

Un ejemplo claro de esto es la página principal. Aquí, se muestran videos en formato de cuadrícula, similar a otras plataformas. Sin embargo, también se emplea un componente para modificar la estructura. Con este, se puede filtrar por videos sugeridos por el algoritmo de recomendación, acceder a los videos más

Capítulo 4. Innovaciones

recientes añadidos a una comunidad y ver el contenido más actual de creadores de contenido específicos.

Por otra parte, el usuario también tiene un mayor grado de libertad al elegir la disposición de la interfaz. Por el momento, se ha implementado una función que ayuda a filtrar videos verticales y, por tanto, estos no aparecerán en la página principal. Además, los anuncios voluntarios que se encuentran en la primera fila de la cuadrícula pueden ser ocultos mediante los ajustes de usuario.

En la página principal se hace uso de un *Carousel* para filtrar con mayor facilidad entre las distintas categorías de video más comunes. Cuando hay un elemento seleccionado, este se ve claramente reflejado mediante un botón llamativo con los colores de la plataforma.

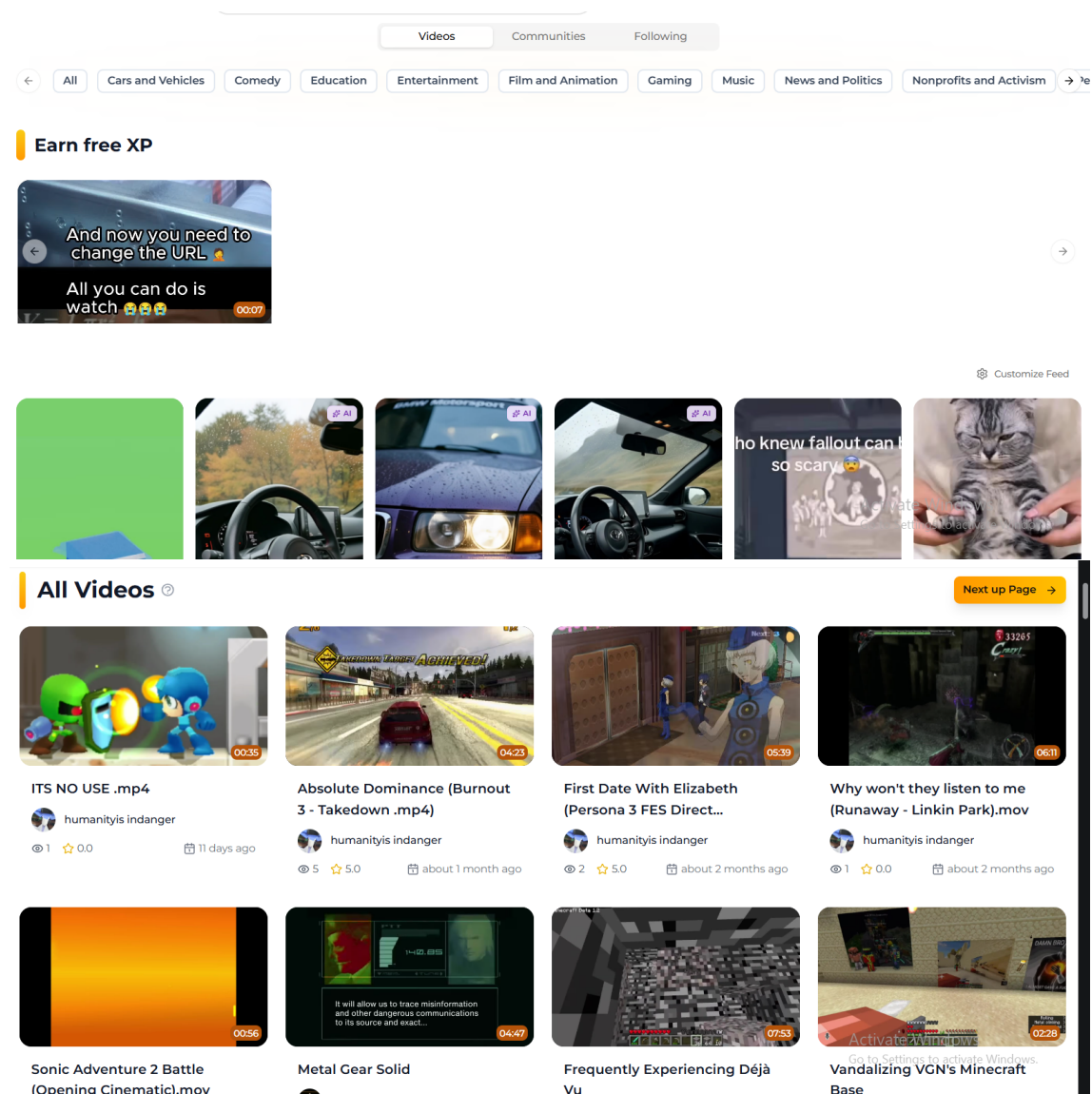


Figura 4.4: Página principal de Booster

Los videos mostrados en la cuadrícula presentan una mejor descripción. Cada

video es un rectángulo con esquinas redondeadas y con una etiqueta superpuesta que indica la duración. Además, al arrastrar el ratón sobre un video, se empieza a reproducir un gif del mismo y se observan estadísticas de popularidad sobre él como las visitas y la calificación promedio.¹ Estos detalles, junto con el título, la antigüedad y el creador, se pueden observar debajo del video.

Además, otra mejora visual con respecto a otras plataformas es la inclusión de una etiqueta 'AI' para aquellos videos que hayan sido creados mediante herramientas de inteligencia artificial.

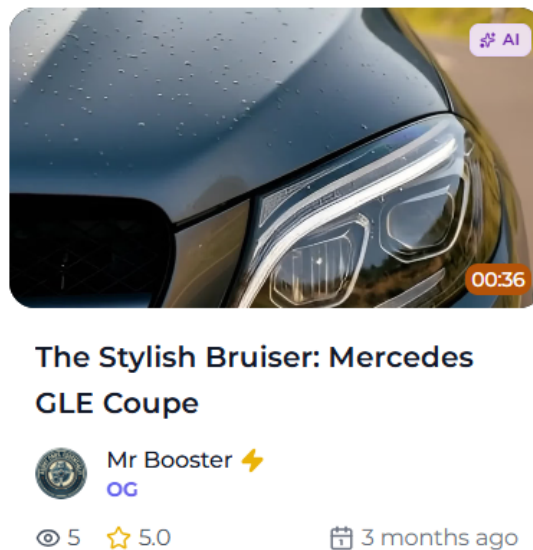


Figura 4.5: Video marcado con etiqueta IA

Es importante resaltar que esta sección es responsiva, es decir, se adapta la interfaz de usuario según el tamaño de la pantalla del usuario para evitar arruinar componentes y la estructura de la página en otros escenarios.

Vista de Videos

En otra sección donde se han incorporado numerosas mejoras ha sido en la vista de videos. Cuando un usuario mira un video, este aparece en modo ventana a la izquierda del monitor y ocupando aproximadamente $\frac{2}{3}$ de la anchura de la página en ese momento. El tercio restante, se aprovecha para incluir información adicional o relacionada. Las plataformas de video actuales suelen incorporar todos estos datos de forma desordenada y poco compacta. Booster intenta organizar las distintas partes, consiguiendo así una interfaz más limpia y ordenada como se puede observar en la figura.

A la derecha de un video aparece en primer lugar el creador del mismo. Adicionalmente, se incluye información sobre el nivel del canal del usuario y los puntos de Boost que este tiene. Esto viene acompañado de un componente de barra de progreso para comunicar de forma más visual esta información. Abajo de este bloque comienza una sección donde el usuario puede filtrar por:

¹Para Booster, se ha optado por cambiar los métodos de calificación de videos como el 'me gusta' por una calificación entre 1 y 5.

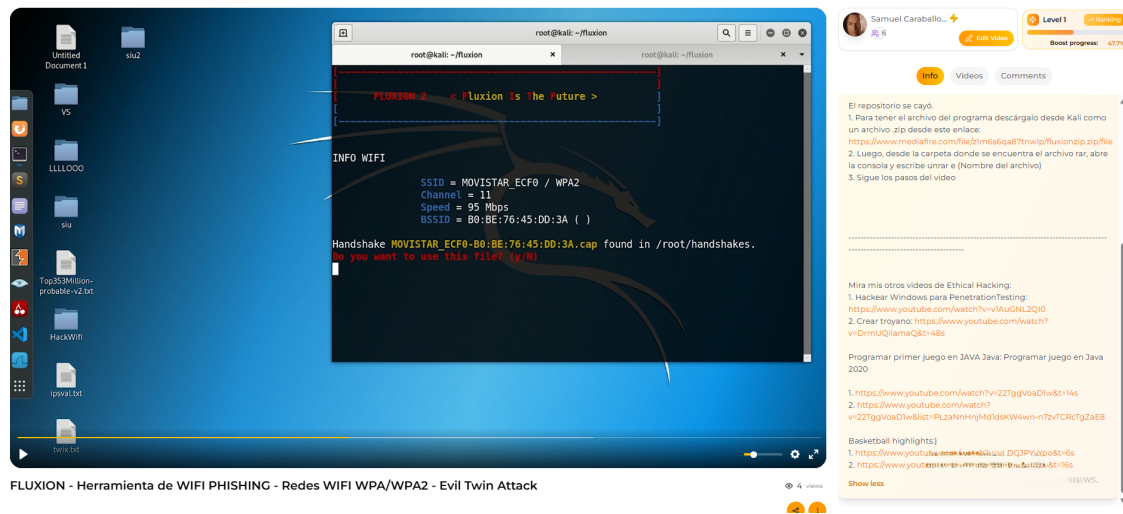


Figura 4.6: Vista de Video - Descripción

- **Más Información:** Se muestran más detalles sobre el video como la descripción (la cual se puede colapsar), la fecha de carga del video y su popularidad.
- **Videos:** En esta sección aparecen videos similares al que ya se está reproduciendo. El algoritmo de recomendación toma en cuenta los datos del video actual y le muestra al usuario los mejores candidatos para ver a continuación.
- **Comentarios:** Se acceden a la sección de comentarios donde se pueden crear, ver y responder.

Barra lateral

La barra lateral también presenta mejoras. Esta es visible en todas las rutas dentro de la aplicación. Sin embargo, en la vista de videos esta desaparece. Para volver a mostrarla, es necesario presionar un botón y aparecerá en modo flotante.

Además, se divide en varias secciones y permite navegar por toda la aplicación de forma rápida y accesible. Los primeros 6 componentes corresponden a las rutas más importantes dentro de la plataforma incluyendo:

- **Next Up:** Vista de videos rápidos donde el usuario puede arrastrar su cursor para pasar al siguiente.
- **Explorer:** Página principal de la aplicación donde los videos se muestran en formato de cuadrícula. Se pueden filtrar, buscar y acceder a la sección de videos recientes en comunidades y creadores.
- **Following:** Lista de los canales a los que el usuario sigue
- **Market:** Espacio reservado para la compra de puntos de experiencia, compra de artículos, recepción de premios, desbloqueo de *assets*, entre otros ítems de personalización.
- **Communities:** Zona destinada a la creación y búsqueda de comunidades.
- **Top Channels:** Clasificación de los mejores canales en la plataforma. Se puede filtrar por puntos de experiencia o por cantidad de seguidores.

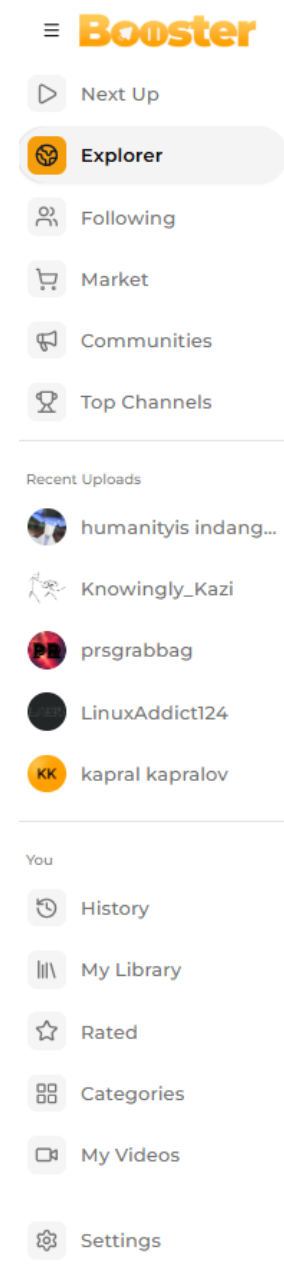


Figura 4.7: Barra lateral de Booster

Estos 6 componentes determinan el flujo habitual de uso de la aplicación. En la parte inferior, hay más secciones, pero estas son dependientes del usuario. Aquí, se muestran 5 canales que el usuario esté siguiendo y con cargas más recientes de video. De esta forma, se tiene un acceso rápido a las nuevas actualizaciones.

Finalmente, se incluyen apartados destinados a explicar el funcionamiento de

la plataforma así como las reglas y las pautas de la comunidad. Asimismo, se ha implementado un apartado que permite realizar ajustes de usuario sobre el algoritmo de recomendación. Posteriormente, se ampliarán las opciones de tal forma que mediante esta sección, se cambien más opciones.

Como se observa en la figura, la barra lateral tiene íconos para mejorar la usabilidad y además el apartado seleccionado tiene señales visuales de activación con la paleta de color de la marca de la plataforma y con esquinas redondeadas.

Temas de interfaz

En *Booster* la personalización es un componente importante. Por esa razón, se permite al usuario seleccionar una apariencia de interfaz que desee y ajustar la aplicación a sus gustos. Por el momento, la plataforma cuenta con 3 tipos distintos de temas de usuario. Estos son el tema claro, el tema oscuro y el tema "retro". Con tecnologías web como Tailwind CSS, esta clase de personalización puede implementarse de forma estructurada mediante clases utilitarias como `dark:` o `retro:`, lo que facilita la construcción de interfaces consistentes, mantenibles y adaptables, mejorando así la experiencia de desarrollo.

Estos ajustes se pueden modificar desde el canal de usuario, presionando sobre la opción de Personalizar Canal y dirigiéndose a la opción de apariencia de la aplicación, como se observa en la figura 4.8.

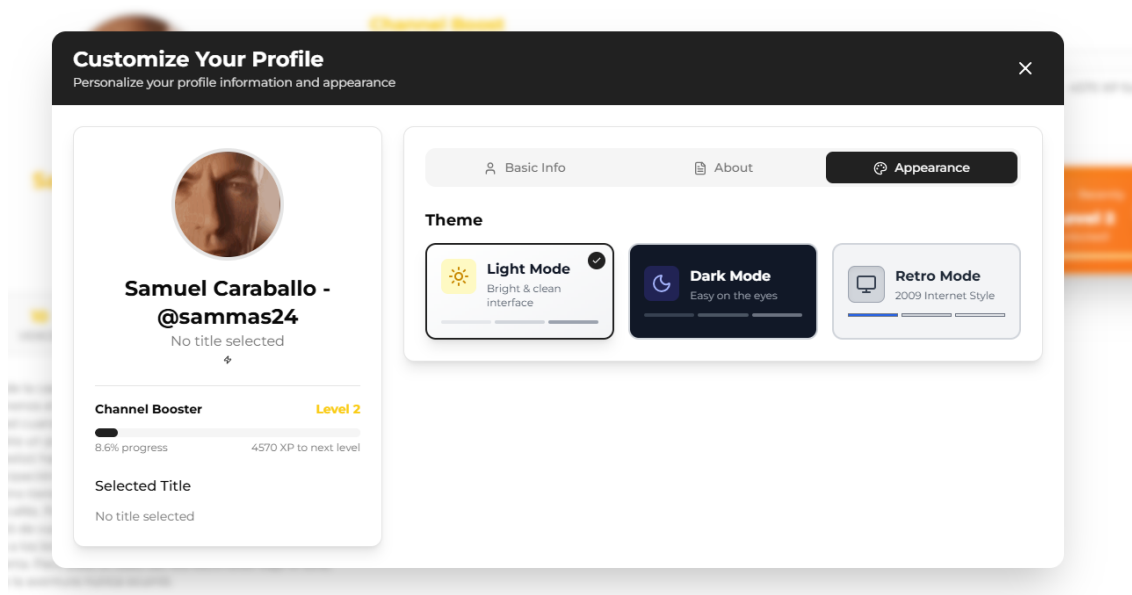


Figura 4.8: Seleccionador de temas de interfaz

El modo negro es el tema principal de la aplicación. Esta decisión se debe a que una parte importante de los usuarios muestra una preferencia por las interfaces oscuras. Esto lo evidencia un estudio académico el cual reportó que el 79.7% de los participantes prefería el modo oscuro en sus teléfonos móviles [29]. Adicionalmente, las guías oficiales de diseño de Android indican que las interfaces oscuras mejoran la visibilidad para personas con sensibilidad a la luz y facilitan el uso en entornos con baja luminosidad [30].

4.2. Mejoras e Innovaciones



(a) Vista de Videos - Tema oscuro



(b) Vista de Videos - Tema claro



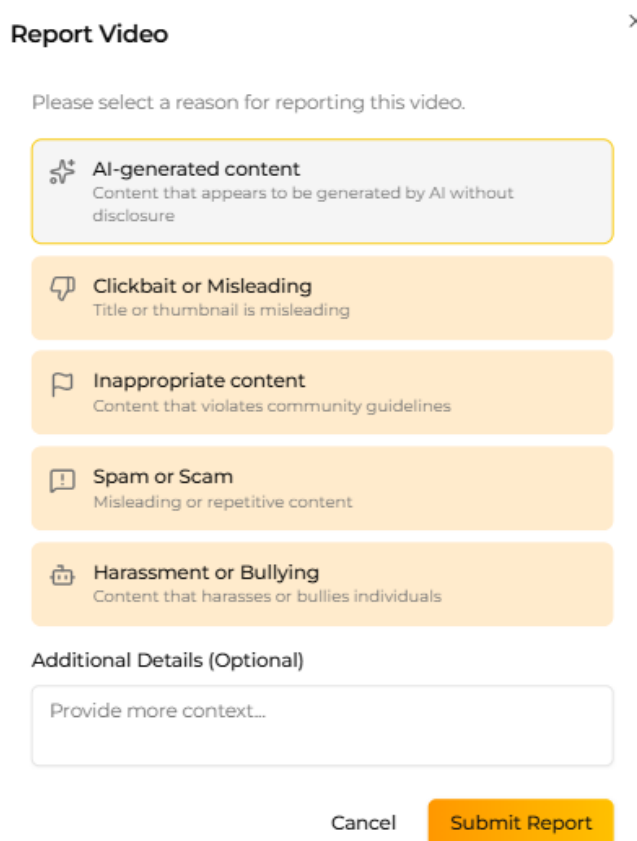
(c) Vista de Videos - Tema retro

Figura 4.9: Variantes de temas de interfaz en la vista de videos

El tema oscuro es el principal en *Booster*, aunque también se permite cambiar a un tema claro. El tema retro se puede adquirir con puntos de experiencia a través de la sección de *marketplace*.

4.2.2. Mejoras en los métodos de moderación

En *Booster* también se han hecho mejoras en los métodos de moderación. A diferencia de las plataformas de streaming convencionales, donde el control del contenido lo hace la propia aplicación, aquí se permite a los usuarios reportar videos y participar en votaciones de moderación. Cuando un espectador considera que el video es inapropiado o infringe alguno de las políticas establecidas por la aplicación, puede seleccionar una opción debajo de cada video para reportar el contenido. Una vez se presiona este botón, se crea un *modal* en donde se debe seleccionar la razón que el usuarios considera que el video incumple, como se puede ver en la figura.



The image shows a 'Report Video' modal window. At the top, it says 'Report Video' with a close button (X). Below this, it asks 'Please select a reason for reporting this video.' There are five options, each with an icon and a description: 1. 'AI-generated content' (robot icon) - 'Content that appears to be generated by AI without disclosure'. 2. 'Clickbait or Misleading' (speech bubble icon) - 'Title or thumbnail is misleading'. 3. 'Inappropriate content' (flag icon) - 'Content that violates community guidelines'. 4. 'Spam or Scam' (warning sign icon) - 'Misleading or repetitive content'. 5. 'Harassment or Bullying' (two people icon) - 'Content that harasses or bullies individuals'. Below these options is a section titled 'Additional Details (Optional)' with a text input field labeled 'Provide more context...'. At the bottom right, there are two buttons: 'Cancel' and 'Submit Report'.

Figura 4.10: Ventana flotante con opciones para reportar un video

A continuación, después de que el usuario haya seleccionado el motivo por el que el video infringe una norma, junto con una descripción opcional, se crea un evento de votación debajo de la descripción del video, como se observa en la figura, y queda registrado en el sistema. Esto es visible para todos los usuarios que ingresan al video y se les permite votar si están de acuerdo. Particularmente,

4.2. Mejoras e Innovaciones

si un usuario observa que el video fue denunciado por contenido inapropiado, puede dar su opinión al respecto y votar a favor o en contra.

Cuando una gran mayoría de usuarios están de acuerdo en una misma decisión, automáticamente se toma acción en algunos casos. En otros, se informa a los administradores del reporte y de esta forma, se puede tomar acción inmediata con videos que rompen las normas de comunidad.

Por tanto, este nuevo sistema de moderación tiene una doble función. Por un lado, constituye un mecanismo más justo para controlar el contenido de la aplicación; por otro, dado el carácter público de las votaciones, cumple el propósito de informar al usuario de aquel contenido que, posiblemente incumple las reglas de comunidad.

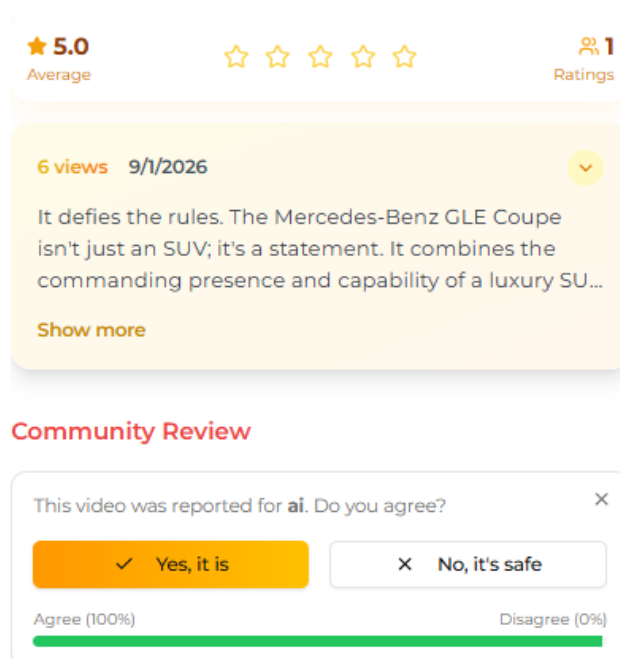


Figura 4.11: Votación de incumplimiento

En esta figura, se muestra un reporte sobre contenido hecho con IA para el video actual. Algún espectador anterior creó esta votación y según se observa, el usuario actual ha votado a favor. Como la gran mayoría de usuarios está de acuerdo en una misma decisión, en este caso, se toma acción inmediata. Esto implica, aplicarle una etiqueta *AI generated* al video. Esto revela de forma explícita, que fue creado en su mayor parte con herramientas de inteligencia artificial. En otros escenarios, es necesario que los administradores del sistema hagan una revisión manual para aplicarle las sanciones al video o al creador de

contenido.

4.2.3. Mejoras en los comentarios

Los comentarios de un video, también presentan mejoras visuales y estructurales. A diferencia de la gran mayoría de plataformas de streaming, los comentarios en Booster están organizados en formato de hilo o árbol de respuestas similar a aplicaciones como *Reddit*. Asimismo, el diseño de los mismos es moderno y atractivo debido a las tecnologías web modernas de diseño empleadas como *Tailwindcss*. Además, se incluyen iconos de *Lucide-React-Icons* para mejorar la usabilidad.

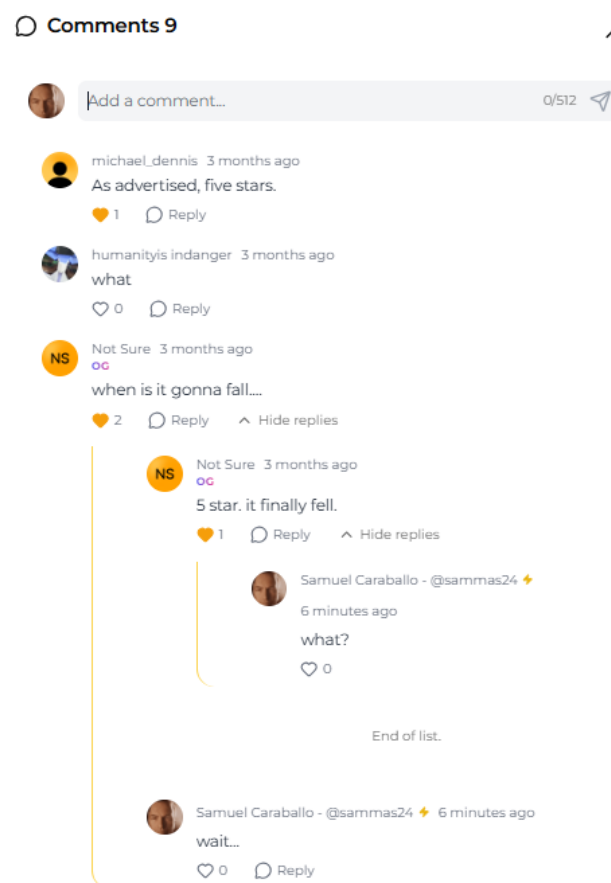


Figura 4.12: Comentarios de un video

Como se puede observar en la figura, los comentarios pueden tener respuestas. Esto requirió crear una estructura de árbol para modelar los mensajes que dependen de otros. En la implementación elegida, cada comentario principal es la raíz de un árbol. Al añadir una respuesta se crea un nuevo vértice, se indica el nodo ancestro y se añade una arista entre ellos. Al cargar la vista de los comentarios, la aplicación obtiene la raíz de cada árbol. Posteriormente, si el usuario quiere ver respuestas, selecciona la opción para desplegar los hijos adyacentes

de un nodo y la aplicación obtiene de la base de datos los descendientes ².

Los comentarios se cargan bajo demanda y también se ordenan y priorizan según su popularidad. Cada comentario dispone de un botón de *like*, lo cual cambia su relevancia. Al cargar los comentarios principales (nodos raíz), la aplicación los organiza por número de *likes* en orden descendente. Las respuestas se ordenan de la misma forma. Como futura ampliación, se podrían implementar algoritmos más sofisticados de recomendación de comentarios que tengan en cuenta el nivel de interacción, la antigüedad, el tono del mensaje, el contexto del mensaje, entre otros aspectos.

De esta forma, se consigue una interfaz más clara y moderna de los comentarios de los usuarios. Cabe destacar, que la profundidad máxima se ha limitado a 3 hijos para evitar que los componentes se superpongan y se desborden.

4.2.4. Mejoras en los canales de usuario

Los canales de usuario en *Booster* también presentan mejoras. Estos no se limitan únicamente a mostrar un perfil básico del contenido del creador sino que también, se incluye una estructura más completa con componentes que facilitan la visibilidad de la marca personal, la personalización y la interacción con sus seguidores.

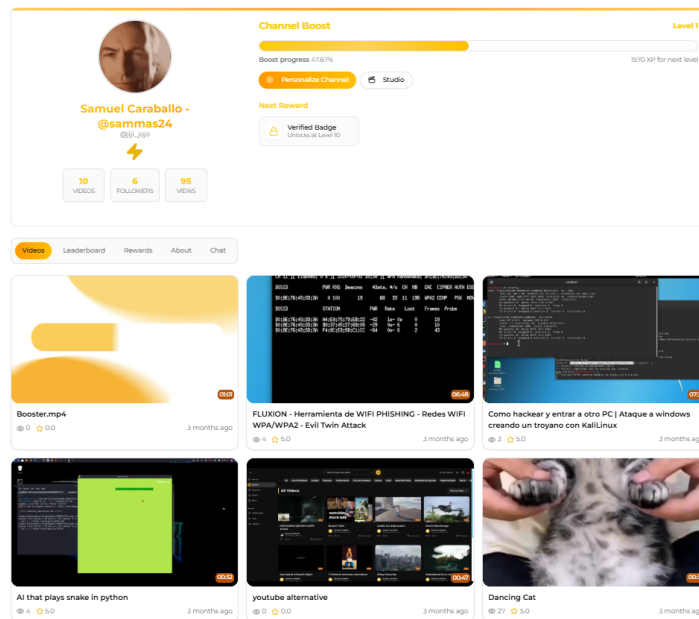


Figura 4.13: Canal de usuario. Punto de vista del propietario

En la Figura 4.13 se observa la vista básica del canal de un usuario en *Booster* cuando el propietario ha iniciado sesión. En este caso, no se ofrecen las opciones de seguimiento ni *Boost* para evitar el auto-impulso. En cambio, se sustituyen por ajustes de personalización y un acceso directo al *studio*, sección donde se administran, gestionan y se consultan las estadísticas del canal.

²Se ha considerado que el ancestro de un nodo raíz es él mismo.

Capítulo 4. Innovaciones

Como se puede observar, en la parte superior de la interfaz se encuentra la información principal del creador. Aquí, se muestran datos como el nombre del canal, su foto de perfil, su nombre de usuario, el número de videos publicados, el número de seguidores y el total de visitas en todos sus videos. Asimismo, en la parte derecha, se observan los **puntos de Boost** del canal junto con su **Nivel de Canal**. Esta barra de progreso indica la cantidad de **puntos de experiencia (XP)** que son necesarios invertir para alcanzar el siguiente nivel. Estas inversiones de XP son donadas por los espectadores que desean impulsar la visibilidad del creador en la plataforma y participar en las recompensas asociadas al incremento de nivel.

En la figura 4.14, se observa la información que un espectador del canal se encuentra.

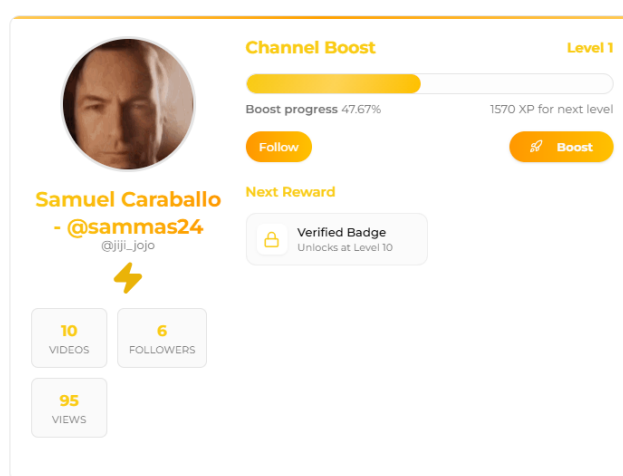


Figura 4.14: Canal de usuario. Punto de vista del espectador

Aquí se permite seguir al creador haciendo clic en *Follow*. Además, dispone de la opción para realizar un *Boost*. Es decir, al presionar este botón, el usuario puede invertir una cierta cantidad de puntos de experiencia aumentando así los puntos de Boost del canal. De esta forma, el usuario pierde puntos pero colabora en la visibilidad y éxito del creador, además de recibir premios cuando el nivel de canal aumente. En la figura 4.15 se observa el *modal* desplegado cuando un usuario desea invertir puntos en un canal. Se dan distintas opciones de cantidades. En caso de que el usuario no disponga de la suficiente cantidad de puntos (XP), se genera un error. Por el contrario, el creador de contenido aumenta sus puntos de Boost y se le restan al usuario que invirtió. Cuando el nivel de usuario ha incrementado recientemente, se muestra una insignia pública durante 24 horas para indicar a todos los visitantes del ascenso, como se observa en la figura 4.16.

Al invertir puntos en un canal, el usuario entra automáticamente en una clasificación global. Como se muestra en la figura 4.17, este ranking ordena a los participantes de mayor a menor cantidad de XP invertida a lo largo del tiempo en ese canal. Se encuentra fácilmente accesible con la pestaña de *Leaderboard*.

En la figura 4.17 se muestra una clasificación de 7 usuarios. Además, si el

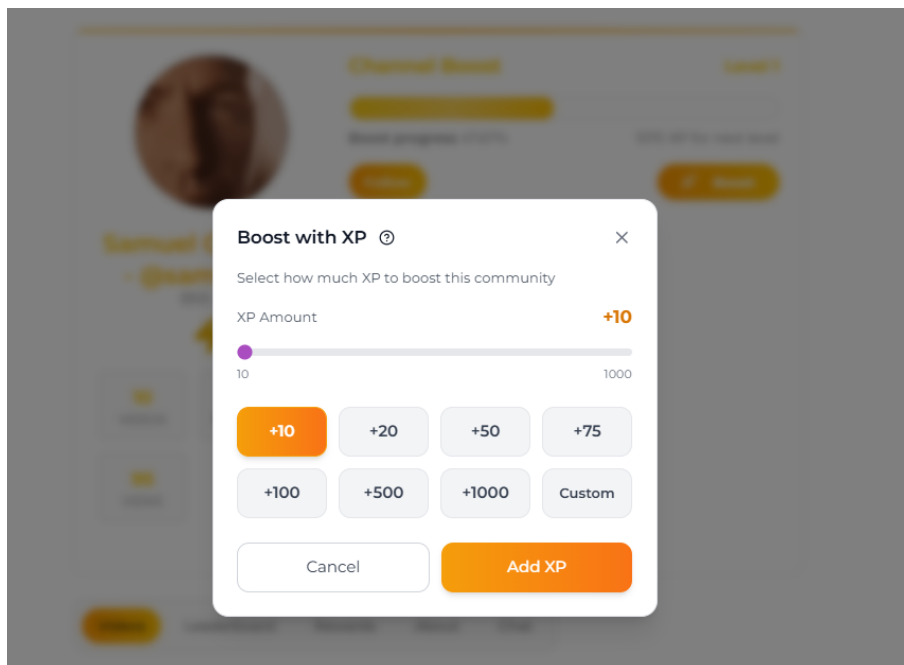


Figura 4.15: Modal para seleccionar cantidades de puntos a donar

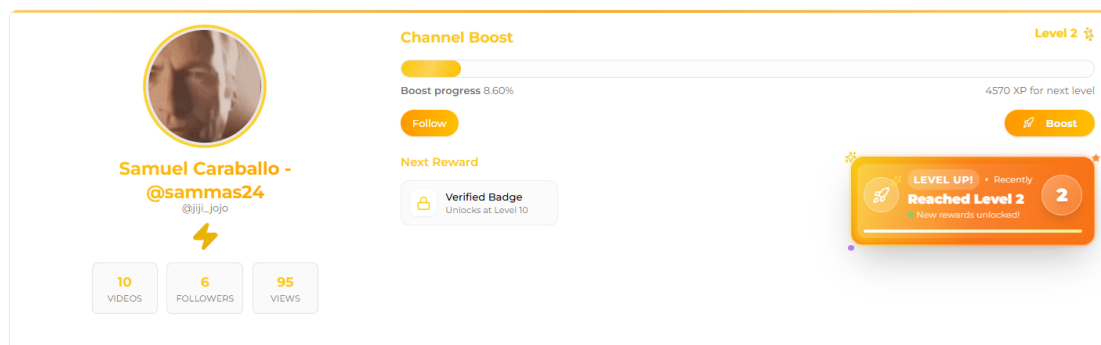


Figura 4.16: Ascenso de nivel reciente en un canal

Capítulo 4. Innovaciones

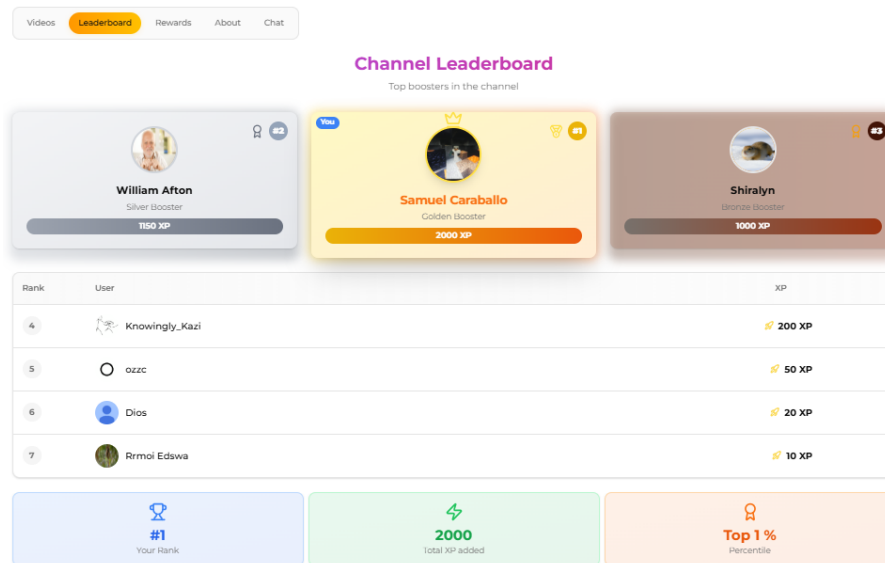


Figura 4.17: Clasificación de un canal de usuario

usuario en sesión está dentro del ranking, se incluye una etiqueta que lo destaca de los demás y en la parte baja, se indica la posición, la cantidad de puntos de experiencia donados y el centil superior.

Cada canal de usuario tiene una indicación visual del nivel alcanzado junto con las recompensas desbloqueadas hasta el momento. Esta información es accesible con la pestaña de *Rewards* como se observa en la figura 4.18. Durante esta etapa de desarrollo no se han implementado premios asociados a un canal, pero se contempla como ampliación para el futuro.

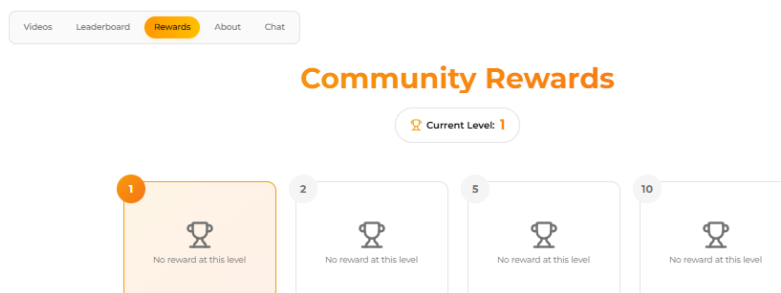


Figura 4.18: Recompensas y nivel del canal

Por otro lado, también se incluye más información personal del usuario en la pestaña *About*. En esta, se indica información sobre redes sociales adicionales como YouTube, Instagram, Discord, Facebook, TikTok, entre otros. Además, el usuario puede añadir una descripción de su canal la cual se mostrará en esta sección como se ve en la figura 4.19.

Finalmente, una funcionalidad nueva que no presentan las plataformas de streaming de videos es un *chat* de comunidad del canal. Este es accesible mediante la pestaña *Chat* como se ve en la figura 4.20. En este, cada seguidor del canal puede enviar mensajes e interactuar directamente con otros seguidores e inclu-

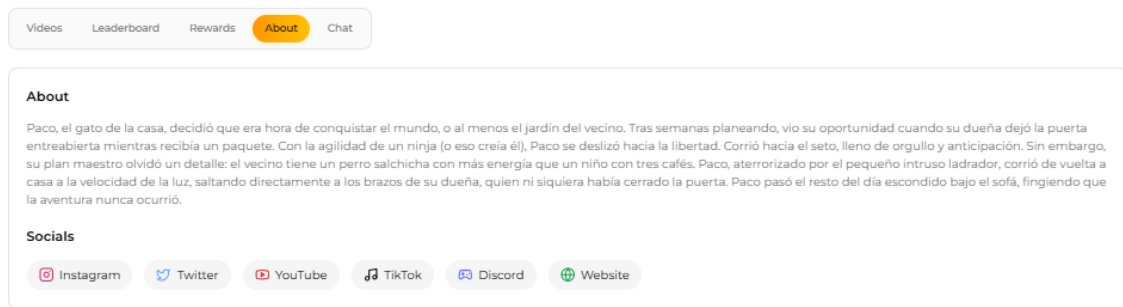


Figura 4.19: Información adicional de un canal

so el creador de contenido. De esta manera, hay una comunicación más directa entre espectadores y creadores. Esto opera de forma similar a los servidores de Discord donde los mensajes se muestran en orden cronológico y cualquiera puede participar. Por el momento, esta funcionalidad está disponible en todos los canales. Sin embargo, en un futuro se considera ofrecer este beneficio como premio al alcanzar un nivel alto. Esto permitiría incentivar el progreso del creador de contenido y la inversión de puntos de los seguidores optimizando, a su vez, la carga en los servidores y las bases de datos.

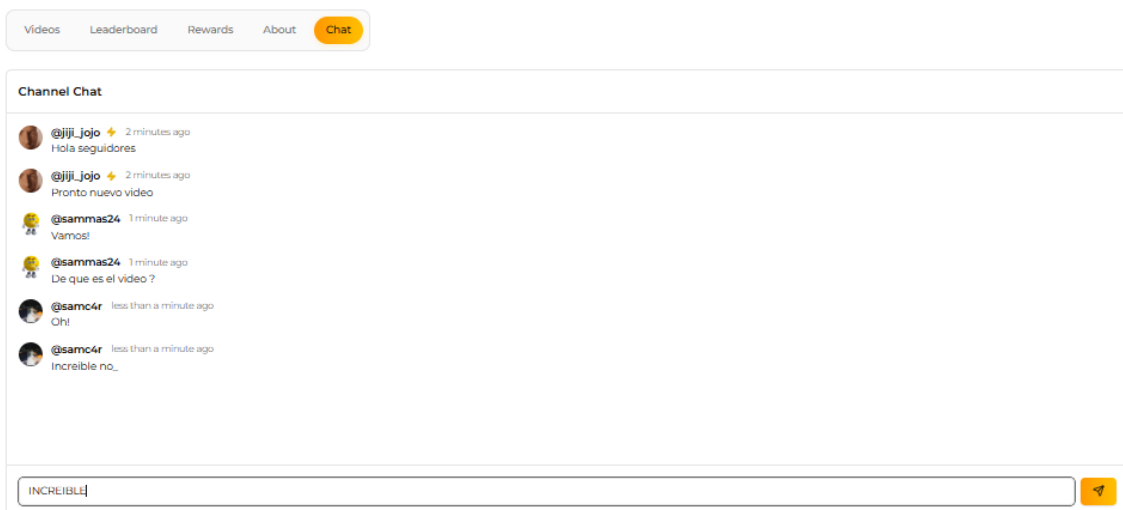


Figura 4.20: Chat de comunidad de un canal

4.2.5. Incorporación de gamificación

En *Booster* los métodos de gamificación juegan un papel importante. Es una capa estructural del servicio pensada para incrementar la participación y la retención. Para esto, se implementan distintos mecanismos inspirados en lógicas de videojuegos, combinando recompensas y reconocimiento público.

Uno de los componentes principales son las tablas de clasificación. *Booster* presenta una sección de *leaderboard global* en donde se compara el desempeño de los 100 mejores usuarios de la plataforma teniendo en cuenta la cantidad de seguidores, los puntos de Boost y el nivel del canal (4.21). Esto genera un entorno de competición y visibiliza los perfiles más activos.

Capítulo 4. Innovaciones

Además, se implementa un *leaderboard de comunidad* el cual se encuentra dentro de todos los canales (4.17). A diferencia de la clasificación anterior, en esta se reconocen a los seguidores más destacados de un usuario según la cantidad de puntos de experiencia invertidos. De esta manera, cada comunidad puede reconocer a sus miembros más fieles.

Otro mecanismo esencial es el sistema de puntos de experiencia o XP. Esta se obtiene principalmente por visualizaciones de anuncios dentro de la plataforma. En un futuro se implementarán más métodos de obtención de puntos basados en la interacción, como comentar, participar o consumir contenido. Esta moneda virtual permite a los usuarios realizar compras de artículos de personalización e impulsar la visibilidad de los creadores de contenido en los algoritmos de recomendación. La opción de donar XP posibilita a los seguidores apoyar el contenido de otro usuario e invertir puntos para desbloquear nuevas funcionalidades y recompensas de la comunidad. A su vez, se posicionan en un ranking que destaca a los miembros más activos y fieles del canal.

A todo esto, se suma la sección de *marketplace*. Aquí, los usuarios pueden comprar mediante el uso de XP, artículos de personalización, temas de interfaz, insignias, entre otras recompensas. Esto convierte a los puntos de experiencia en algo deseable, generando una motivación adicional para conseguirlos. El usuario decide en qué gastar sus puntos y cómo administrar sus recursos dentro de la plataforma, convirtiendo así los puntos en una moneda.

Otro mecanismo empleado son los anuncios recompensados. Este es un sistema inspirado en los videojuegos, donde una visualización de anuncios premia al usuario con una cantidad de moneda virtual, en este caso, puntos de experiencia. Este enfoque convierte una práctica percibida como intrusiva en una transacción beneficiosa para ambas partes. La plataforma monetiza, mientras el usuario recibe una compensación, lo que mejora la aceptación del formato publicitario.

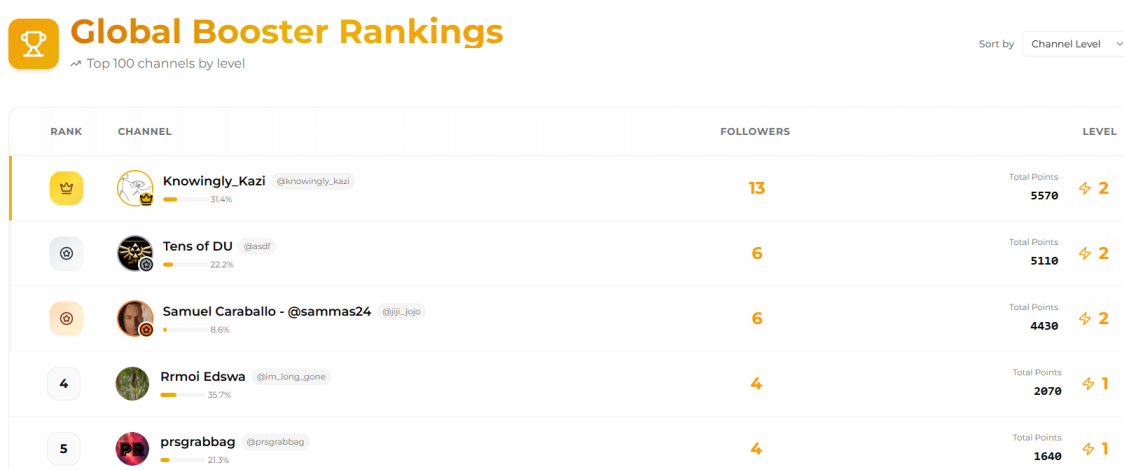


Figura 4.21: Leaderboard de los 100 mejores canales en *Booster*

Capítulo 5

Análisis

En esta sección se estudian las capacidades básicas que debe tener un producto mínimo viable de la aplicación, así como los actores que interactúan con el sistema y los casos de uso y diagramas de secuencia más importantes. Finalmente, se describe el diagrama entidad-relación de la base de datos.

5.1. Requisitos Funcionales

En esta sección se recopilan los requisitos funcionales más importantes que cumple el producto mínimo viable de la aplicación. Los requisitos funcionales son aquellos que definen los comportamientos, acciones y servicios que el sistema debe proporcionar al usuario.

En *Booster*, los requisitos funcionales se recopilan a continuación separados por distintos bloques.

5.1.1. Gestión de usuario

- RF-GU 1.** El sistema debe permitir a los usuarios registrarse en la plataforma con un correo electrónico o nombre de usuario.
- RF-GU 2.** El sistema debe enviar un código de verificación al correo proporcionado para completar el registro o tras un periodo de inactividad de un usuario registrado.
- RF-GU 3.** El sistema debe permitir a los usuarios iniciar sesión con un correo electrónico o nombre de usuario.
- RF-GU 4.** El sistema debe permitir a los usuarios cerrar sesión.
- RF-GU 5.** El sistema debe permitir a los usuarios recuperar su contraseña.
- RF-GU 6.** El sistema debe permitir al usuario subir una foto de perfil.
- RF-GU 7.** El sistema debe permitir a los usuarios editar la foto de perfil, nombre, nombre de usuario y correo electrónico.

RF-GU 8. El sistema debe permitir al usuario registrarse e iniciar sesión con una cuenta de Google.

5.1.2. Carga, gestión y moderación de videos

RF-CGMV 1. El sistema debe permitir al usuario cargar un video a la plataforma.

RF-CGMV 2. El sistema debe permitir al usuario agregar título, descripción y categoría al video.

RF-CGMV 3. El sistema debe permitir al usuario seleccionar la visibilidad del video.

RF-CGMV 4. El sistema debe permitir al usuario asociar un video a una comunidad.

RF-CGMV 5. El sistema debe permitir al usuario eliminar un video propio.

RF-CGMV 6. El sistema debe almacenar la información del video y sus metadatos.

RF-CGMV 7. El sistema debe implementar Adaptive Bitrate Streaming para la reproducción de video, lo que implica generar versiones de diferentes calidades.

RF-CGMV 8. El sistema debe permitir al usuario reportar un video.

RF-CGMV 9. El sistema debe permitir al usuario participar en votaciones para decidir si un video infringe las normas o no.

5.1.3. Reproducción y consumo de contenido

RF-RCC 1. El sistema debe permitir al usuario visualizar un video.

RF-RCC 2. El sistema debe permitir al usuario pausar, reanudar, adelantar y retroceder el video.

RF-RCC 3. El sistema debe permitir al usuario cambiar la calidad de reproducción cuando existan varias resoluciones disponibles.

RF-RCC 4. El sistema debe permitir al usuario reproducir videos en pantalla completa.

RF-RCC 5. El sistema debe registrar visualizaciones del video.

RF-RCC 6. El sistema debe permitir al usuario navegar al siguiente video desde la interfaz de visualización.

RF-RCC 7. El sistema debe recomendar videos relacionados.

RF-RCC 8. El sistema debe mostrar información del video, como título, descripción, número de visualizaciones, fecha de publicación y creador.

5.1.4. Exploración y búsqueda

- RF-EB 1.** El sistema debe permitir al usuario buscar videos por título, descripción o creador.
- RF-EB 2.** El sistema debe permitir al usuario filtrar videos por categorías.
- RF-EB 3.** El sistema debe recomendar al usuario videos en una cuadrícula principal.
- RF-EB 4.** El sistema debe permitir al usuario visualizar contenido reciente de comunidades y creadores que sigue.
- RF-EB 5.** El sistema debe permitir al usuario filtrar videos hechos por inteligencia artificial o con formato vertical.
- RF-EB 6.** El sistema debe permitir al usuario explorar comunidades y ver su contenido reciente.

5.1.5. Interacción social

- RF-IS 1.** El sistema debe permitir al usuario calificar un video en una escala de 1 a 5 estrellas.
- RF-IS 2.** El sistema debe permitir al usuario comentar en un video.
- RF-IS 3.** El sistema debe permitir al usuario responder a un comentario existente.
- RF-IS 4.** El sistema debe permitir al usuario editar o eliminar sus propios comentarios.
- RF-IS 5.** El sistema debe mostrar los comentarios en estructura jerárquica o en árbol.
- RF-IS 6.** El sistema debe recomendar los comentarios más relevantes o populares primero.
- RF-IS 7.** El sistema debe permitir a un usuario donar puntos de experiencia a un canal.

5.1.6. Comunidades

- RF-COM 1.** El sistema debe permitir al usuario crear una comunidad.
- RF-COM 2.** El sistema debe permitir al usuario unirse a comunidades.
- RF-COM 3.** El sistema debe permitir al usuario abandonar comunidades.
- RF-COM 4.** El sistema debe permitir al usuario consultar las comunidades a las que pertenece.
- RF-COM 5.** El sistema debe permitir al usuario consultar los videos más recientes asociados a una comunidad.

Capítulo 5. Análisis

RF-COM 6. El sistema debe permitir al usuario buscar comunidades por nombre o descripción.

RF-COM 7. El sistema debe permitir al usuario filtrar comunidades por categorías.

5.1.7. Canales

RF-CAN 1. El sistema debe permitir al usuario gestionar y personalizar su canal.

RF-CAN 2. El sistema debe permitir al usuario visualizar la pestaña de videos de un canal.

RF-CAN 3. El sistema debe permitir al usuario visualizar la pestaña ranking de canal con la clasificación de los usuario donantes.

RF-CAN 4. El sistema debe permitir al usuario acceder a la pestaña de recompensas desbloqueadas de un canal.

RF-CAN 5. El sistema debe permitir al usuario visualizar la pestaña de información o descripción del canal.

RF-CAN 6. El sistema debe permitir al usuario acceder al chat comunitario de un canal o comunidad.

RF-CAN 7. El sistema debe permitir al usuario consultar su propio canal.

RF-CAN 8. El sistema debe permitir al usuario seguir a otros canales.

RF-CAN 9. El sistema debe permitir al usuario dejar de seguir a otros canales.

RF-CAN 10. El sistema debe permitir al usuario consultar los canales que sigue.

5.1.8. Gamificación

RF-GAM 1. El sistema debe tener un recuento de los puntos de experiencia (XP) de cada usuario.

RF-GAM 2. El sistema debe permitir al usuario ganar puntos de experiencia al mirar anuncios.

RF-GAM 3. El sistema debe permitir al usuario consultar los puntos de boost y el nivel de un canal.

RF-GAM 4. El sistema debe mostrar una clasificación de los canales más populares según el número de seguidores y el nivel de canal.

RF-GAM 5. El sistema debe permitir al usuario visualizar artículos, recompensas o personalizaciones disponibles en la tienda.

RF-GAM 6. El sistema debe permitir al usuario comprar artículos de personalización en la tienda con puntos de experiencia.

RF-GAM 7. El sistema debe permitir al usuario visualizar su inventario de artículos adquiridos.

RF-GAM 8. El sistema debe mostrar badges o iconos especiales en el perfil del usuario.

RF-GAM 9. El sistema debe reflejar visualmente los cambios de nivel recientes en los canales.

5.2. Requisitos No Funcionales

Los requisitos no funcionales son aquellos que definen cómo debe comportarse el sistema en términos de rendimiento, seguridad, usabilidad y escalabilidad, en lugar de qué acciones se deben realizar.

RNF 1. Rendimiento: El sistema debe ser capaz de cargar las páginas en un tiempo aceptable (menos de 3 segundos) y reproducir videos sin interrupciones o buffering excesivo.

RNF 2. Seguridad: El sistema debe proteger la información de los usuarios. Se deben implementar medidas de seguridad frente a ataques comunes como inyección SQL y prevenir el uso abusivo de la plataforma mediante la implementación de *rate limits*.

RNF 3. Usabilidad: El sistema debe tener una interfaz intuitiva y fácil de usar.

RNF 4. Disponibilidad: El sistema debe estar activo la mayor cantidad de tiempo posible, minimizando el tiempo de inactividad.

RNF 5. Escalabilidad: El sistema debe ser capaz de manejar un aumento en el número de usuarios, interacciones y contenido sin degradar el rendimiento.

RNF 6. Compatibilidad: El sistema debe funcionar correctamente en diferentes navegadores web y dispositivos. Esto incluye navegadores populares como Chrome, Firefox, Safari y Edge, así como dispositivos móviles y de escritorio.

RNF 7. Mantenibilidad: El sistema debe estar diseñado con patrones claros, utilizando buenas prácticas de desarrollo para facilitar su mantenimiento y evolución.

RNF 8. Modularidad: El sistema debe estar diseñado de tal forma que sus componentes sean independientes y puedan evolucionar o ser reemplazados sin afectar al resto del sistema.

5.3. Actores

En un sistema software, los actores son entidades externas que interactúan con el sistema. Pueden ser usuarios humanos, otros sistemas o dispositivos. En el caso de *Booster*, los actores principales son:

- **Usuario visitante:** Un usuario que accede a la plataforma sin iniciar sesión. Este puede explorar, buscar y consumir contenido, acceder a canales

y rankings, pero no puede interactuar con el sistema ni guardar sus preferencias.

- **Usuario registrado:** Un usuario que ha creado una cuenta e iniciado sesión en la plataforma. Este, además de las capacidades de un usuario visitante, puede cargar videos, interactuar con el contenido, participar en comunidades, seguir canales, personalizar su experiencia y ganar puntos de experiencia.
- **Creador de contenido:** Un usuario registrado que se dedica a crear videos en la plataforma. Además de consumir, puede gestionar su contenido y su canal.
- **Sistema de procesamiento de videos:** Es el encargado de procesar los videos cargados por un usuario. Cuando un video es subido, procesa la transcodificación, implementa Adaptive Bitrate Streaming y genera miniaturas para servir los videos.

5.4. Casos de uso

Un caso de uso es una descripción de cómo un actor interactúa con el sistema para alcanzar un objetivo. A continuación se muestran los diagramas de caso de uso más importantes detectados y posteriormente una descripción de cada uno de ellos.

5.4.1. Gestión de usuarios

Los casos de uso relacionados con la gestión de usuario son:

1. Registro de usuario
2. Iniciar sesión
3. Recuperar contraseña
4. Cerrar sesión
5. Gestionar perfil de usuario

Diagrama

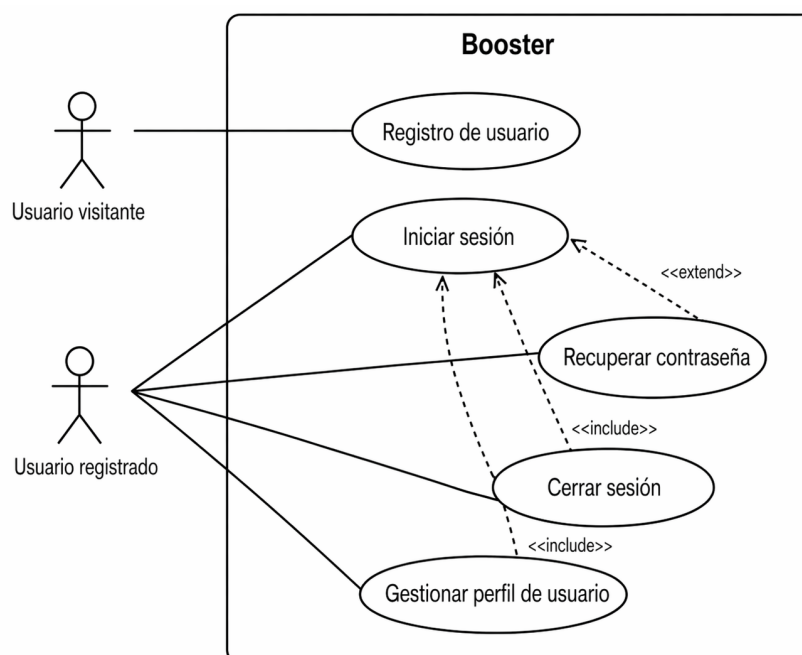


Figura 5.1: Gestión de usuarios en la plataforma - Casos de uso

Descripción

Caso de uso: Registro de usuario	
Actores	Usuario visitante
Descripción	Permite a un usuario crear una cuenta nueva en la plataforma mediante correo electrónico, nombre de usuario y contraseña. También es posible utilizar una cuenta de Google.
Precondiciones	El usuario no ha iniciado sesión y accede a la pantalla de registro.

Flujo principal	1. El usuario accede al formulario de registro. 2. El sistema solicita los datos necesarios para crear la cuenta. 3. El usuario introduce un correo electrónico, nombre de usuario y contraseña. 4. El sistema valida la información introducida. 5. El sistema envía un código de verificación al correo proporcionado. 6. El usuario introduce el código de verificación recibido. 7. El sistema verifica el código introducido. 8. El sistema crea la cuenta de usuario. 9. El sistema confirma que el registro se ha realizado correctamente. 10. El sistema redirige al usuario a la pantalla de inicio de sesión.
Flujo alternativo al paso 3: registro con Google	
3a. El usuario selecciona la opción de registro con Google. 3b. El sistema redirige al proveedor de autenticación de Google (OAuth) para completar el proceso. 3c. Si el proceso falla o es cancelado, el sistema muestra un mensaje de error y regresa a la pantalla de registro. 3d. Si la autenticación es correcta, el sistema crea la cuenta del usuario con los datos recibidos. 3e. El sistema confirma el registro exitoso y redirige al usuario a la pantalla principal.	
Flujo alternativo al paso 4: datos inválidos o inseguros	
4a. Si los datos introducidos no cumplen las reglas de validación, como contraseñas débiles, el sistema muestra los errores correspondientes en el formulario. 4b. El usuario corrige los errores señalados. 4c. El sistema vuelve a validar la información introducida.	
Flujo alternativo al paso 4: correo o nombre de usuario existente	
4a. Si el correo electrónico o el nombre de usuario ya existe, el sistema informa del conflicto y solicita datos diferentes. 4b. El usuario introduce un correo o nombre de usuario distinto. 4c. El sistema vuelve a validar la información introducida.	
Flujo alternativo al paso 7: código incorrecto o expirado	
7a. Si el código introducido es incorrecto o ha expirado, el sistema muestra un mensaje de error y solicita un nuevo intento. 7b. El usuario introduce nuevamente el código de verificación recibido. 7c. El sistema vuelve a verificar el código.	

5.4. Casos de uso

Postcondiciones	La cuenta del usuario se crea y se almacena en la base de datos.
Requisitos relacionados	RF-GU 1, RF-GU 2, RF-GU 8

Caso de uso: Iniciar sesión	
Actores	Usuario registrado
Descripción	Permite a un usuario autenticarse en la plataforma mediante correo electrónico o nombre de usuario y contraseña. También es posible iniciar sesión con una cuenta de Google.
Precondiciones	El usuario dispone de una cuenta y no ha iniciado sesión.
Flujo principal	1. El usuario accede a la pantalla de inicio de sesión. 2. El sistema solicita las credenciales. 3. El usuario introduce su correo electrónico o nombre de usuario y contraseña. 4. El sistema valida las credenciales. 5. El sistema autentica al usuario. 6. El sistema inicia la sesión y redirige al usuario a la pantalla principal.
Flujo alternativo al paso 3: inicio de sesión con Google	
3a. El usuario selecciona la opción de iniciar sesión con Google. 3b. El sistema redirige al proveedor de autenticación de Google (OAuth) para completar el proceso. 3c. Si el proceso falla o es cancelado, el sistema muestra un mensaje de error y regresa a la pantalla de inicio de sesión. 3d. Si la autenticación es correcta, el sistema autentica al usuario. 3e. El sistema inicia la sesión y redirige al usuario a la pantalla principal.	
Flujo alternativo al paso 3: recuperar contraseña	
3a. El usuario selecciona la opción de recuperar contraseña. 3b. El sistema redirige al usuario al caso de uso <i>Recuperar contraseña</i>.	
Flujo alternativo al paso 4: credenciales incorrectas	
4a. Si las credenciales introducidas no son correctas, el sistema muestra un mensaje de error. 4b. El sistema regresa a la pantalla de inicio de sesión y solicita un nuevo intento.	
Flujo alternativo al paso 4: verificación adicional por inactividad	

Capítulo 5. Análisis

4a. Si el sistema detecta un periodo prolongado de inactividad de la cuenta, envía un código de verificación al correo electrónico registrado. 4b. El usuario introduce el código de verificación recibido. 4c. El sistema valida el código introducido. 4d. Si el código es correcto, el flujo continúa en el paso 5 del Flujo principal.	
Flujo alternativo al paso 4a: código de verificación incorrecto o expirado	
4ai. Si el código introducido es incorrecto o ha expirado, el sistema muestra un mensaje de error y solicita un nuevo intento. 4aii. Se regresa al paso 4b.	
Postcondiciones	El usuario accede a la plataforma con una sesión activa.
Requisitos relacionados	RF-GU 2, RF-GU 3, RF-GU 8

Caso de uso: Recuperar contraseña	
Actores	Usuario registrado
Descripción	Permite a un usuario restablecer la contraseña de su cuenta. Para esto, el sistema envía un código o enlace de verificación al correo electrónico asociado.
Precondiciones	El usuario dispone de una cuenta previamente registrada y no ha iniciado sesión.

Flujo principal	1. El usuario accede a la opción de recuperar contraseña desde la pantalla de inicio de sesión. 2. El sistema solicita el correo electrónico asociado a la cuenta. 3. El usuario introduce su correo electrónico. 4. El sistema valida que exista una cuenta asociada a ese correo. 5. El sistema envía un código al correo electrónico registrado. 6. El usuario introduce el código de verificación. 7. El sistema valida el código. 8. El sistema solicita al usuario una nueva contraseña y su confirmación. 9. El usuario introduce la nueva contraseña y su confirmación. 10. El sistema valida la nueva contraseña. 11. El sistema actualiza la contraseña de la cuenta. 12. El sistema confirma que el restablecimiento se ha realizado correctamente y redirige al usuario a la pantalla de inicio de sesión.
Flujo alternativo al paso 4: correo no registrado	
4a. Si el correo electrónico introducido no está asociado a ninguna cuenta, el sistema muestra un mensaje de error solicitando al usuario que introduzca un correo válido y registrado.	
Flujo alternativo al paso 7: código inválido	
7a. Si el código introducido no es válido, el sistema muestra un mensaje de error.	
7b. El sistema solicita un nuevo intento de verificación.	
Flujo alternativo al paso 7: código expirado	
7a. Si el código introducido ha expirado, el sistema lo informa.	
7b. El sistema solicita al usuario que reinicie el proceso de recuperación de contraseña.	
Flujo alternativo al paso 10: contraseña inválida	
10a. Si la nueva contraseña no coincide con la confirmación o esta no cumple con las reglas de seguridad establecidas, el sistema muestra los errores correspondientes.	
10b. El usuario introduce una nueva contraseña válida y su confirmación.	
10c. El sistema vuelve a validar la contraseña introducida.	
Postcondiciones	La contraseña de la cuenta queda actualizada en la base de datos y el usuario puede volver a iniciar sesión con la nueva contraseña.

Requisitos relacionados	RF-GU-5
--------------------------------	---------

Caso de uso: Cerrar sesión	
Actores	Usuario registrado
Descripción	Permite a un usuario finalizar su sesión de forma segura.
Precondiciones	El usuario ha iniciado sesión en la plataforma.
Flujo principal	1. El usuario accede a la opción de cerrar sesión. 2. El sistema invalida la sesión activa del usuario. 3. El sistema redirige al usuario a la pantalla principal.
Flujo alternativo al paso 2: sesión expirada	
2a. Si la sesión del usuario ya ha expirado, el sistema informa de que no existe una sesión activa válida. 2b. El sistema redirige igualmente al usuario a la pantalla principal.	
Postcondiciones	La sesión del usuario queda finalizada y este deja de estar autenticado en la plataforma.
Requisitos relacionados	RF-GU-4

Caso de uso: Gestionar perfil de usuario	
Actores	Usuario registrado
Descripción	Permite a un usuario registrado consultar y modificar la información básica de su perfil. Esto incluye, cambiar la foto de perfil, su nombre, nombre de usuario y su correo electrónico.
Precondiciones	El usuario ha iniciado sesión en la plataforma.
Flujo principal	1. El usuario accede a su perfil en la barra de navegación. 2. El usuario selecciona la opción de editar perfil (<i>Manage account</i>). 3. El usuario modifica uno o varios de los siguientes datos: foto de perfil, nombre, nombre de usuario o correo electrónico. 4. El sistema valida la información introducida. 5. El sistema guarda los cambios realizados. 6. El sistema confirma que la actualización del perfil se ha realizado correctamente.
Flujo alternativo al paso 4: datos inválidos	

4a. Si alguno de los datos introducidos no cumple las reglas de validación, el sistema muestra los errores correspondientes. 4b. El usuario corrige los errores señalados. 4c. El sistema vuelve a validar la información.	
Flujo alternativo al paso 4: correo o nombre de usuario ya existente	
4a. Si el nuevo correo electrónico o nombre de usuario ya está asociado a otra cuenta, el sistema informa del conflicto. 4b. El usuario introduce un correo electrónico o nombre de usuario diferente. 4c. El sistema vuelve a validar la información.	
Flujo alternativo al paso 4: error al subir la foto de perfil	
4a. Si la imagen seleccionada no cumple el formato o tamaño permitido, el sistema muestra un mensaje de error. (Tamaño recomendado 1:1; Límite de peso es 10MB.) 4b. El usuario selecciona una imagen válida. 4c. El sistema permite continuar con la edición del perfil.	
Postcondiciones	La información actualizada del perfil queda almacenada en la base de datos del sistema.
Requisitos relacionados	RF-GU 6, RF-GU 7

5.4.2. Carga, gestión y moderación de videos

Los casos de uso identificados relacionados con la carga, gestión y moderación de videos son:

1. Subir video
2. Editar metadatos del video
3. Eliminar video
4. Reportar video
5. Votar en moderación de contenido
6. Procesar video

Diagrama

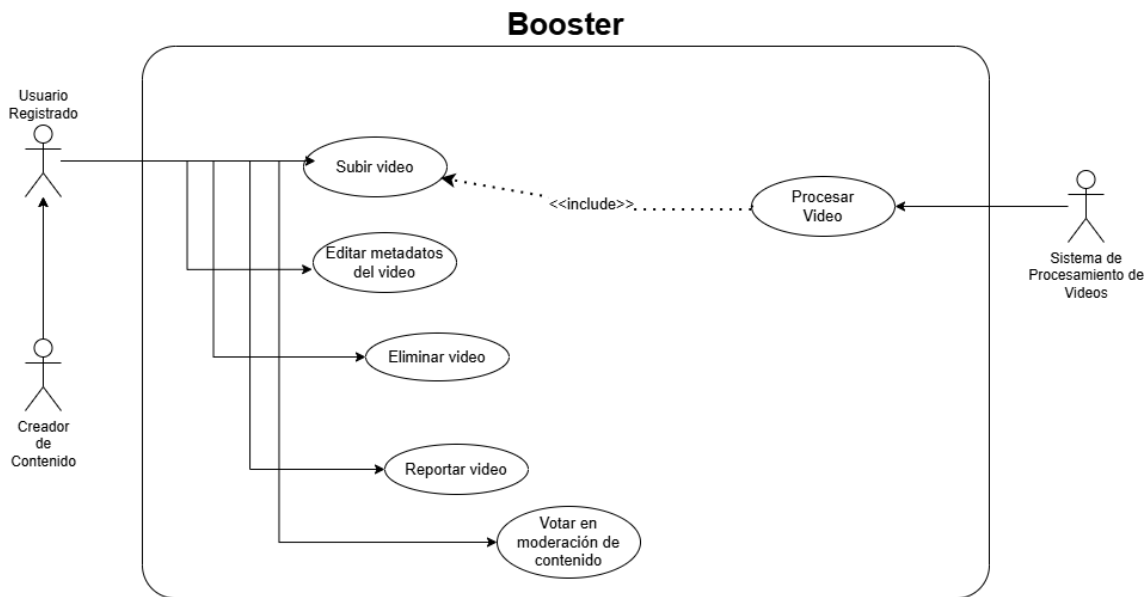


Figura 5.2: Carga de Videos a la plataforma - Casos de uso

Descripción

Caso de uso: Subir video	
Actores	Usuario registrado, Creador de Contenido, Sistema de Procesamiento de Videos.
Descripción	Permite a un usuario registrado cargar un video a la plataforma, indicando sus datos básicos, su visibilidad y, opcionalmente, la comunidad a la que será asociado.
Precondiciones	El usuario ha iniciado sesión en la plataforma y dispone de un archivo de video válido.

Flujo principal	1. El usuario selecciona la opción de subir video. 2. El sistema muestra el formulario de carga. 3. El usuario selecciona el archivo de video que desea subir. 4. El sistema recibe el archivo seleccionado y comienza a subirse al servidor. 5. El usuario introduce el título, la descripción y la categoría del video. 6. El usuario selecciona la visibilidad del video. 7. El usuario indica, opcionalmente, la comunidad a la que desea asociar el video. 8. El sistema almacena el video y sus metadatos. 9. El sistema registra el video en la plataforma e inicia su procesamiento. 10. El sistema confirma que la carga se ha realizado correctamente.
Flujo alternativo al paso 3: archivo inválido	
3a. Si el archivo seleccionado no corresponde a un formato de video permitido, excede el tamaño soportado o el límite de tiempo (10 minutos), el sistema muestra un mensaje de error. 3b. El usuario selecciona un archivo válido. 3c. El sistema permite continuar con el proceso de carga.	
Flujo alternativo al paso 5: datos inválidos	
5a. Si alguno de los metadatos introducidos no cumple las reglas de validación como un título con más de 100 caracteres, el sistema muestra los errores correspondientes. 5b. El usuario corrige la información. 5c. El sistema vuelve a validar los metadatos.	
Flujo alternativo al paso 7: comunidad no válida	
7a. Si la comunidad seleccionada no existe o el usuario no tiene permisos para publicar en ella, se muestra un mensaje de error. 7b. El usuario selecciona otra comunidad o decide no asociar el video a ninguna. 7c. El sistema continúa con el proceso de publicación.	
Postcondiciones	El video y sus metadatos quedan almacenados en el sistema y el procesamiento del contenido queda iniciado.
Requisitos relacionados	RF-CGMV 1, RF-CGMV 2, RF-CGMV 3, RF-CGMV 4, RF-CGMV 6

Caso de uso: Editar metadatos del video

Actores	Usuario registrado o Creador de Contenido
----------------	---

Descripción	Permite al propietario de un video modificar su título, descripción, categoría, visibilidad y comunidad asociada.
Precondiciones	El usuario ha iniciado sesión en la plataforma y es el creador del video que desea editar.
Flujo principal	1. El usuario accede a la sección de gestión de sus videos (<i>studio</i>). 2. El sistema muestra la lista de videos del usuario. 3. El usuario selecciona el video que desea editar. 4. El sistema muestra los metadatos del video. 5. El usuario modifica uno o varios de los siguientes datos: título, descripción, categoría, visibilidad o comunidad asociada. 6. El sistema valida la información introducida. 7. El usuario guarda los cambios. 8. El sistema actualiza la información del video. 9. El sistema confirma que la edición se ha realizado correctamente.
Flujo alternativo al paso 3: video no editable	
3a. Si el video no existe o no pertenece al usuario, el sistema muestra un mensaje de error. 3b. El sistema impide continuar con la edición y regresa a la lista de videos.	
Flujo alternativo al paso 6: datos inválidos	
6a. Si alguno de los metadatos introducidos no cumple las reglas de validación, el sistema muestra los errores correspondientes. 6b. El usuario corrige la información. 6c. El sistema vuelve a validar los metadatos.	
Flujo alternativo al paso 5: comunidad no válida	
5a. Si la comunidad seleccionada no existe o el usuario no tiene permisos para publicar en ella, se muestra un mensaje de error. 5b. El usuario selecciona otra comunidad o elimina la asociación. 5c. El sistema permite continuar con la edición.	
Postcondiciones	Los metadatos actualizados del video quedan almacenados en el sistema.
Requisitos relacionados	RF-CGMV 2, RF-CGMV 3, RF-CGMV 4, RF-CGMV 6

Caso de uso: Eliminar video	
Actores	Usuario registrado o Creador de Contenido

5.4. Casos de uso

Descripción	Permite al propietario de un video eliminarlo de la plataforma.
Precondiciones	El usuario ha iniciado sesión en la plataforma y es el creador del video que desea eliminar.
Flujo principal	1. El usuario accede a la sección de gestión de sus videos. 2. El sistema muestra la lista de videos del usuario. 3. El usuario selecciona la opción de eliminar en el video deseado. 4. El sistema solicita confirmación antes de proceder. 5. El usuario confirma la eliminación. 6. El sistema elimina el video de la plataforma y actualiza la base de datos. 7. El sistema confirma que el video ha sido eliminado correctamente.
Flujo alternativo al paso 3: video no eliminable	
3a. Si el video no existe o no pertenece al usuario, el sistema muestra un mensaje de error. 3b. El sistema impide la operación y regresa a la lista de videos.	
Flujo alternativo al paso 4: cancelación de la eliminación	
4a. El usuario decide no confirmar la eliminación. 4b. El sistema cancela la operación y no hay cambios.	
Postcondiciones	El video se elimina de la base de datos y deja de estar disponible en la plataforma.
Requisitos relacionados	RF-CGMV 5

Caso de uso: Reportar video	
Actores	Usuario registrado
Descripción	Permite a un usuario registrado reportar un video por posible incumplimiento de las normas de la plataforma.
Precondiciones	El usuario ha iniciado sesión y accede a un video.

Flujo principal	1. El usuario accede a un video de la plataforma. 2. El usuario selecciona la opción de reportar video. 3. El sistema muestra el formulario de reporte. 4. El usuario selecciona el motivo del reporte y, opcionalmente, añade información adicional. 5. El usuario confirma el envío del reporte. 6. El sistema registra el reporte asociado al video y al usuario. 7. El sistema confirma que el reporte ha sido enviado correctamente. 8. El sistema abre una votación de moderación para ese video con el motivo seleccionado.
Flujo alternativo al paso 4: información incompleta	
4a. Si el motivo del reporte no ha sido seleccionado o la información requerida está incompleta, el sistema muestra un mensaje de error. 4b. El usuario completa la información solicitada. 4c. El sistema permite continuar con el envío del reporte.	
Flujo alternativo al paso 5: reporte duplicado	
5a. Si el usuario ya había reportado previamente el video por el mismo motivo, el sistema informa de ello. 5b. El sistema impide registrar un nuevo reporte duplicado.	
Flujo alternativo al paso 5: votación abierta	
5a. Si el video había sido reportado por el mismo motivo, el sistema agrega un voto a favor en la votación de moderación y le indica al usuario que ya existía una votación abierta por el mismo motivo.	
Postcondiciones	El reporte del video queda almacenado en el sistema para su posterior revisión o votación.
Requisitos relacionados	RF-CGMV-8

Caso de uso: Votar en moderación de contenido	
Actores	Usuario registrado
Descripción	Permite a un usuario registrado participar en una votación para decidir si un video reportado infringe o no las normas de la plataforma.
Precondiciones	El usuario ha iniciado sesión y existe una votación abierta sobre un video.

Flujo principal	1. El usuario accede a un video. 2. El sistema indica que existe una votación de moderación abierta para el video en la sección de detalles del video. 3. El usuario revisa el contenido reportado y las razones. 4. El usuario emite su voto sobre si el video infringe o no las normas. 5. El sistema registra el voto emitido. 6. El sistema confirma que la votación se ha realizado correctamente.
Flujo alternativo al paso 2: no hay votaciones	
2a. El sistema impide continuar con la acción de votar e indica que no hay votaciones disponibles.	
Flujo alternativo al paso 4: voto ya emitido	
4a. Si el usuario ya ha votado previamente sobre ese caso, el sistema informa de ello. 4b. El sistema impide registrar un nuevo voto para el mismo proceso de moderación.	
Postcondiciones	El voto del usuario queda almacenado en el sistema y asociado al proceso de moderación correspondiente.
Requisitos relacionados	RF-CGMV 9

Caso de uso: Procesar video	
Actores	Sistema de procesamiento de videos
Descripción	Permite al sistema procesar automáticamente un video cargado, generando las copias necesarias para su reproducción en distintas calidades y <i>bitrates</i> y almacenando los resultados asociados.
Precondiciones	Existe un video cargado en la plataforma pendiente de procesamiento.

Flujo principal	1. El sistema registra que un nuevo video ha sido cargado. 2. El sistema de procesamiento recibe el archivo del video. 3. El sistema de procesamiento analiza el archivo recibido. 4. El sistema de procesamiento genera las distintas versiones de calidad del video. 5. El sistema de procesamiento genera los recursos necesarios para implementar <i>Adaptive Bitrate</i>. 6. El sistema de procesamiento almacena los archivos procesados y le informa al sistema sobre los datos actualizados. 7. El sistema marca el video como procesado y está disponible para su reproducción en la plataforma.
Flujo alternativo al paso 3: archivo corrupto o no procesable	
3a. Si el archivo se encuentra corrupto o no puede ser interpretado correctamente, el sistema de procesamiento registra el error y se lo informa al sistema. 3b. El sistema marca el video como fallido. 3c. El sistema notifica que el video no ha podido ser procesado.	
Flujo alternativo al paso 4: fallo durante la transcodificación	
4a. Si ocurre un error durante la generación de calidades, el sistema de procesamiento registra la incidencia. 4b. El sistema de procesamiento reintenta la operación. 4c. Si el error persiste, el sistema de procesamiento informa al sistema y el video queda marcado como no disponible.	
Postcondiciones	El video queda procesado y disponible para reproducción en la plataforma.
Requisitos relacionados	RF-CGMV 6, RF-CGMV 7

5.4.3. Reproducción y consumo de contenido

Los casos de uso identificados relacionados con la reproducción y consumo de contenido son:

1. Reproducir video
2. Controlar ajustes de reproducción
3. Información del video
4. Videos relacionados

Diagrama

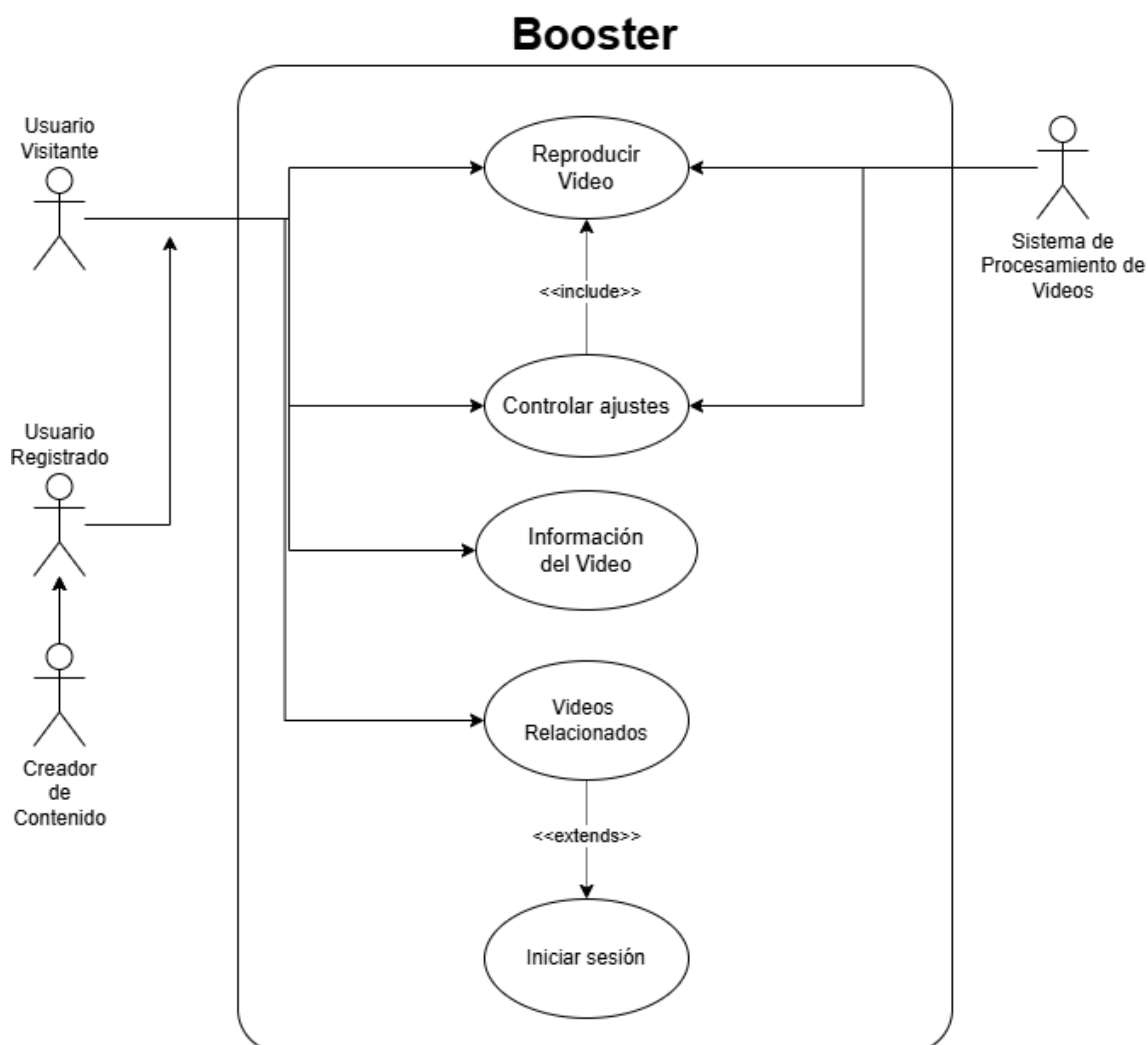


Figura 5.3: Reproducción y consumo de contenido

Descripción

Caso de uso: Reproducir video	
Actores	Usuario visitante, Usuario registrado o Creador de Contenido y Sistema de procesamiento de videos
Descripción	Permite al usuario visualizar un video disponible en la plataforma.
Precondiciones	El video existe y está disponible para su visualización en la plataforma.

Flujo principal	1. El usuario accede a un video desde la plataforma. 2. El sistema carga el reproductor de video. 3. El sistema le pide al sistema de procesamiento de videos los recursos. 4. El sistema inicia la reproducción del video. 5. El sistema registra la visualización del video. 6. El usuario visualiza el contenido.
Flujo alternativo al paso 2: error de carga	
2a. Si el reproductor no puede cargarse correctamente, el sistema muestra un mensaje de error. 2b. El sistema impide la reproducción del video.	
Postcondiciones	El video ha sido reproducido y su visualización queda registrada.
Requisitos relacionados	RF-RCC 1, RF-RCC 5

Caso de uso: Controlar ajustes de reproducción	
Actores	Usuario visitante, Usuario registrado o Creador de Contenido y Sistema de procesamiento de videos
Descripción	Permite al usuario controlar la reproducción del video mediante acciones como pausar, reanudar, adelantar, retroceder o cambiar la calidad y la velocidad de reproducción.
Precondiciones	El video se encuentra en reproducción.
Flujo principal	1. El usuario accede a los controles del reproductor. 2. El usuario pausa o reanuda el video. 3. El usuario adelanta o retrocede el video. 4. El usuario cambia la velocidad de reproducción. 5. El usuario cambia la calidad de reproducción si existen varias resoluciones disponibles y el sistema le pide al sistema de procesamiento de videos los recursos necesarios. 6. El usuario activa o desactiva el modo de pantalla completa. 7. El sistema aplica los cambios solicitados en tiempo real.
Flujo alternativo al paso 5: calidad no disponible	
5a. Si la calidad seleccionada no está disponible, el sistema mantiene la calidad actual.	

5.4. Casos de uso

Postcondiciones	El video se reproduce con los ajustes seleccionados por el usuario.
Requisitos relacionados	RF-RCC 2, RF-RCC 3, RF-RCC 4

Caso de uso: Información del video	
Actores	Usuario visitante, Usuario registrado o Creador de Contenido
Descripción	Permite al usuario consultar la información asociada a un video.
Precondiciones	El usuario accede a la interfaz de visualización de un video.
Flujo principal	1. El usuario accede a la página de visualización de un video. 2. El sistema muestra el título del video. 3. El sistema muestra la descripción del video. 4. El sistema muestra el número de visualizaciones. 5. El sistema muestra la fecha de publicación. 6. El sistema muestra el creador del contenido así como sus puntos de Boost y nivel.
Postcondiciones	El usuario visualiza la información disponible del video.
Requisitos relacionados	RF-RCC 8

Caso de uso: Videos relacionados	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario visualizar y acceder a videos recomendados relacionados con el contenido actual.
Precondiciones	El usuario se encuentra visualizando un video.
Flujo principal	1. El usuario accede a la interfaz de visualización de un video. 2. El sistema muestra una lista de videos relacionados según las preferencias, historial y gustos del usuario en la pestaña de <i>Videos</i>. 3. El usuario selecciona uno de los videos recomendados. 4. El sistema carga el nuevo video seleccionado. 5. El sistema inicia la reproducción del nuevo contenido.
Flujo alternativo al paso 2: usuario no logueado	

2a. Si el usuario no está logueado no hay videos relacionados y personalizados. 2b. El sistema muestra una lista alternativa de contenido similar popular sin tener en cuenta las preferencias del usuario.	
Postcondiciones	El usuario accede a nuevo contenido recomendado o alternativo.
Requisitos relacionados	RF-RCC 6, RF-RCC 7

5.4.4. Exploración y búsqueda

Los casos de uso identificados relacionados con la exploración y búsqueda de contenido son:

1. Buscar videos
2. Explorar y filtrar videos
3. Explorar comunidades

Diagrama

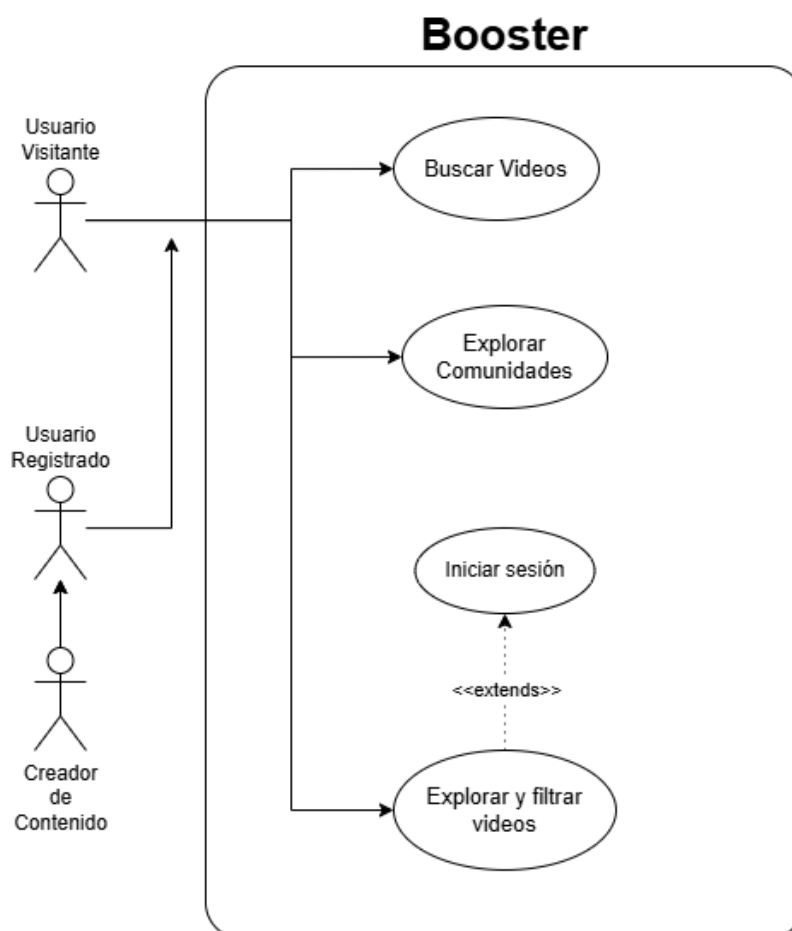


Figura 5.4: Exploración y búsqueda de contenido - Casos de uso

Descripción

Caso de uso: Buscar videos	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario buscar videos en la plataforma a partir de su título, descripción o creador. La búsqueda puede apoyarse también en mecanismos de búsqueda semántica.
Precondiciones	El usuario accede a la interfaz de búsqueda de la plataforma.

Flujo principal	1. El usuario accede a la barra o sección de búsqueda. 2. El sistema solicita el término de búsqueda. 3. El usuario introduce una consulta. 4. El sistema procesa la consulta. 5. El sistema busca videos por título, descripción o creador. 6. El sistema muestra la lista de resultados encontrados. 7. El usuario selecciona uno de los resultados para acceder al contenido.
Flujo alternativo al paso 5: sin resultados	
5a. Si no se encuentran videos coincidentes, el sistema informa de que no existen resultados para la consulta realizada. 5b. El sistema invita al usuario a realizar otra búsqueda.	
Postcondiciones	El usuario obtiene una lista de videos o creadores de contenido relacionados con la consulta introducida.
Requisitos relacionados	RF-EB 1

Caso de uso: Explorar y filtrar videos	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario explorar la cuadrícula principal de videos recomendados y aplicar filtros sobre el contenido disponible, por ejemplo por categoría, contenido generado por inteligencia artificial o formato vertical. Además, permite explorar los videos más recientes de las comunidades y canales que sigue
Precondiciones	El usuario accede a la pantalla principal o a la sección de exploración de videos.

Flujo principal	1. El usuario accede a la sección principal de exploración de videos. 2. El sistema muestra una cuadrícula de videos recomendados. 3. El usuario accede a las opciones de filtrado. 4. El usuario selecciona uno o varios filtros, como categoría, contenido hecho por inteligencia artificial o formato vertical. 5. El sistema aplica los filtros seleccionados. 6. El sistema actualiza la cuadrícula de videos mostrada. 7. El usuario explora los resultados filtrados y selecciona un video si lo desea. 8. El usuario accede a la pestaña de comunidades. 9. El sistema le muestra los videos más recientes de las comunidades que sigue. 10. El usuario accede a la pestaña de canales. 11. El sistema muestra los videos más recientes de los canales que sigue.
Flujo alternativo al paso 2: error al cargar recomendaciones	
2a. Si no es posible cargar la cuadrícula principal de recomendaciones, el sistema muestra un mensaje de error. 2b. El sistema permite reintentar la carga.	
Flujo alternativo al paso 4: filtros sin resultados	
4a. Si la combinación de filtros seleccionada no produce resultados, el sistema lo informa.	
Flujo alternativo a los pasos 8 y 10: usuario sin sesión	
i. El usuario no puede acceder a las secciones que requieren autenticación. ii. El sistema muestra un mensaje solicitando al usuario que inicie sesión para acceder a estas pestañas.	
Postcondiciones	El usuario visualiza una cuadrícula de videos adaptada a los filtros seleccionados.
Requisitos relacionados	RF-EB 2, RF-EB 3, RF-EB 4, RF-EB 5

Caso de uso: Explorar comunidades	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario explorar comunidades en la plataforma y ver el contenido reciente de estas.
Precondiciones	El usuario accede a la sección de exploración de comunidades.

Flujo principal	1. El usuario accede a la sección de exploración de comunidades. 2. El sistema muestra comunidades destacados o recientes. 3. El usuario navega entre las opciones disponibles. 4. El usuario selecciona una comunidad o canal para consultar su contenido. 5. El sistema muestra la información y el contenido asociado a la comunidad o canal seleccionado.
Flujo alternativo al paso 2: sin contenido disponible	
2a. Si no existen comunidades o canales disponibles para mostrar, el sistema informa de ello. 2b. El sistema mantiene la sección accesible para futuras consultas.	
Postcondiciones	El usuario visualiza comunidades, y accede a su contenido reciente.
Requisitos relacionados	RF-EB 6

5.4.5. Interacción social

Los casos de uso identificados relacionados con la interacción social son:

1. Calificar video
2. Gestión de comentarios
3. Donar puntos de experiencia

Diagrama

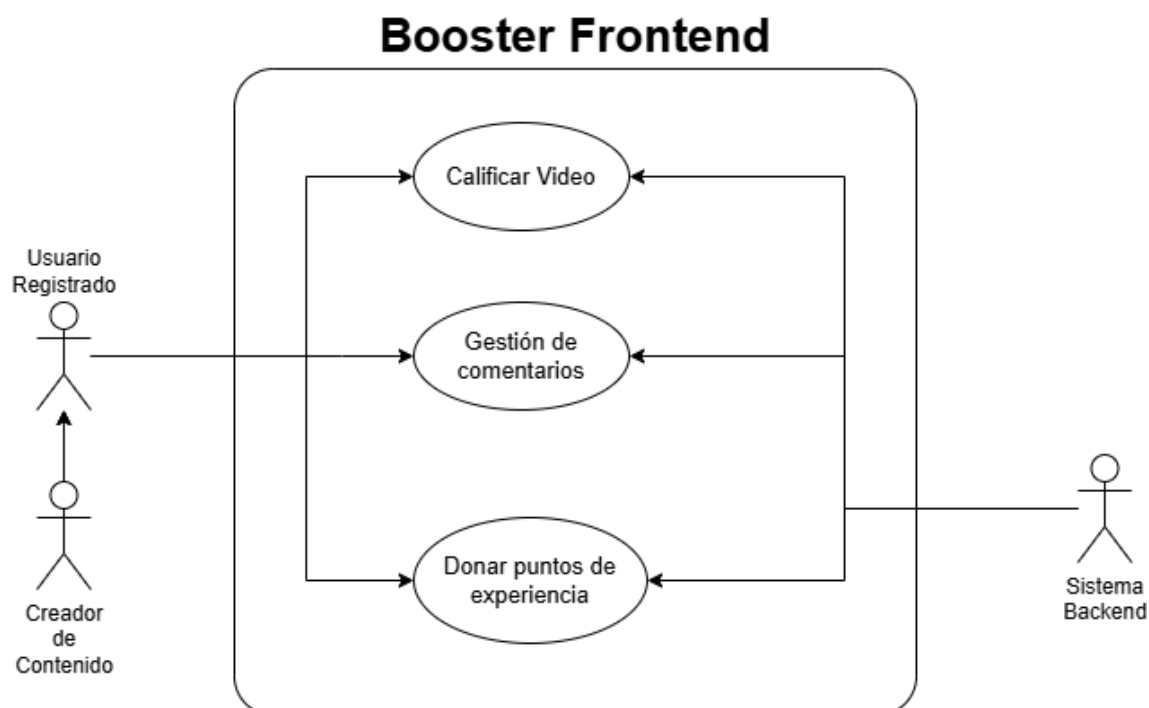


Figura 5.5: Interacciones en la plataforma - Casos de uso

Descripción

Caso de uso: Calificar video	
Actores	Usuario registrado
Descripción	Permite a un usuario registrado calificar un video mediante una puntuación de 1 a 5 estrellas.
Precondiciones	El usuario ha iniciado sesión en la plataforma y accede a un video disponible.
Flujo principal	1. El usuario accede a la página de visualización de un video. 2. El sistema muestra la opción de calificación mediante estrellas. 3. El usuario selecciona una puntuación entre 1 y 5 estrellas. Si el usuario no tenía una sesión iniciada, el sistema le solicita que inicie sesión para calificar el video. 4. El sistema registra la calificación emitida por el usuario. 5. El sistema actualiza la valoración promedio asociada al video. 6. El sistema confirma que la calificación se ha realizado correctamente.

Postcondiciones	La calificación del usuario queda almacenada y asociada al video correspondiente.
Requisitos relacionados	RF-IS 1

Caso de uso: Gestión de comentarios	
Actores	Usuario registrado, Usuario visitante
Descripción	Permite a los usuarios visualizar comentarios de un video organizados en estructura de árbol de respuestas y, en el caso de usuarios registrados, crear comentarios, responder a comentarios existentes, así como editar o eliminar sus propios comentarios.
Precondiciones	El usuario accede a la página de visualización de un video. Para comentar, responder, editar o eliminar comentarios es necesario haber iniciado sesión.
Flujo principal	1. El usuario accede a la sección de comentarios de un video. 2. El sistema muestra los comentarios organizados en estructura de árbol de respuestas. 3. El sistema ordena los comentarios mostrando primero los más populares. 4. El usuario registrado escribe un nuevo comentario o responde a un comentario existente. 5. El sistema registra el comentario o la respuesta. 6. El sistema actualiza la estructura de comentarios del video. 7. El usuario puede editar o eliminar sus propios comentarios. 8. El sistema actualiza el comentario editado o elimina el comentario seleccionado.
Flujo alternativo al paso 4: usuario no autenticado	
4a. Si el usuario no ha iniciado sesión, el sistema impide crear comentarios y respuestas. 4b. El sistema abre una ventana flotante para iniciar sesión.	
Flujo alternativo al paso 4: comentario inválido	
4a. Si el comentario no cumple las reglas de validación, como una longitud por encima de 512 caracteres, el sistema no permite seguir escribiendo y muestra un mensaje de error. 4b. El usuario corrige el contenido del comentario. 4c. El sistema vuelve a validar la información introducida.	
Flujo alternativo al paso 7: comentario no editable o no eliminable	

7a. Si el comentario no pertenece al usuario, el sistema impide su edición o eliminación. 7b. El sistema envía un mensaje de error y no hay cambios.	
Postcondiciones	Los comentarios del video muestran en orden descendente de popularidad y quedan organizados en estructura de árbol de respuestas. En su caso, se crean, actualizan o eliminan comentarios según las acciones del usuario.
Requisitos relacionados	RF-IS 2, RF-IS 3, RF-IS 4, RF-IS 5, RF-IS 6

Caso de uso: Donar puntos de experiencia	
Actores	Usuario registrado
Descripción	Permite a un usuario registrado donar puntos de experiencia a un canal dentro de la plataforma.
Precondiciones	El usuario ha iniciado sesión, accede a un canal y dispone de puntos de experiencia suficientes para realizar la donación.
Flujo principal	1. El usuario accede a la página de un canal. 2. El sistema muestra la opción de donar puntos (<i>Boost Channel</i>). 3. El usuario selecciona la cantidad de puntos que desea donar. 4. El usuario confirma la donación. 5. El sistema valida que el usuario dispone de puntos suficientes. 6. El sistema descuenta los puntos del saldo del usuario. 7. El sistema suma los puntos donados al canal correspondiente. 8. El sistema actualiza la cantidad de puntos donados por parte del usuario hacia el canal y cambia su posición en el ranking si es necesario.
Flujo alternativo al paso 3: cantidad inválida	
3a. Si la cantidad introducida no es válida, es decir, está fuera del rango permitido, el sistema muestra un mensaje de error. 3b. El usuario introduce una cantidad de puntos válida. 3c. Se vuelve a validar la cantidad introducida.	
Flujo alternativo al paso 4: puntos insuficientes	
4a. Si el usuario no dispone de puntos de experiencia suficientes, el sistema muestra un mensaje de error. 4b. El sistema impide continuar con la operación.	

Flujo alternativo al paso 7: cambio de nivel del canal	
7a. El sistema detecta que el canal ha ascendido de nivel. 7b. El sistema le muestra una alerta de cambio de nivel al usuario que donó. 7c. El sistema incluye una insignia debajo de la barra del canal durante 48 horas que indica el nuevo nivel alcanzado.	
Postcondiciones	La cantidad donada queda descontada del saldo del usuario y registrada a favor del canal correspondiente como puntos de Boost.
Requisitos relacionados	RF-IS 7 RF-GAM 9

5.4.6. Comunidades

1. Interactuar con comunidades
2. Consultar comunidades
3. Buscar y filtrar comunidades

Diagrama

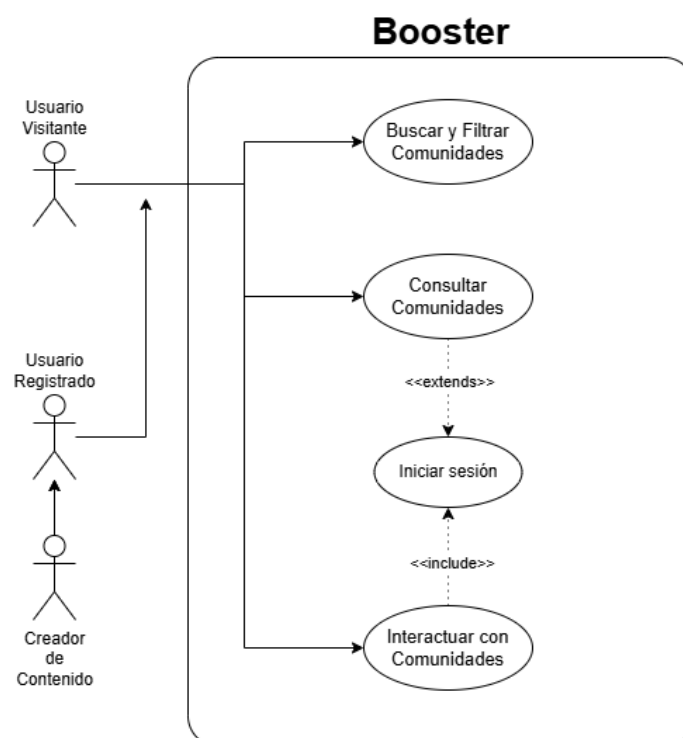


Figura 5.6: Comunidades - Casos de uso

Descripción

Caso de uso: Interactuar con comunidades	
Actores	Usuario registrado
Descripción	Permite a un usuario registrado crear comunidades, unirse a comunidades existentes y abandonarlas.
Precondiciones	El usuario ha iniciado sesión en la plataforma.

Flujo principal	1. El usuario accede a la sección de comunidades. 2. El usuario selecciona una acción: crear, unirse o abandonar una comunidad. 3. En caso de creación, el usuario introduce los datos de la comunidad. 4. El sistema valida la información introducida. 5. El sistema crea la comunidad o actualiza la pertenencia del usuario. 6. El sistema confirma que la operación se ha realizado correctamente. 7. El usuario accede a una comunidad y decide si unirse o abandonarla si la está siguiendo. 8. El sistema actualiza la cantidad de miembros de la comunidad y la relación del usuario con ella.
Flujo alternativo al paso 2: usuario sin sesión	
3a. El usuario no puede crear comunidades sin una sesión iniciada. 3b. El sistema lo informa y solicita al usuario que introduzca sus credenciales para iniciar sesión.	
Flujo alternativo al paso 3: datos inválidos en creación	
3a. Si los datos introducidos para la comunidad no cumplen las reglas de validación, como nombres largos, el sistema muestra un mensaje de error. 3b. El usuario corrige la información. 3c. El sistema vuelve a validar los datos.	
Postcondiciones	La comunidad es creada o la relación del usuario con la comunidad queda actualizada.
Requisitos relacionados	RF-COM 1, RF-COM 2, RF-COM 3

Caso de uso: Consultar comunidades	
Actores	Usuario registrado
Descripción	Permite al usuario consultar las comunidades a las que pertenece y visualizar el contenido asociado a ellas.
Precondiciones	El usuario ha iniciado sesión en la plataforma.

Flujo principal	1. El usuario accede a la pestaña de comunidades. 2. El sistema muestra las comunidades a las que pertenece el usuario junto con sus videos más recientes. 3. El usuario selecciona una comunidad. 4. El sistema muestra todos los videos asociados a una comunidad.
Flujo alternativo al paso 2: sin comunidades	
2a. Si el usuario no pertenece a ninguna comunidad, el sistema muestra las más populares. 2b. El sistema invita al usuario a explorar o unirse a comunidades.	
Postcondiciones	El usuario visualiza las comunidades a las que pertenece y su contenido reciente.
Requisitos relacionados	RF-COM 4, RF-COM 5

Caso de uso: Buscar y filtrar comunidades	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario buscar comunidades por nombre, descripción y aplicar filtros según las categorías.
Precondiciones	El usuario accede a la sección de comunidades.
Flujo principal	1. El sistema muestra las comunidades más destacadas y, en caso de que el usuario tenga una sesión activa, el sistema muestra comunidades relacionadas a sus intereses. 2. El usuario introduce un término de búsqueda o selecciona algunas categorías. 3. El sistema procesa la búsqueda o los filtros aplicados. 4. El sistema muestra las comunidades que coinciden con los criterios. 5. El usuario selecciona una comunidad para consultar su contenido.
Flujo alternativo al paso 3: búsqueda sin resultados	
3a. Si no existen comunidades que coincidan con el criterio de búsqueda o filtros aplicados, el sistema lo informa.	
Postcondiciones	El usuario obtiene una lista de comunidades según los criterios de búsqueda o filtros.
Requisitos relacionados	RF-COM 6, RF-COM 7

5.4.7. Canales

1. Gestionar canal
2. Consultar canal y canales seguidos
3. Interactuar con canales

Diagrama

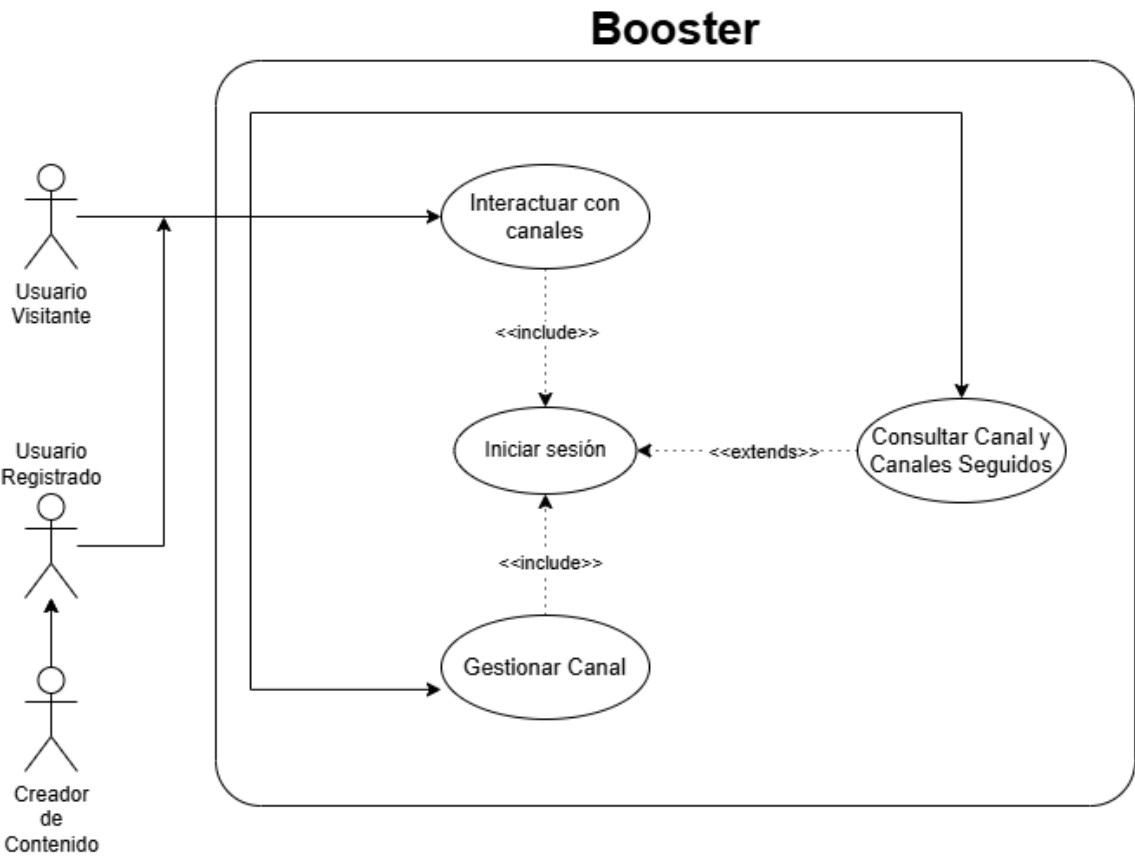


Figura 5.7: Canales - Casos de uso

Descripción

Caso de uso: Gestionar canal	
Actores	Usuario registrado
Descripción	Permite a un usuario registrado gestionar y personalizar su propio canal dentro de la plataforma.
Precondiciones	El usuario ha iniciado sesión en la plataforma.

Flujo principal	1. El usuario accede a la sección de su canal (<i>My Channel</i>). 2. El sistema muestra la información actual del canal. 3. El usuario selecciona la opción de personalizar canal. 4. El usuario modifica los elementos disponibles de personalización del canal. 5. El sistema valida los cambios introducidos. 6. El sistema guarda la nueva configuración del canal y confirma los cambios.
Flujo alternativo al paso 5: datos inválidos	
5a. Si los datos introducidos no cumplen las reglas de validación, como un nombre superior a 50 caracteres, el sistema muestra un mensaje de error. 5b. El usuario corrige la información señalada. 5c. El sistema vuelve a validar los cambios introducidos.	
Postcondiciones	La configuración y personalización del canal quedan actualizadas en el sistema.
Requisitos relacionados	RF-CAN 1, RF-CAN 7

Caso de uso: Consultar canal y canales seguidos	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario consultar la información de un canal, sus videos, ranking, recompensas desbloqueadas, descripción y, en el caso de usuarios registrados, los canales que sigue.
Precondiciones	El usuario accede a la página de un canal disponible en la plataforma. Para consultar los canales seguidos, el usuario debe haber iniciado sesión.

Flujo principal	1. El usuario accede a la página de un canal. 2. El sistema muestra la página principal del canal. 3. El usuario selecciona la pestaña de videos. 4. El sistema muestra los videos asociados al canal. 5. El usuario selecciona la pestaña de ranking. 6. El sistema muestra la clasificación de usuarios donantes del canal. 7. El usuario selecciona la pestaña de recompensas desbloqueadas. 8. El sistema muestra las recompensas desbloqueadas del canal. 9. El usuario selecciona la pestaña de información. 10. El sistema muestra la descripción e información del canal. 11. Si el usuario ha iniciado sesión y es seguidor del canal, puede acceder a la pestaña de chat comunitario. 12. El sistema muestra el chat comunitario del canal.
Flujo alternativo al paso 1: consultar canal propio	
1a. Si el usuario ha iniciado sesión, puede acceder a su propio canal desde su perfil. 1b. El sistema muestra la página del canal propio del usuario.	
Flujo alternativo al paso 1: consultar canales seguidos	
1a. Si el usuario ha iniciado sesión, puede acceder a la sección de canales seguidos (<i>Following</i>). 1b. El sistema muestra la lista de canales que sigue el usuario en sesión. 1c. El usuario selecciona uno de los canales seguidos. 1d. El sistema muestra la página del canal seleccionado.	
Postcondiciones	El usuario visualiza la información, contenido o canales seguidos solicitados.
Requisitos relacionados	RF-CAN 2, RF-CAN 3, RF-CAN 4, RF-CAN 5, RF-CAN 7, RF-CAN 10

Caso de uso: Interactuar con canales	
Actores	Usuario registrado
Descripción	Permite a un usuario registrado seguir canales, dejar de seguirlos y acceder al chat comunitario asociado a un canal.

Precondiciones	El usuario ha iniciado sesión en la plataforma y accede a un canal disponible.
Flujo principal	1. El usuario accede a la página de un canal. 2. El sistema muestra las opciones de interacción disponibles. 3. El usuario selecciona la opción de seguir canal (<i>Follow</i>). 4. El sistema registra la relación de seguimiento entre el usuario y el canal. 5. El usuario accede al chat comunitario del canal. 6. El sistema muestra el chat asociado. 7. El usuario puede dejar de seguir el canal. 8. El sistema elimina la relación de seguimiento entre el usuario y el canal.
Flujo alternativo al paso 3: canal ya seguido	
3a. Si el usuario ya sigue el canal, el sistema informa de ello. 3b. El sistema ofrece la opción de dejar de seguir el canal.	
Flujo alternativo al paso 5: acceso al chat no disponible	
5a. Si el chat comunitario no está disponible o el usuario no sigue al canal, el sistema muestra un mensaje informativo. 5b. El sistema no permite el acceso al chat comunitario, impidiendo al usuario leer mensajes anteriores y enviar nuevos mensajes.	
Postcondiciones	La relación de seguimiento del usuario con el canal queda actualizada y el usuario puede acceder al chat comunitario si está disponible.
Requisitos relacionados	RF-CAN 6, RF-CAN 8, RF-CAN 9

5.4.8. Gamificación

1. Ganar puntos de experiencia
2. Consultar puntos de boost y nivel de canal
3. Consultar rankings
4. Consultar tienda de recompensas
5. Comprar artículos de personalización
6. Equipar y consultar artículos de personalización

Diagrama

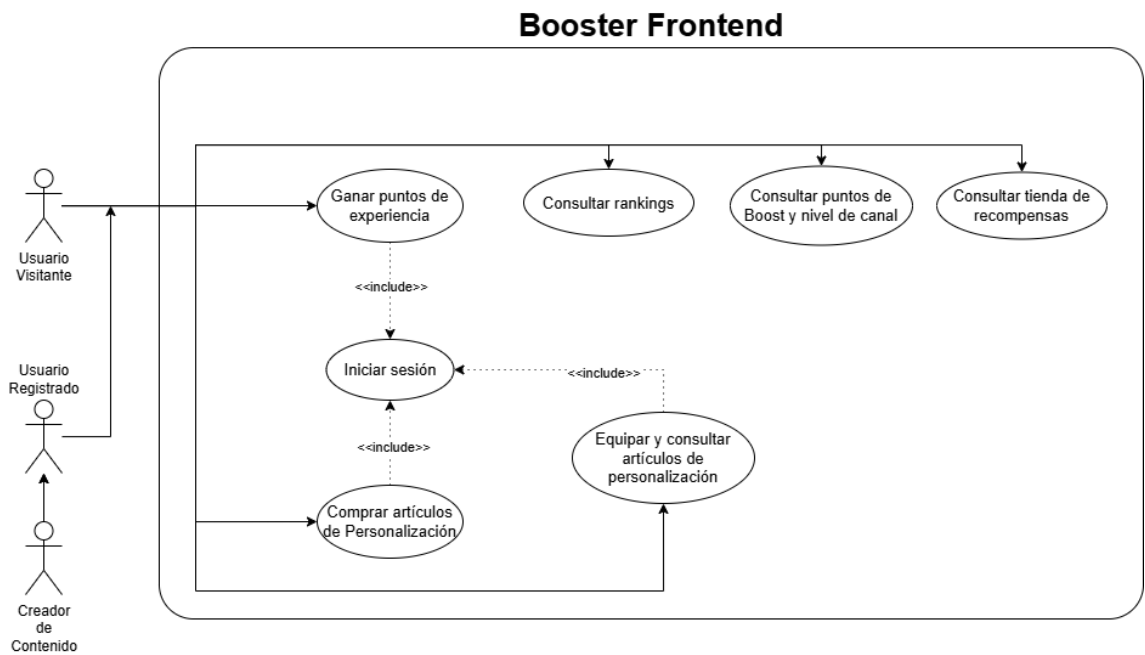


Figura 5.8: Gamificación- Casos de uso

Descripción

Caso de uso: Ganar puntos de experiencia	
Actores	Usuario registrado
Descripción	Permite al usuario ganar puntos de experiencia (XP) al visualizar anuncios.
Precondiciones	El usuario ha iniciado sesión en la plataforma.
Flujo principal	1. El usuario accede a un anuncio con recompensas. 2. El usuario visualiza el anuncio. 3. El sistema reproduce el anuncio. 4. El sistema verifica que el anuncio ha sido visualizado correctamente. 5. El sistema asigna los puntos de experiencia correspondientes al usuario. 6. El sistema actualiza el recuento total de XP del usuario. 7. El sistema incrementa el contador de visitas del anuncio.
Flujo alternativo al paso 3: anuncio no completado	
3a. Si el usuario abandona el anuncio antes de haber visto al menos 10 segundos o antes de la finalización del mismo, el sistema no otorga puntos de experiencia. 3b. El sistema informa de que la recompensa no ha sido concedida.	
Postcondiciones	El recuento de XP del usuario se actualiza.

Requisitos relacionados	RF-GAM 1, RF-GAM 2
--------------------------------	--------------------

Caso de uso: Consultar puntos de boost y nivel de canal	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario consultar los puntos de boost acumulados y el nivel asociado a un canal.
Precondiciones	El usuario accede a la página de un canal.
Flujo principal	1. El usuario accede a la página de un canal. 2. El sistema muestra los puntos de boost acumulados del canal junto con su barra de progreso. 3. El sistema muestra el nivel actual del canal y los puntos necesarios para alcanzar el siguiente nivel. 4. Si el canal ha ascendido de nivel recientemente, se incluye una insignia debajo de la barra del canal durante 48 horas que indica el nuevo nivel alcanzado.
Postcondiciones	El usuario visualiza el estado de progreso del canal.
Requisitos relacionados	RF-GAM 3, RF-GAM 9

Caso de uso: Consultar rankings	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario visualizar la clasificación de los canales más populares según métricas como número de seguidores y nivel.
Precondiciones	El usuario accede a la sección de <i>Top Channels</i> .
Flujo principal	1. El usuario accede a la sección de <i>Top Channels</i> y decide si filtrar por seguidores o por nivel. 2. El sistema obtiene los datos de los 100 mejores canales en la plataforma. 3. El sistema ordena los canales según seguidores o nivel. 4. El sistema muestra la clasificación resultante.
Postcondiciones	El usuario visualiza la clasificación de canales disponible.
Requisitos relacionados	RF-GAM 4

Caso de uso: Consultar tienda de recompensas	
Actores	Usuario visitante, Usuario registrado
Descripción	Permite al usuario ver los artículos disponibles en la tienda como recompensas, ítems, insignias, títulos, iconos, entre otras personalizaciones y beneficios.
Precondiciones	El usuario accede a la tienda.
Flujo principal	1. El usuario accede a la tienda. 2. El sistema muestra los artículos disponibles. 3. El usuario navega entre las opciones disponibles. 4. El sistema muestra la información detallada de cada artículo y su precio.
Postcondiciones	El usuario visualiza los artículos disponibles en la tienda.
Requisitos relacionados	RF-GAM 5

Caso de uso: Comprar artículos de personalización	
Actores	Usuario registrado
Descripción	Permite al usuario adquirir artículos de personalización utilizando puntos de experiencia.
Precondiciones	El usuario ha iniciado sesión y dispone de puntos de experiencia suficientes.
Flujo principal	1. El usuario accede a la tienda. 2. El usuario selecciona un artículo. 3. El sistema muestra el coste del artículo. 4. El usuario confirma la compra. 5. El sistema valida que dispone de suficientes puntos. 6. El sistema descuenta los puntos correspondientes y guarda la transacción. 7. El sistema añade el artículo al inventario del usuario.
Flujo alternativo al paso 5: puntos insuficientes	
5a. Si el usuario no dispone de suficientes puntos, el sistema muestra un mensaje de error. 5b. El sistema impide completar la compra y le ofrece alternativas para obtener más puntos como completar misiones, mirar anuncios voluntarios, comprarlos.	
Postcondiciones	El artículo queda registrado en el inventario del usuario.

5.4. Casos de uso

Requisitos relacionados	RF-GAM 6, RF-GAM 7
--------------------------------	--------------------

Caso de uso: Equipar y consultar artículos de personalización	
Actores	Usuario registrado
Descripción	Permite al usuario consultar su inventario de artículos adquiridos y equiparlos en su perfil.
Precondiciones	El usuario ha iniciado sesión y dispone de artículos en su inventario.
Flujo principal	1. El usuario accede a su inventario. 2. El sistema muestra los artículos adquiridos. 3. El usuario selecciona un artículo. 4. El usuario decide equipar el artículo. 5. El sistema actualiza la apariencia del perfil del usuario.
Flujo alternativo al paso 1: inventario vacío	
1a. Si el usuario no dispone de artículos, el sistema informa de ello.	
Postcondiciones	El perfil del usuario refleja los artículos equipados y sus elementos visuales asociados.
Requisitos relacionados	RF-GAM 7, RF-GAM 8

5.5. Diagramas de secuencia

En esta sección se recopilan algunos de los diagramas de secuencia más importantes en la aplicación final.

5.5.1. Carga de videos

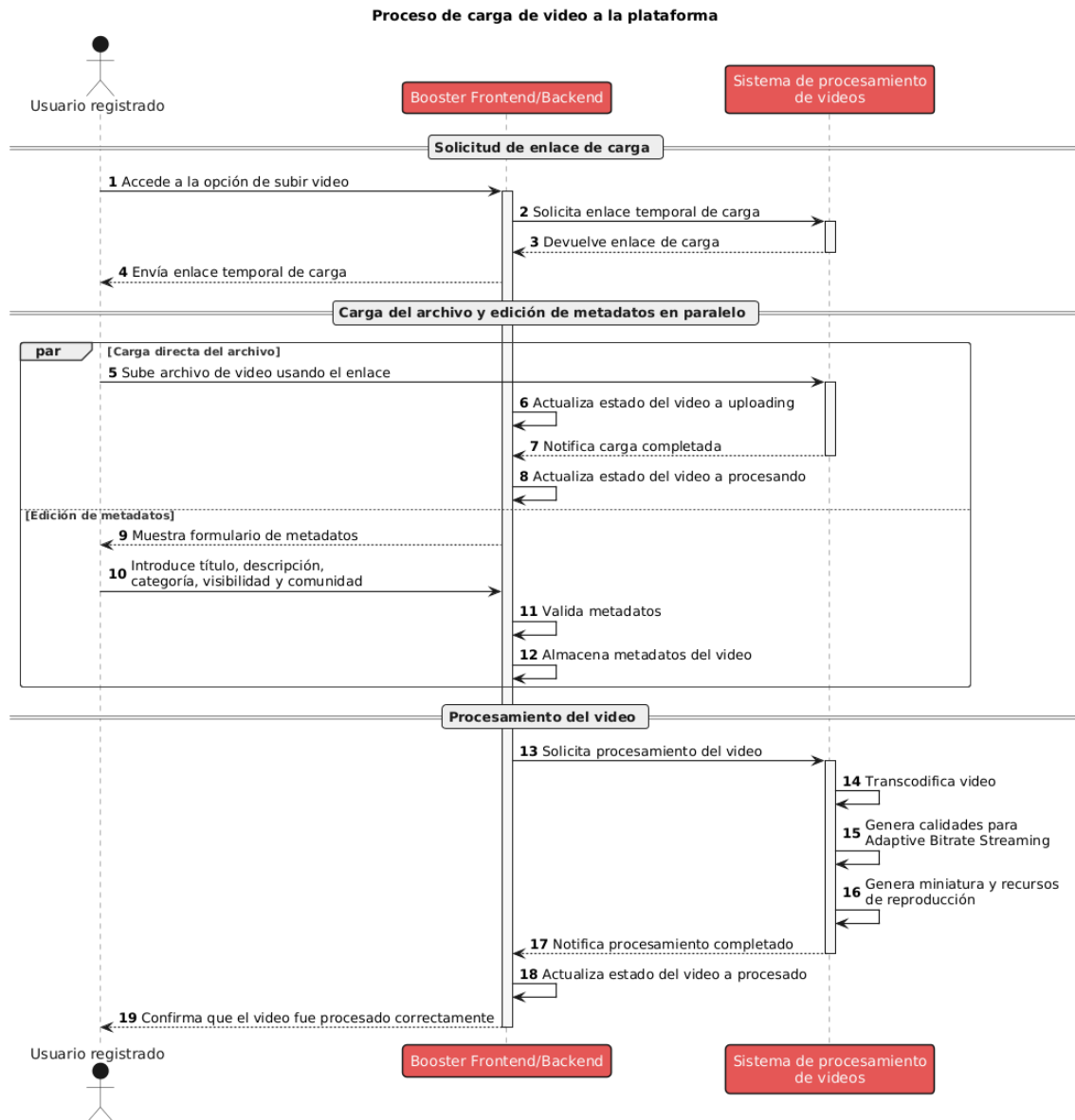


Figura 5.9: Carga de Video - Diagrama de Secuencia

La figura 5.9 muestra el proceso de carga de un video a la plataforma. Este comienza cuando un usuario registrado accede a la sección de creadores (*studio*) y selecciona la opción de subir un video. A continuación, el sistema le solicita al Sistema de Procesamiento de Videos un enlace temporal de carga para que

5.5. Diagramas de secuencia

el usuario suba el archivo directamente a este servicio sin pasar por el servidor de la plataforma. De esta forma, el archivo estaría solamente en el sistema de procesamiento y se evita sobrecargar el servidor de la plataforma con archivos pesados. Cuando se obtiene el enlace de carga, el sistema le indica al usuario que suba el archivo de video mediante un formulario. A su vez, mientras el sistema de procesamiento se encarga de recibir el archivo, procesarlo, almacenarlo y transcodificarlo, el usuario puede editar los metadatos del video, como título, descripción, visibilidad, entre otros. Una vez que el sistema de procesamiento finaliza con el procesamiento del video, se lo notifica al sistema de backend de la plataforma y el video pasa a estar disponible para su reproducción.

5.5.2. Registro

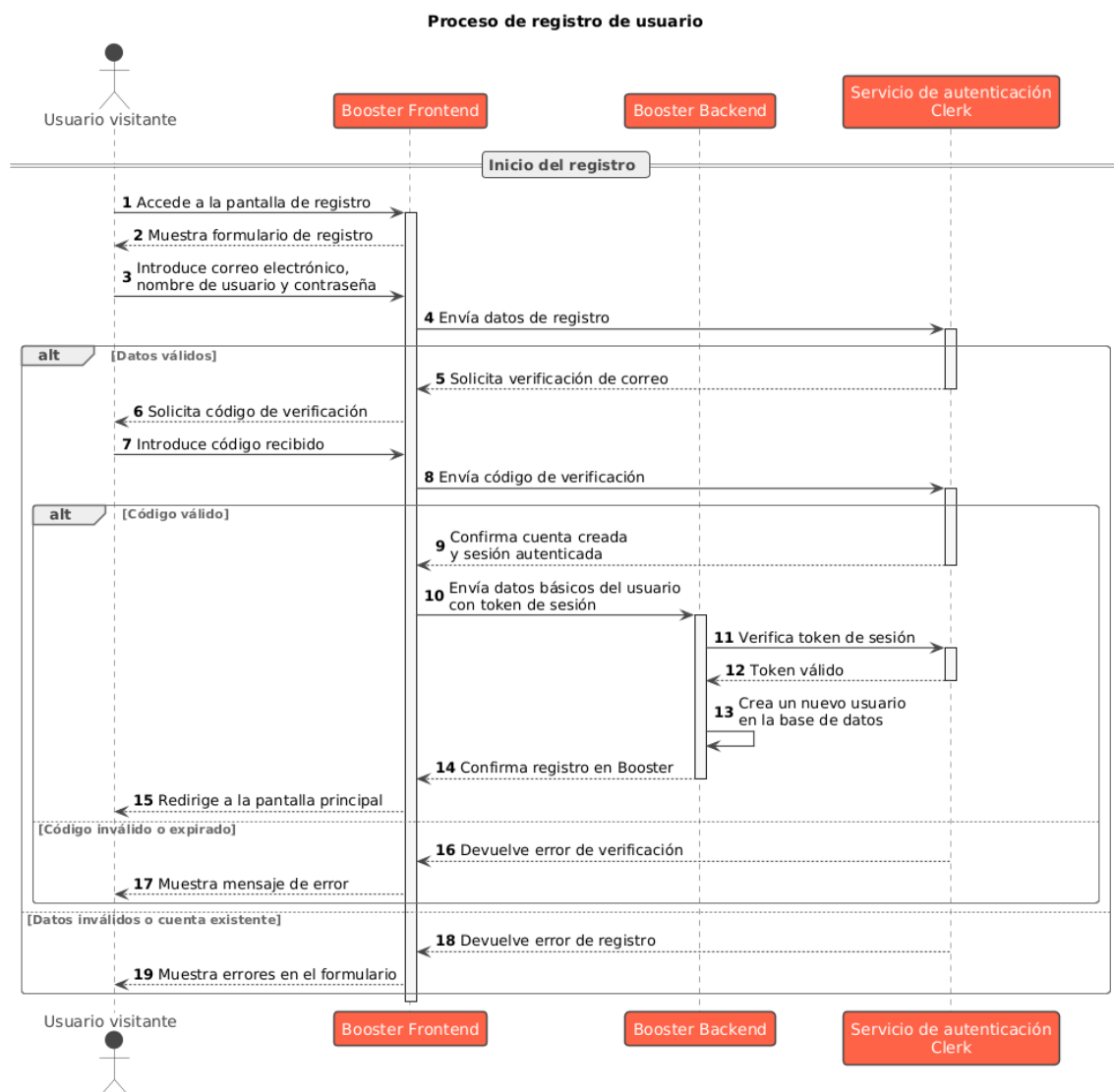


Figura 5.10: Registro - Diagrama de Secuencia

Capítulo 5. Análisis

La figura 5.10 muestra el proceso de registro de un nuevo usuario en la plataforma. El proceso lo inicia un usuario visitante que accede al formulario de registro. Aquí, introduce sus datos personales requerido como nombre de usuario, correo electrónico y contraseña. Posteriormente, el sistema valida la información introducida, verificando que el nombre de usuario y correo no estén ya registrados y que la contraseña cumpla con las norma de seguridad. A continuación, el sistema envía un correo de verificación al correo y espera a que el usuario confirme su cuenta. Cuando se completan estos pasos, se crea un nuevo usuario en el sistema y se crea un nuevo canal.

5.5.3. Inicio de sesión

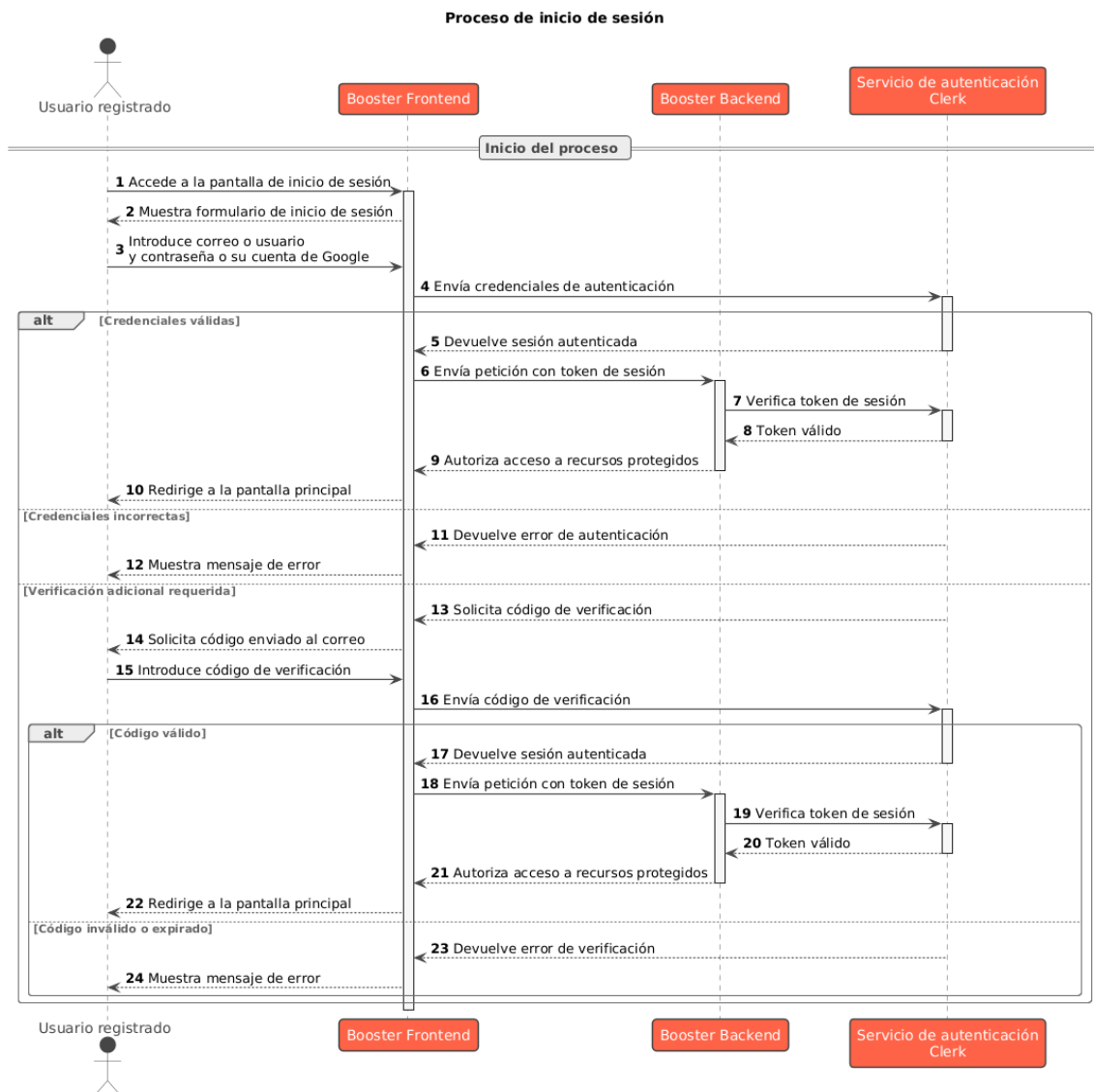


Figura 5.11: Inicio de sesión - Diagrama de Secuencia

5.5. Diagramas de secuencia

El inicio de sesión, representado en la figura 5.11 sigue un flujo similar al registro. En este caso, el que inicia la acción es un usuario con una cuenta previamente registrada en el sistema. El usuario accede al formulario de inicio de sesión e introduce sus credenciales, que pueden ser su nombre de usuario o correo electrónico junto con su contraseña. El sistema valida las credenciales introducidas, verificando que correspondan a una cuenta existente y que la contraseña sea correcta. En algunos casos, el sistema puede requerir una autenticación adicional, solicitando un código de verificación al correo registrado. Esto ocurre cuando se detecta un inicio de sesión desde un dispositivo o ubicación no habitual o, el usuario lleva un tiempo sin iniciar sesión.

Si la validación es exitosa, el sistema inicia una nueva sesión para el usuario y le permite acceder a las funcionalidades disponibles para usuarios registrados. En caso de que las credenciales sean incorrectas, el sistema muestra un mensaje de error e invita al usuario a intentarlo nuevamente o a recuperar su contraseña en caso de olvido.

5.5.4. Vista de Video

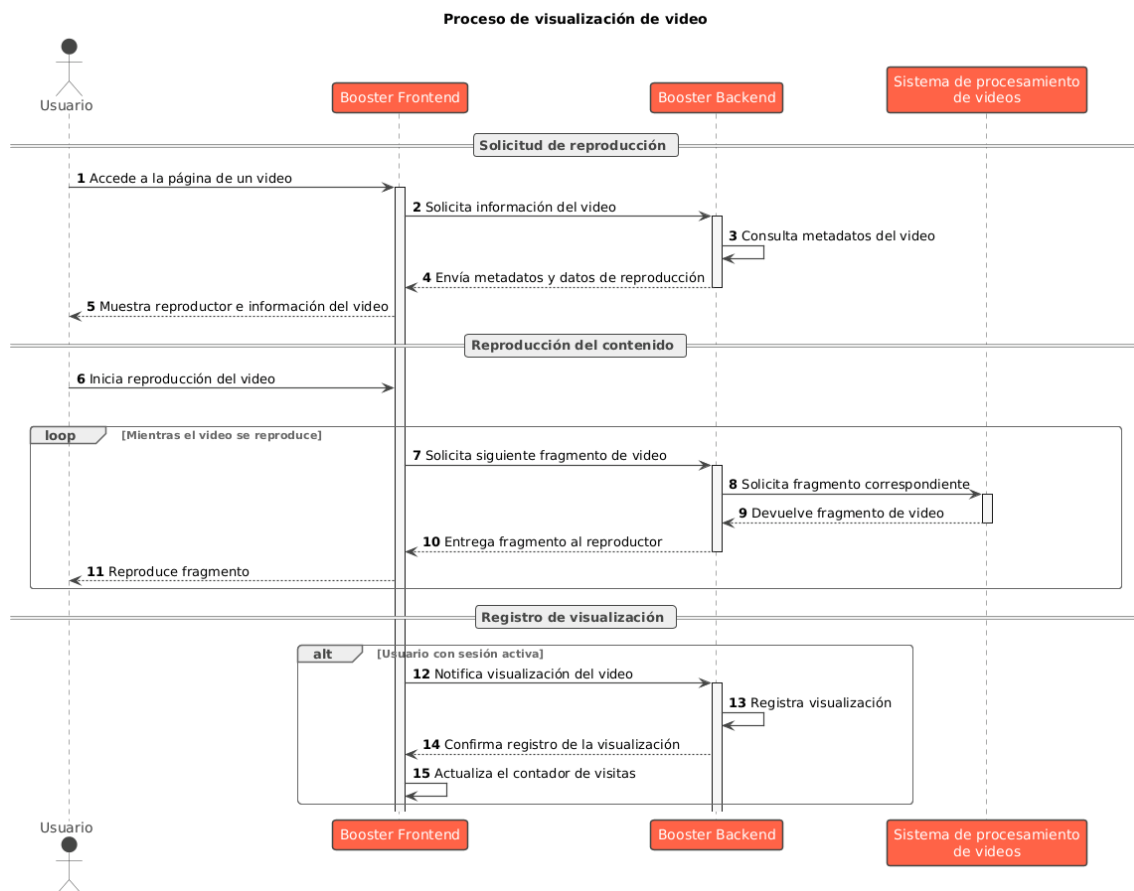


Figura 5.12: Vista de Video - Diagrama de Secuencia

La figura 5.12 detalla el proceso que se sigue cuando un usuario accede a un video para visualizarlo. Como se puede observar, el sistema de backend va solicitando fragmentos del video conforme avanza la reproducción. Además, el sistema de backend se encarga de actualizar la cantidad de visitas del video si el usuario ha iniciado sesión y notifica al sistema de frontend para que actualice la información en la página del video.

5.6. Diagrama Conceptual de la Base de Datos

En esta sección se presenta un diagrama conceptual de la base de datos, siguiendo el modelo entidad relación de Chen. Como el diagrama completo es muy extenso, se ha dividido en varias partes y se han omitido algunos atributos para facilitar su lectura. Más adelante en la sección de diseño de la base de datos se presenta la información completa de las tablas y sus relaciones.

En primer lugar, con respecto a la publicación de contenido se tiene lo siguiente:

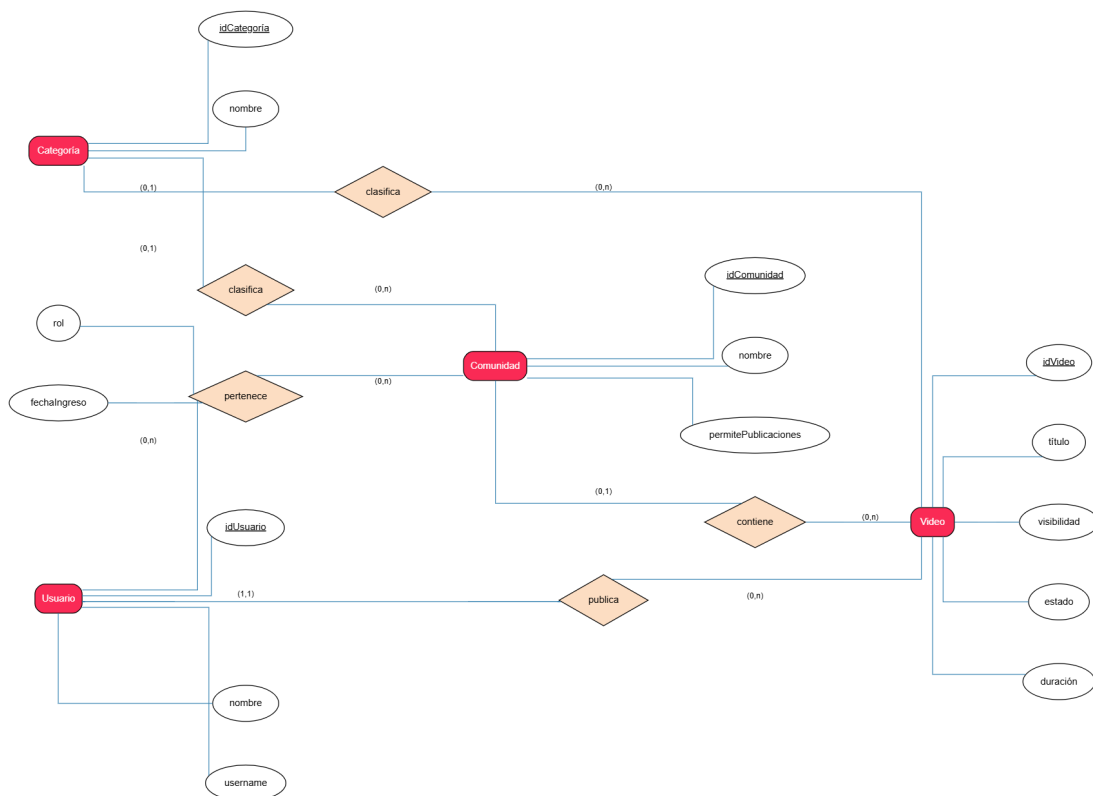


Figura 5.13: Diagrama Entidad Relación - Comunidades, Usuarios y Videos

En 5.13 se observa el diagrama conceptual de datos que relaciona a las entidades usuario, comunidad, videos y categoría. Estas relaciones permiten cargar un video a la plataforma y relacionan el contenido. Como se puede ver, cualquier usuario puede publicar una cantidad arbitraria de videos y estos, solo pueden pertenecer a un único usuario. Es decir, no existen videos que no tengan un

5.6. Diagrama Conceptual de la Base de Datos

propietario o autor. Por otro lado, al publicar un video se puede seleccionar opcionalmente una categoría disponible en el sistema. De la misma forma, un video se puede asociar a como mucho una sola comunidad, pero una comunidad, puede tener un gran número de videos asociados.

Una comunidad es creada por un usuario pero puede pertenecer a varios usuarios denominados "moderadores", los cuales se pueden ir añadiendo. Por lo tanto, si una comunidad no permite publicar contenido, rechazará todos los videos que no hayan sido creador por alguno de los moderadores de la comunidad.

El atributo de *visibilidad* de un video, determina si este es público o no en la plataforma. Esto quiere decir que, en caso de que la visibilidad sea privada, el algoritmo de recomendación no tendrá en cuenta este video para mostrarlo a los usuarios. Además, mientras el estado no llegue a *procesado*, se seguirá ignorando por los algoritmos que sugieren contenido.

Por otro lado, con respecto a las interacciones sociales se tiene el siguiente diagrama:

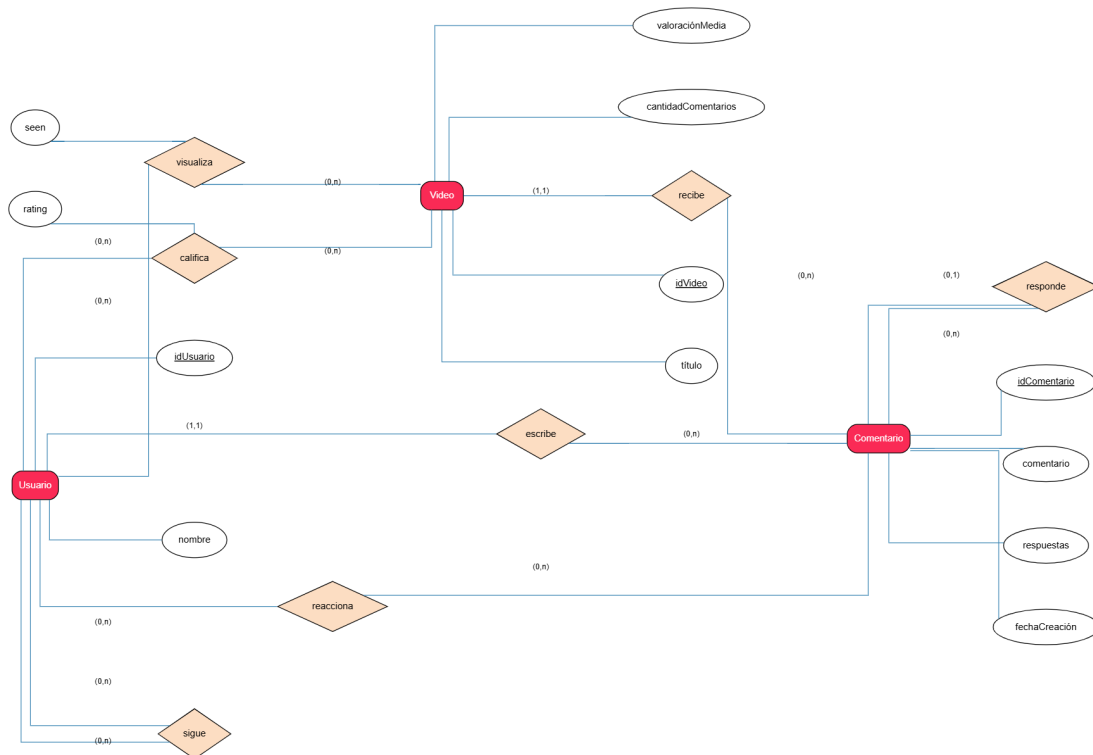


Figura 5.14: Diagrama Entidad Relación - Interacción Social

En 5.14, se describen las relaciones entre las entidades usuario, video y comentarios que corresponden a interacciones sociales dentro de la plataforma. Como se puede observar, un usuario es capaz de visualizar un número arbitrario de videos a lo largo del tiempo en la plataforma. Además, un video, puede ser visto por muchos usuarios. Para dibujar gráficas con estadísticas de visitas, se debe

conocer cuándo se ha visto un video. Además, un usuario puede calificar cada video en una escala del 1 al 5 y estos, pueden tener un gran número de calificaciones que determinan la valoración media del contenido. Cabe destacar que un usuario solo puede dejar a lo sumo una calificación en un video. Es decir, no se puede calificar el mismo video 2 veces seguidas, pero es posible modificar o eliminar la valoración.

Asimismo, un usuario puede crear un comentario en un video. Cada usuario puede escribir un número indefinido de comentarios pero estos solo pueden tener un único autor. No existen comentarios anónimos, siempre se debe conocer el usuario que lo creó. Además, cada uno de estos debe estar asociado a exactamente un video. Es decir, no existen comentarios que no tengan un video o que estén repetidos en varios. Sin embargo, un video no tiene límite superior de comentarios ni otras restricciones adicionales.

Otro aspecto importante a destacar es que, un comentario puede tener muchas respuestas modelado mediante un atributo respuestas que mantiene la cuenta de estas. Sin embargo, un comentario solo puede responder como máximo a un único mensaje padre.

5.6. Diagrama Conceptual de la Base de Datos

Por último, en cuanto a la gamificación y la moderación del contenido se obtiene el siguiente diagrama:

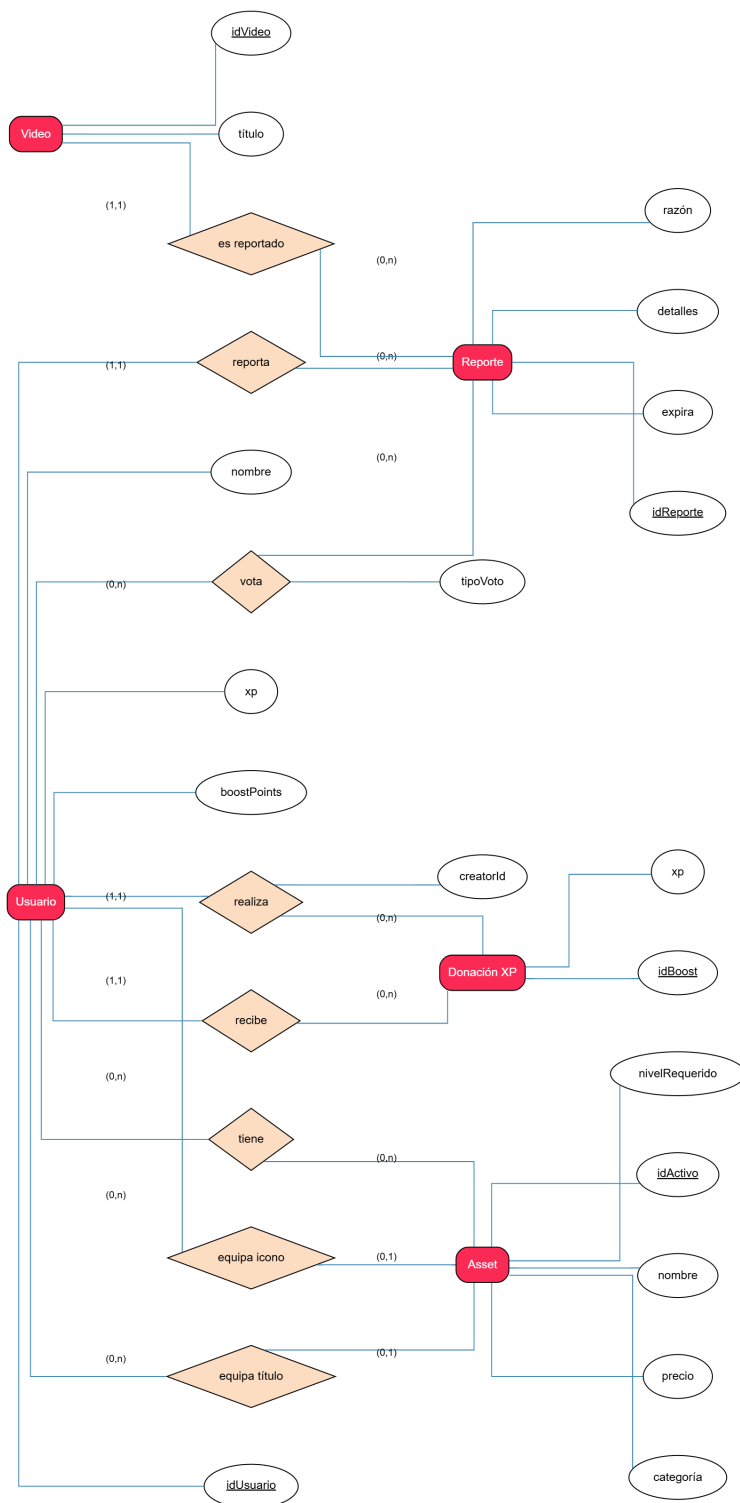


Figura 5.15: Diagrama Entidad Relación - Gamificación y Moderación

En 5.15, se describen las relaciones entre las entidades video, reporte, usuario, donación XP y asset. Estas detallan el vínculo que tienen las acciones relativas a la moderación de contenido y otros aspectos de gamificación como la donación de puntos de experiencia o la compra de artículos.

Como se puede observar en la parte superior del gráfico, un video puede tener un número indefinido de reportes pero un reporte, siempre estará asociado a un único video. Además, este reporte siempre será creado por un usuario quien puede crear un varios reportes. Además, los usuarios son capaces de votar en una denuncia indicando su tipo de voto: en contra o a favor.

Más abajo, se ve que un usuario puede realizar 0 donaciones o las que quiera hacia otro canal. Para esto, se necesita saber el identificador del canal que recibe la donación de puntos. Además, un usuario puede recibir un número indefinido de donaciones.

Por otro lado, un usuario puede equiparse un número arbitrario de assets o títulos y cada título puede pertenecer a muchos usuarios. Además, cada uno de estos tiene un precio y en algunos casos, un nivel requerido para desbloquearlo.

Capítulo 6

Diseño y Arquitectura

6.1. Patrones y mejores prácticas

En esta sección se describen los patrones de diseño y las mejores prácticas empleadas en el desarrollo de *Booster*. El objetivo es analizar las decisiones de diseño adoptadas que permitieron crear una aplicación modular, mantenible, segura y escalable.

6.1.1. Mono-Repositorio con Next.js

Se ha optado por utilizar un mono-repositorio ¹ mediante el framework Next.js, que permite una gestión centralizada del código. Además, la integración del *Frontend* con el *Backend* se realiza de forma más fluida gracias a las capacidades que ofrece Next.js. Ambos componentes están en el mismo repositorio, mejorando así la experiencia de desarrollo y despliegue. A continuación, se detalla la estructura de archivos del proyecto.

¹Se entiende por mono-repositorio a aquel proyecto software donde se utiliza un único repositorio para almacenar el código. Es decir, es un repositorio que contiene el código de todos los módulos de la aplicación que, aunque están relacionados, podrían funcionar de forma independiente.

6.1.2. Estructura general de archivos

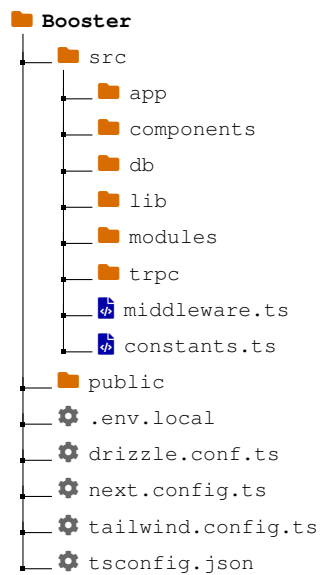


Figura 6.1: Estructura general del proyecto Booster

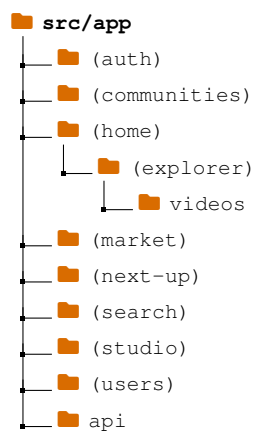


Figura 6.2: Estructura de rutas de la aplicación - Directorio /src/app

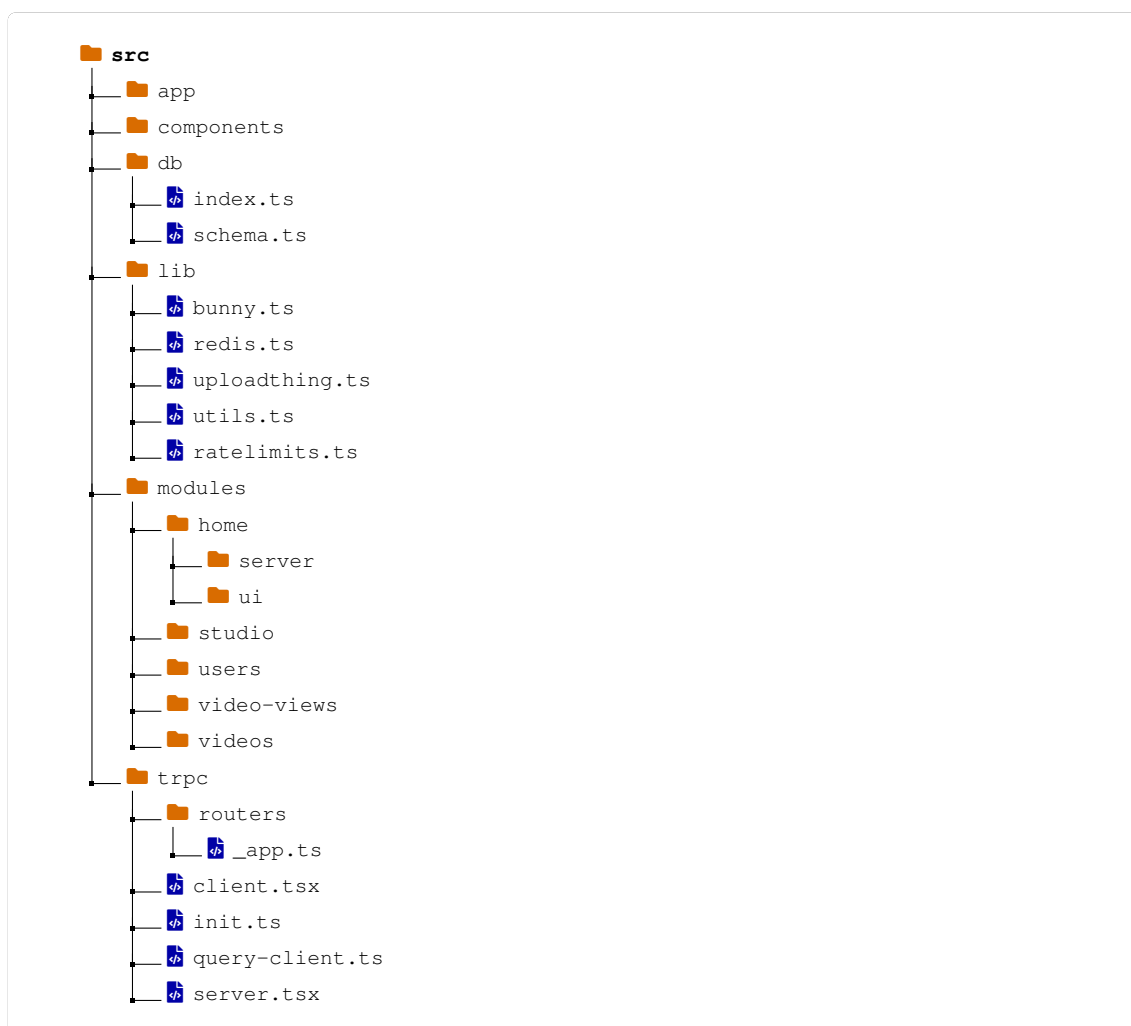


Figura 6.3: Estructura del backend y frontend

En la figura 6.1 se observa la estructura de la aplicación. El directorio `/src` contiene el código fuente de la aplicación y se divide en varias subcarpetas. Por otra parte, el directorio `/public` contiene los archivos estáticos que se sirven directamente al cliente. En *Booster* se ha utilizado exclusivamente para almacenar imágenes como logos de la aplicación.

En la raíz del proyecto se encuentran además archivos de configuración. El archivo `.env.local` se utiliza para almacenar variables de entorno y `api-keys` fundamentales para el correcto funcionamiento de los proveedores y para mejorar la seguridad. Otros archivos como `drizzle.config.ts`, `next.config.ts`, `tailwind.config.ts` y `tsconfig.json` contienen configuraciones específicas para Drizzle, Next.js, Tailwind CSS y TypeScript respectivamente.

Las figuras 6.2 y 6.3 detallan el contenido de la organización de rutas de la aplicación y de los componentes del *backend* respectivamente.

6.1.3. Separación de funcionalidades y Arquitectura modular

El proyecto emplea el principio de separación de funcionalidades dividiendo en capas los módulos con distinto propósito. En la figura 6.3 se observan diferentes directorios donde cada uno cumple un objetivo específico:

- **/db:** Se encarga de la gestión de la base de datos *NeonDB* y su ORM *Drizzle-ORM*.
- **/components:** Recopila componentes de la interfaz de usuario que se reutilizan a lo largo de toda la aplicación.
- **/lib:** Contiene archivos de configuración de servicios externos. Además, incorpora funciones de utilidades (*utils.ts*) que implementan funciones específicas empleadas en distintas partes del proyecto. También incluye un archivo *ratelimits.ts* que especifica los límites de uso de la aplicación para aliviar la carga en el servidor y proteger a la aplicación.
- **/modules:** Contiene las implementaciones tanto del *Frontend* como del *Backend* de los distintos componentes de la aplicación. De esta forma, se consigue una **arquitectura modular**, dividiendo cada elemento, como son el *studio*, *home*, *videos*, *video-view*, *users*, en componentes independientes, cada uno con una funcionalidad específica. Esta decisión facilita la mantenibilidad del código debido a que cada módulo concentra la lógica de una parte concreta de la aplicación. Por tanto, cada funcionalidad puede evolucionar de forma independiente. Dentro de cada uno de estos directorios se encuentra un subdirectorio */ui* que implementa la interfaz de usuario de este componente y su interacción con el *backend*. Además, existe otro subdirectorio */server* que se encarga de gestionar la lógica de negocio relativa a ese componente. Normalmente, en esta sección es donde se crean, recuperan, actualizan y eliminan registros en la base de datos mediante procedimientos implementados en un archivo *procedures.ts*. Por ejemplo, el */server* de *home* se encarga de recuperar la información necesaria para mostrar en la página principal, como la lista de videos recomendados.
- **/trpc:** Contiene los archivos de configuración de tRPC para establecer una comunicación entre el cliente y el servidor.

6.1.4. Rutas agrupadas en Next.js

En el directorio */src/app*, como se observa en la figura 6.2, se utilizan grupos de rutas mediante paréntesis, como *(auth)* y *(home)*. Este método permite organizar la lógica de las rutas sin afectar directamente la URL final. De esta forma, distintos componentes que pertenezcan a una misma sección se pueden agrupar según su propósito funcional sin alterar el comportamiento de la aplicación. Esta es una buena práctica recomendada en la documentación de Next.js [31].

6.1.5. Acceso a datos mediante ORM

La interacción con la base de datos se realiza mediante un ORM, más concretamente con *Drizzle-ORM* como se analizó en 3.1.3. Dentro de `/src/app/db` se encuentra un archivo `schema.ts` que permite definir el esquema de datos, tablas y relaciones de forma tipada con TypeScript, facilitando así la integración con la aplicación.

6.1.6. Esquemas de validación

Una buena práctica empleada en el desarrollo de *Booster* consiste en la utilización de esquemas de validación para controlar los datos que se manejan en la aplicación. Antes de que se ejecute la lógica del sistema, se comprueba que los datos cumplen con ciertas restricciones. Mediante la utilización de una herramienta conocida como **Zod**, se realiza esta validación, permitiendo reducir los errores y las entradas inválidas.

```
1 getFollowersByUserId: baseProcedure
2   .input(z.object({ userId: z.string().uuid() }))
3   .query(async ({ ctx, input }) => {
4     const { userId: creatorId } = input;
5     const { clerkUserId } = ctx;
```

Este es un fragmento del código del procedimiento `getFollowersByUserId` dentro del archivo `/src/modules/follows/server/procedure.ts` que comprueba que el identificador recibido por el procedimiento sea de tipo `uuid` y `string` antes de realizar la operación.

6.1.7. Centralización de integraciones externas

Los archivos de configuración de servicios externos se encuentran todos en `/src/lib` para mejorar la organización del código y facilitar su mantenibilidad. Estos ficheros contienen información de conexión con los servicios de terceros y utilizan claves de API alojadas en el archivo `.env.local`

6.1.8. Seguridad mediante variables de entorno

El proyecto utiliza variables de entorno para almacenar información sensible. Se almacenan en el archivo `.env.local`, el cual contiene claves API, secretos de webhooks, claves públicas, claves privadas, credenciales, entre otros. Este archivo no se incluye en el repositorio público y solo es conocido por el servicio de *hosting*.

6.1.9. Middleware para control de accesos

Cada solicitud hacia una ruta específica del *backend* pasa primero por un middleware, cuya implementación se encuentra en el fichero `middleware.ts`. En *Booster* esto permite proteger rutas y comprobar la sesión del usuario antes de ejecutar determinadas acciones. Gracias a esto, se evita reutilizar la misma lógica de validación en varias partes.

6.1.10. Reutilización de componentes visuales

Al emplear React.js (3.2.1) para la interfaz de usuario se pueden crear componentes reutilizables. Estos se almacenan en la carpeta `/components`, permitiendo así manejar una apariencia visual coherente en toda la aplicación y evitar repetir el mismo código en múltiples partes. Además, esta decisión facilita la evolución del diseño de la aplicación. Cuando se desea actualizar un elemento visual, se modifica el componente y los cambios quedan reflejados en todas las partes donde se utilice.

6.1.11. Prefetch de datos

En *Booster*, se utiliza tRPC para optimizar la obtención de datos mediante técnicas de precarga. Esto permite recuperar la información desde el lado del servidor antes de que los componentes del cliente la necesiten, como se analizó en 3.1.4.

En esta implementación, los componentes del servidor encargados de la precarga (*prefetch*) se encuentran dentro del directorio `/src/app`. Aquí, se ejecutan procedimientos tRPC y se preparan los datos correspondientes que posteriormente serán consumidos por el cliente. A continuación, los componentes cliente, organizados en `/src/modules`, acceden a esos datos disponibles en la caché y, por tanto, se reducen las solicitudes al servidor. Esto mejora la experiencia de usuario y disminuye los tiempos de carga, generando así una navegación más fluida.

Además, mientras se realiza este proceso, se emplean elementos visuales de carga como *Skeletons*. En caso de error, se muestra una indicación visual adecuada.

6.1.12. Convención de nombres y estructura

El proyecto utiliza una estructura de carpetas clara y con nombres descriptivos para los módulos y las secciones. Esto facilita la navegación dentro del código y la mantenibilidad del mismo.

Asimismo, las variables dentro del proyecto siguen la convención *camelCase* y emplean nombres descriptivos.

6.1.13. Procedimientos públicos y protegidos

En *Booster* se implementan dos tipos de procedimientos para interactuar con el servidor: público y privado. Un procedimiento público denotado como *baseProcedure* puede ser consumido sin requerir una sesión autenticada. Por otro lado, un procedimiento privado (*protectedProcedure*) requiere verificar la identidad del usuario antes de permitir la ejecución de una acción.

Esto es útil para implementar mecanismos que requieran identificar previamente al usuario y, además, evitar repetir el mismo código en distintos lugares. Basta con definir una vez las comprobaciones de sesión dentro de *protectedProcedure* y tRPC se encarga de ejecutarlas en cada procedimiento protegido.

6.2. Diseño de la Autenticación

Como se analizó en 3.1.6, *Booster* utiliza *Clerk*, un proveedor externo para la gestión de usuarios dentro de la aplicación. Además, se siguieron las recomendaciones y los pasos de integración descritos por la documentación oficial de *Clerk* para Next.js [32]. A continuación, se detallan los elementos más relevantes a tener en cuenta para la integración de este servicio de autenticación.

6.2.1. Middleware

La protección de rutas se implementa mediante el uso de un middleware identificado por *Clerk* como `clerkMiddleware()` y encontrado en el archivo `middleware.ts`. Esto permite interceptar solicitudes y comprobar si el usuario tiene una sesión válida antes de acceder a distintas partes de la aplicación.

Esto se emplea para proteger rutas que requieren un inicio de sesión previo del usuario como es el `/studio`. El `/studio` es la sección donde un creador de contenido puede gestionar sus videos; por tanto, solo tiene sentido con una sesión activa del usuario.

A continuación, se muestran fragmentos del archivo `middleware.ts`.

```

1 import { clerkMiddleware, createRouteMatcher } from '@clerk/nextjs/server';
2
3 const isProtectedRoute = createRouteMatcher([
4   "/studio(.*)",
5 ])
6
7 export default clerkMiddleware(async (auth, req) => {
8   if(isProtectedRoute(req)) await auth.protect();
9 });
10
11 export const config = {
12   matcher: [
13     // Skip Next.js internals and all
14     // static files, unless found in search params
15     '/((?!_next|[^?]*\\.(?:html?|css|js(?!on)|jpe?g|webp|png
16     |gif|svg|ttf|woff2?|csv|docx?|xlsx?|zip|webmanifest|xml)).*)',
17
18     // Always run for API routes
19     '/(api|trpc)(.*)',
20   ],
21 };

```

Aquí se observa cómo se implementa *Clerk* para aplicar reglas de autenticación antes de permitir el acceso a ciertas rutas. Se importa `clerkMiddleware`, que integra el sistema de autenticación de Clerk en Next.js, y `createRouteMatcher` para definir patrones de rutas a las cuales aplicar reglas.

La constante `isProtectedRoute` define como ruta protegida solo a `/studio(.*)`. Esto significa que cualquier acceso a una ruta que comience por `/studio` activará la lógica de protección de rutas, que en este caso, requiere una sesión activa del usuario.

Dentro de `clerkMiddleware()`, se comprueba si la solicitud entrante coincide con una ruta protegida mediante `if(isProtectedRoute(req))`. Si la condición se cumple, se ejecuta `auth.protect()`, que impide el acceso a usuarios no autenticados. De esta forma, la lógica de protección no tiene que repetirse dentro de cada página.

Finalmente, la propiedad `matcher` indica sobre qué solicitudes debe ejecutarse el middleware. La primera regla evita aplicar el middleware sobre archivos internos de Next.js y archivos estáticos, como imágenes, hojas de estilo, fuentes o documentos. La segunda regla hace que el middleware también se ejecute sobre rutas `/api` y `/trpc`. Es importante notar que esto no hace que todas estas rutas queden automáticamente protegidas. En cambio, esto solo indica que un acceso a ellas invoca una llamada al middleware. La protección se aplica cuando la ruta coincide con lo definido en `isProtectedRoute`, porque es el único caso donde se ejecuta `auth.protect()`.

6.2.2. Usuario Clerk vs Usuario Interno

En la aplicación se debe coordinar la identificación de un usuario gestionado por *Clerk* con su correspondiente usuario almacenado en la base de datos interna. Por esa razón, cada usuario de la tabla `users` contiene un identificador interno (*id*) y un atributo *clerkId*. El *id* se usa en la aplicación para identificar al usuario y realizar consultas con la base de datos, funcionando como clave primaria de la entidad usuario, mientras que *clerkId* se utiliza para asociar el usuario identificado por *Clerk* al usuario interno.

Esto permite que la aplicación tenga su propio modelo de datos sin depender del proveedor externo.

6.2.3. Creación y Sincronización de Usuarios

Cuando se crea un nuevo usuario en la aplicación mediante un registro, se modifican sus datos o se eliminan, se hace mediante los servicios de *Clerk*. Sin embargo, como también se deben reflejar los cambios en la base de datos interna, se utiliza un *webhook* para sincronizar los datos. Este *webhook* se encuentra definido en la ruta `/src/app/api/users/webhook/route.ts` y recibe eventos enviados por *Clerk* cuando se crea, actualiza o elimina un usuario. El campo *clerkId* es enviado por *Clerk* en estas peticiones para identificar al usuario modificado y actúa como vínculo entre ambos sistemas.

6.3. Diseño de la Base de Datos

En la sección 5.6, se presentaron los diagramas conceptuales de la base de datos. Aquí, se detallan las decisiones de diseño de la misma junto con las características de las tablas teniendo en consideración sus atributos y las restricciones.

El diagrama entidad relación con todos los atributos, relaciones, claves prima-

6.3. Diseño de la Base de Datos

rias y foráneas se encuentra accesible en este enlace ². En la siguiente figura se observa una captura de pantalla del sitio web que genera el diagrama a partir del esquema definido en `/src/db/schema.ts`

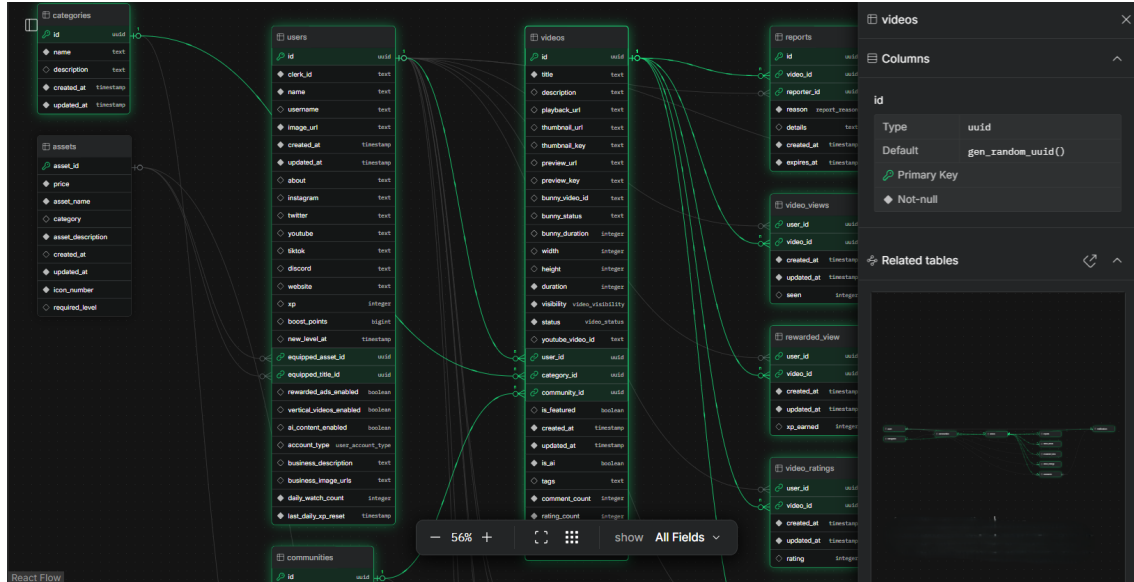


Figura 6.4: Diagrama Entidad Relación - LiamERD

Los datos siempre están almacenados en tablas y el esquema está adaptado a un sistema *PostgreSQL*. Además, para identificar una entidad se emplean claves primarias y para establecer relaciones se utilizan claves foráneas.

6.3.1. Tablas

En esta sección se describen los tipos de datos que almacenan cada una de las tablas junto con su descripción y las restricciones consideradas.

Usuarios				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	id	Identificador del usuario.	primary key, default random	uuid
Claves foráneas	equipped_asset_id	Identificador del asset equipado por el usuario.	foreign key, nullable	uuid
	equipped_title_id	Identificador del título equipado por el usuario.	foreign key, nullable	uuid
Atributos	clerk_id	Identificador del usuario en Clerk.	unique, not null	text
	name	Nombre del usuario.	not null	text

²https://liambx.com/erd/p/github.com/SamC4r/Booster/blob/main/src/db/schema.ts?showMode=ALL_FIELDS&hidden=eJwNy9ENwzAIBcCFvBMimCRIBSwejttrtm_-7yLbThNsYMFwBvhRD0n2H9Y88pxZ3Fga6bCmtSnmdTtJHozG-i9YuuRnvPDI2SD5sjj_6QyX5

Capítulo 6. Diseño y Arquitectura

Usuarios				
	Nombre en BD	Descripción	Restricciones	Dominio
	username	Nombre de usuario utilizado dentro de la plataforma.	nullable	text
	image_url	URL de la imagen de perfil del usuario.	not null	text
	created_at	Momento de creación del usuario.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización de los datos del usuario.	default now, not null	timestamp
	about	Descripción personal o biografía breve del usuario.	nullable	text
	instagram	Enlace al de Instagram del usuario.	nullable	text
	twitter	Enlace al perfil de Twitter del usuario.	nullable	text
	youtube	Enlace al canal de YouTube del usuario.	nullable	text
	tiktok	Enlace al perfil de TikTok del usuario.	nullable	text
	discord	Enlace al perfil de Discord del usuario.	nullable	text
	website	Enlace al sitio web del usuario.	nullable	text
	xp	Cantidad de puntos de experiencia acumulados por el usuario.	default 0	integer
	boost_points	Cantidad de puntos donados hacia el usuario.	default 0	bigint
	new_level_at	Momento en el que el usuario alcanzó un nuevo nivel.	nullable	timestamp
	rewarded_ads_enabled	Indica si el usuario habilita los anuncios voluntarios recompensados.	default false	boolean
	vertical_videos_enabled	Indica si el usuario habilita los videos en formato vertical.	default true	boolean
	ai_content_enabled	Indica si el usuario habilita el contenido hecho por IA en la plataforma.	default true	boolean

Cuadro 6.1: Descripción de la tabla users

6.3. Diseño de la Base de Datos

Videos				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	id	Identificador único del video dentro del sistema.	primary key, default random	uuid
Claves foráneas	user_id	Identificador del usuario creador del video.	foreign key, not null, on delete cascade, on update cascade	uuid
	category_id	Identificador de la categoría asociada al video.	foreign key, nullable, on delete set null	uuid
	community_id	Identificador de la comunidad asociada al video.	foreign key, nullable, on delete set null	uuid
Atributos	title	Título del video.	not null	text
	description	Descripción del video.	nullable	text
	playback_url	URL utilizada para reproducir el video.	nullable	text
	thumbnail_url	URL de la imagen miniatura del video.	nullable	text
	thumbnail_key	Clave interna asociada a la miniatura del video.	nullable	text
	preview_url	URL de la vista previa del video.	nullable	text
	preview_key	Clave interna asociada a la vista previa del video.	nullable	text
	bunny_video_id	Identificador del video dentro del servicio externo de procesamiento y almacenamiento de video bunny.net.	unique, nullable	text
	bunny_status	Estado del video dentro del servicio de bunny.net.	nullable	text
	bunny_duration	Duración registrada por el servicio externo de video bunny.net.	nullable	integer
	width	Anchura del video en píxeles.	nullable	integer
	height	Altura del video en píxeles.	nullable	integer
	duration	Duración total del video en segundos.	default 0, not null	integer
	visibility	Visibilidad del video dentro de la plataforma (público o privado).	default private, not null	enum video_visibility

Videos				
	Nombre en BD	Descripción	Restricciones	Dominio
	status	Estado interno del procesamiento del video (processing, uploading, processed, completed, error, finished).	default processing, not null	enum video_status
	is_featured	Indica si el video es destacado.	default false, nullable	boolean
	created_at	Momento de creación del video.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización del video.	default now, not null	timestamp
	is_ai	Indica si el video contiene o ha sido generado con contenido de inteligencia artificial.	default false, not null	boolean
	tags	Etiquetas asociadas al video.	nullable	text[]
	comment_count	Cantidad total de comentarios asociados al video.	default 0, not null	integer
	rating_count	Cantidad total de calificaciones recibidas por el video.	default 0, not null	integer
	average_rating	Promedio de todas las calificaciones recibidas por el video.	default 0, not null	real

Cuadro 6.2: Descripción de la tabla videos

Las comunidades pueden ser creadas por cualquier usuario. Además, un usuario puede pertenecer a varias y esto se modela mediante una tabla *Miembros de Comunidad* llamada *community_members* que relaciona a un usuario con una comunidad y su rol en ella (miembro, moderador, administrador).

Comunidades				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	id	Identificador de la comunidad.	primary key, default random	uuid
Claves foráneas	category_id	Identificador de la categoría asociada a la comunidad.	foreign key, nullable	uuid
Atributos	name	Nombre de la comunidad.	unique, not null	text
	description_short	Descripción breve de la comunidad.	nullable	text

6.3. Diseño de la Base de Datos

Comunidades				
	Nombre en BD	Descripción	Restricciones	Dominio
	description_long	Descripción larga de la comunidad.	nullable	text
	banner_url	URL de la imagen del banner de la comunidad.	nullable	text
	icon_url	URL del icono de la comunidad.	nullable	text
	rules	Reglas de la comunidad.	nullable	text
	is_private	Indica si la comunidad es privada. Solo los moderadores pueden ver y crear contenido en caso afirmativo. Por defecto el único moderador es el creador.	default false, not null	boolean
	allow_user_posts	Indica si los usuarios pueden publicar contenido en la comunidad. En caso negativo, solo los moderadores pueden asociar contenido a ella. Por defecto el único moderador es el creador.	default true, not null	boolean
	created_at	Momento de creación de la comunidad.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización de la comunidad.	default now, not null	timestamp

Cuadro 6.3: Descripción de la tabla communities

Miembros de comunidades				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	community_id, user_id	Identificador compuesto que relaciona a un usuario con una comunidad. De esta forma, no hay 2 registros con el mismo id de comunidad y de usuario. Es decir, un usuario no puede pertenecer dos veces a la misma comunidad.	primary key	uuid, uuid
Claves foráneas	community_id	Identificador de la comunidad a la que pertenece el usuario.	foreign key, not null, on delete cascade	uuid

Capítulo 6. Diseño y Arquitectura

Miembros de comunidades				
	Nombre en BD	Descripción	Restricciones	Dominio
	user_id	Identificador del usuario miembro de la comunidad.	foreign key, not null, on delete cascade	uuid
Atributos	role	Rol del usuario dentro de la comunidad (miembro, moderador, administrador).	default member, not null	text
	joined_at	Momento en el que el usuario se unió a la comunidad.	default now, not null	timestamp

Cuadro 6.4: Descripción de la tabla community_members

Actualmente, en *Booster* existe un número de fijo de categorías que fueron añadidas manualmente por los administradores del sistema. Sin embargo, se considera como posible ampliación, permitir a los usuarios crear sus propias categorías.

Categorías				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	id	Identificador de la categoría.	primary key, default random	uuid
Claves foráneas	Ninguna			
Atributos	name	Nombre de la categoría.	unique, not null	text
	description	Descripción de la categoría.	nullable	text
	created_at	Momento de creación de la categoría.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización de la categoría.	default now, not null	timestamp

Cuadro 6.5: Descripción de la tabla categories

De la misma forma que con el caso de las categorías, los *assets* en la plataforma solo son añadidos por administradores del sistema de forma manual.

Assets				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	asset_id	Identificador único del asset.	primary key, default random	uuid
Claves foráneas	Ninguna			
Atributos	price	Precio del asset dentro de la plataforma.	default 0, not null	integer

6.3. Diseño de la Base de Datos

Assets				
	Nombre en BD	Descripción	Restricciones	Dominio
	asset_name	Nombre del asset.	not null	text
	category	Categoría a la que pertenece el asset (icono, insignia, perk).	nullable	text
	asset_description	Descripción del asset.	not null	text
	created_at	Momento de creación del asset.	default now	timestamp
	updated_at	Fecha y hora de última actualización del asset.	default now, not null	timestamp
	icon_number	Número del icono asociado al asset. En el <i>frontend</i> se definen íconos como componentes React para algunos assets y por tanto se necesita saber cuál aplicar en cada caso.	default 0, not null	integer
	required_level	Nivel requerido para desbloquear el asset.	default 0	integer

Cuadro 6.6: Descripción de la tabla assets

Como un usuario puede comprar un número indefinido de assets, se genera la siguiente tabla que modela la relación entre asset y usuario.

Assets de usuarios				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	asset_id, user_id	Identificador compuesto que relaciona un asset con un usuario. Un usuario no puede comprar el mismo asset 2 veces.	primary key	uuid, uuid
Claves foráneas	asset_id	Identificador del asset adquirido por el usuario.	foreign key, not null, on delete cascade, on update cascade	uuid
	user_id	Identificador del usuario dueño del asset.	foreign key, not null, on delete cascade, on update cascade	uuid
Atributos	No hay más atributos.			

Cuadro 6.7: Descripción de la tabla user_assets

Un usuario también podía seguir a otro usuario y por tanto, se genera esta tabla llamada

Capítulo 6. Diseño y Arquitectura

user_follows que registra los seguimientos de un usuario a otro.

Follows de usuarios				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	user_id, creator_id	Identificador compuesto que representa la relación de seguimiento entre un usuario y un creador. Un usuario no puede seguir 2 veces al mismo creador.	primary key	uuid, uuid
Claves foráneas	user_id	Identificador del usuario seguidor.	foreign key, not null, on delete cascade	uuid
	creator_id	Identificador del usuario creador que es seguido.	foreign key, not null, on delete cascade	uuid
Atributos	created_at	Momento de creación de la relación de seguimiento (follow).	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización de la relación de seguimiento.	default now, not null	timestamp

Cuadro 6.8: Descripción de la tabla *user_follows*

Además, otra interacción que puede realizar un usuario sobre otro usuario creador de contenido es donar puntos de experiencia. Esto genera la tabla *boost_transactions* que mantiene un registro de esta acción. Aquí, cabe destacar que se permite a un usuario hacer varias donaciones sobre un mismo creador.

Transacciones de boost				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	boost_id	Identificador de la transacción de boost (donación de puntos de experiencia). En este caso no se utiliza un identificador compuesto para permitir varias donaciones de un mismo usuario sobre un canal.	primary key, default random	uuid
Claves foráneas	creator_id	Identificador del usuario creador que recibe los puntos de experiencia.	foreign key, not null, on delete cascade	uuid
	booster_id	Identificador del usuario que dona los puntos de experiencia.	foreign key, not null, on delete cascade	uuid

6.3. Diseño de la Base de Datos

Transacciones de boost				
	Nombre en BD	Descripción	Restricciones	Dominio
Atributos	xp	Cantidad de puntos de experiencia donados.	not null	integer

Cuadro 6.9: Descripción de la tabla boost_transactions

Al visualizar un video se debe mantener un registro del usuario que lo vio para tener un contador de visitas. Además, este puede incluir una calificación al video y enviar comentarios, por tanto es necesario modelar estas relaciones y guardar estas acciones. Por estas razones surgen las siguientes tablas que modelan las visitas de los videos, las calificaciones dadas y los comentarios recibidos por un video.

Visualizaciones de videos				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	user_id, video_id	Identificador compuesto que relaciona un usuario con un video visualizado. Un usuario puede ver varias veces el video pero en la implementación elegida, se utiliza un identificador compuesto y si el mismo usuario intenta visualizar el contenido otra vez, se incrementa un contador. Otra opción sería utilizar un identificador de la visualización de video.	primary key	uuid, uuid
Claves foráneas	user_id	Identificador del usuario que visualizó el video.	foreign key, not null, on delete cascade	uuid
	video_id	Identificador del video visualizado por el usuario.	foreign key, not null, on delete cascade	uuid
Atributos	created_at	Momento de creación del registro de visualización.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización del registro de visualización.	default now, not null	timestamp
	seen	Cantidad de veces que el usuario ha visualizado el video. Se incrementa si la última visualización del usuario almacenada en updated_at es superior a 12.	default 1	integer

Cuadro 6.10: Descripción de la tabla video_views

Capítulo 6. Diseño y Arquitectura

Calificaciones de videos				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	user_id, video_id	Identificador compuesto que relaciona un usuario con un video calificado. Un usuario no puede calificar dos veces el mismo video.	primary key	uuid, uuid
Claves foráneas	user_id	Identificador del usuario que calificó el video.	foreign key, not null, on delete cascade	uuid
	video_id	Identificador del video calificado por el usuario.	foreign key, not null, on delete cascade	uuid
Atributos	created_at	Momento de creación de la calificación.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización del registro de calificación.	default now, not null	timestamp
	rating	Calificación asignada por el usuario al video en una escala del 1 al 5.	nullable	integer

Cuadro 6.11: Descripción de la tabla video_ratings

Comentarios				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	comment_id	Identificador único del comentario.	primary key, default random	uuid
Claves foráneas	user_id	Identificador del usuario que realiza el comentario.	foreign key, not null, on delete cascade	uuid
	video_id	Identificador del video al que pertenece el comentario.	foreign key, not null, on delete cascade	uuid
	parent_id	Identificador del comentario padre. Cuando el comentario no tiene respuesta, parent_id es igual a comment_id.	foreign key, nullable, on delete cascade	uuid
Atributos	comment	Contenido textual del comentario.	not null	text
	created_at	Momento de creación del comentario.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización del comentario.	default now, not null	timestamp

6.3. Diseño de la Base de Datos

Comentarios				
	Nombre en BD	Descripción	Restricciones	Dominio
	replies	Cantidad de respuestas asociadas al comentario.	default 0, not null	integer

Cuadro 6.12: Descripción de la tabla comments

Un usuario, puede reaccionar a un comentario dándole un *me gusta* y por tanto, se genera la siguiente tabla

Reacciones a comentarios				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	user_id, comment_id	Identificador compuesto que relaciona a un usuario con un comentario reaccionado.	primary key	uuid, uuid
Claves foráneas	comment_id	Identificador del comentario al que pertenece la reacción.	foreign key, not null, on delete cascade	uuid
	user_id	Identificador del usuario que realiza la reacción.	foreign key, not null, on delete cascade	uuid
Atributos	created_at	Momento de creación de la reacción.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización de la reacción.	default now, not null	timestamp

Cuadro 6.13: Descripción de la tabla comment_reactions

Un usuario puede recibir puntos de experiencia por visualizar videos marcados como destacados (featured), que normalmente son anuncios. Por esa razón, surge la tabla *rewarded_view* que mantiene un registro de las visitas que fueron recompensadas junto con la cantida de puntos de experiencia ganados por el usuario.

Capítulo 6. Diseño y Arquitectura

Visualizaciones recompensadas				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	user_id, video_id	Identificador compuesto que relaciona un usuario con un video recompensado visualizado. En este caso se permite que un usuario pueda conseguir recompensa varias veces sobre un mismo video. Sin embargo, esto solo es posible si se cumplen 24 horas desde la última visita recompensada sobre ese video.	primary key	uuid, uuid
Claves foráneas	user_id	Identificador del usuario que visualizó el video recompensado.	foreign key, not null, on delete cascade	uuid
	video_id	Identificador del video recompensado visualizado por el usuario.	foreign key, not null, on delete cascade	uuid
Atributos	created_at	Momento de creación de la visita recompensada.	default now, not null	timestamp
	updated_at	Fecha y hora de última actualización de la visita recompensada.	default now, not null	timestamp
	xp_earned	Cantidad de puntos de experiencia obtenidos por la visualización recompensada.	nullable	integer

Cuadro 6.14: Descripción de la tabla rewarded_view

Finalmente, un usuario puede crear una votación de moderación mediante un reporte o, votar sobre una denuncia activa. Esto se modela con las tablas *reports* y *report_votes*

Reportes				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	id	Identificador del reporte o denuncia.	primary key, default random	uuid
Claves foráneas	video_id	Identificador del video reportado.	foreign key, not null, on delete cascade	uuid
	reporter_id	Identificador del usuario que crea el reporte.	foreign key, nullable, on delete set null	uuid

6.3. Diseño de la Base de Datos

Reportes				
	Nombre en BD	Descripción	Restricciones	Dominio
Atributos	reason	Motivo por el cual se reporta el video. Estas razones vienen predefinidas en un <i>enum</i> llamado <i>report_reasons</i> que incluye: ai, clickbait, spam, entre otros.	not null	enum report_reason
	details	Información adicional asociada al reporte proporcionada por el usuario que lo crea.	nullable	text
	created_at	Momento de creación del reporte.	default now, not null	timestamp
	expires_at	Fecha y hora de expiración del reporte.	not null	timestamp

Cuadro 6.15: Descripción de la tabla reports

Votos de reportes				
	Nombre en BD	Descripción	Restricciones	Dominio
Clave primaria	report_id, user_id	Identificador compuesto que relaciona un reporte con el usuario que vota. No se puede votar 2 veces para el mismo reporte pero sí se puede modificar o eliminar el voto.	primary key	uuid, uuid
Claves foráneas	report_id	Identificador del reporte votado por el usuario.	foreign key, not null, on delete cascade	uuid
	user_id	Identificador del usuario que emite	foreign key, not null, on delete cascade	uuid
Atributos	vote_type	Tipo del voto emitido por el usuario. Puede ser <i>agree</i> o <i>disagree</i>	not null	text
	created_at	Momento en el que el usuario votó.	default now, not null	timestamp

Cuadro 6.16: Descripción de la tabla report_votes

Toda la información de las tablas se encuentra en el archivo `/src/db/schema.ts` donde se especifican estas relaciones y los *enums* empleados.

6.4. Arquitecturas de procesamiento de videos

En esta sección se presentan las alternativas posibles para desarrollar un servicio de procesamiento de videos. En primer lugar, se analiza una solución básica implementando una arquitectura de procesamiento propia y desde cero. Posteriormente, se estudia la solución empleada en la aplicación final mediante el uso de un proveedor externo como *Bunny.net* analizado en 2.2.3.

6.4.1. Arquitectura de procesamiento propia

Esta primera solución se basa en implementar un flujo de procesamiento de videos asíncrono mediante el uso de servicios de Amazon Web Services (2.2.1). Con esto, se consigue tener un registro del archivo de video y distribuirlo de forma eficiente, evitando que el *backend* realice estas tareas. El proceso se ilustra en la figura 6.5

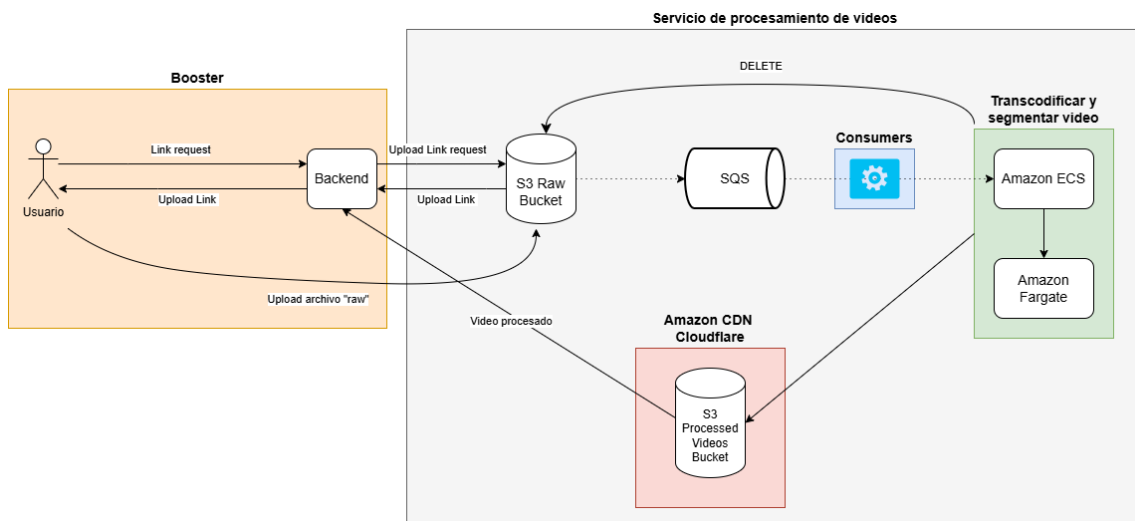


Figura 6.5: Arquitectura de Procesamiento con servicios de Amazon Web Services

El flujo comienza cuando el usuario solicita al *backend* un enlace para cargar un video. En lugar de enviar el video al *backend* y luego al servicio de procesamiento de videos, se envía un enlace de carga al usuario para que suba el video directamente al servicio de procesamiento. Por motivos de seguridad, este enlace de carga se pide a través del *backend*. Además, esta técnica reduce la carga sobre los servidores de *Booster* y evita que tenga que manejar archivos de tamaño elevado.

Posteriormente, el archivo original cargado por el usuario se almacena en un *bucket* de Amazon S3 denominado *S3 Raw Bucket* que funciona como un *Object Storage* explicado en 2.1.5. Este *bucket* almacena el contenido en su estado original antes de procesarlo. Una vez que el archivo se ha sido subido correctamente, S3 genera un evento indicando que se ha creado un nuevo objeto.

Ese evento se envía a una cola de Amazon SQS. Amazon SQS actúa como una cola de procesamiento analizada en 2.1.7. En vez de procesar el video de forma inmediata, el sistema registra un trabajo en la cola para que sea procesado y de esta forma, en momentos de alta demanda el sistema reaccionará mejor.

Los consumers son procesos que están continuamente consultando la cola SQS. Cuando detectan un nuevo mensaje, leen el evento generado por S3 y extraen la información necesaria para procesar el video, principalmente el nombre del bucket y la clave del

6.4. Arquitecturas de procesamiento de videos

archivo. Es importante destacar que estos consumers no transcodifican el video. En cambio, se encargan de identificar qué archivo debe procesarse y lanzan una tarea de procesamiento para realizar ese trabajo.

Para ejecutar el procesamiento, el consumer invoca Amazon ECS, el servicio de Amazon encargado de administrar la ejecución de contenedores. En este caso, el consumer lanza una tarea ECS y le pasa al contenedor las variables de entorno necesarias, como el identificador del video y el nombre del *bucket*. Con esa información, el contenedor encargado de procesar el video sabe exactamente qué archivo debe descargar y transformar y desde qué contenedor debe hacerlo.

Una vez que la tarea se inicia, el contenedor de procesamiento descarga el video original desde el S3 Raw Bucket. A continuación utiliza herramientas como *ffmpeg* (2.1.1) para transcodificar y procesar el video.

La segmentación convierte el video original en fragmentos para la reproducción en streaming. En la solución empleada, se generan segmentos del video en formato MPEG-DASH junto con un *manifest.mpd*. Además, la transcodificación produce diferentes calidades de reproducción como 360p, 720p y 1080p. También se generan miniaturas del video extrayendo un fotograma representativo mediante *ffmpeg*.

Cuando el proceso finaliza, los archivos generados se almacenan en un segundo *bucket* de S3, denominado *Processed Videos Bucket*. Aquí se guardan los recursos preparados para ser entregados a los clientes finales, como los manifiestos, miniaturas, segmentos y las distintas calidades de video generadas. Sobre este *bucket*, se utiliza Amazon CloudFront, la capa de distribución de contenido de Amazon, y se consiguen los beneficios analizados en 2.1.6. Asimismo, cuando el procesamiento de videos termina, se debe avisar al *backend* sobre esto, actualizar el estado del video, añadir la duración e incorporar la URL final del archivo procesado.

6.4.2. Arquitectura de procesamiento con un proveedor

Otra alternativa para implementar el procesamiento de videos es delegar estas tareas a un proveedor externo especializado. En lugar de que *Booster* tenga que descargar los archivos de videos, procesarlos, transcodificarlos, segmentarlos y almacenarlos en *buckets* propios, se ha decidido utilizar *Bunny.net*, más concretamente, el servicio de *Bunny Stream* (2.2.3). Este, se encarga de realizar todas las tareas de procesamiento de video, cuenta con una CDN que facilita su distribución y además, dispone de un gran número de opciones de personalización que se adaptan adecuadamente con los objetivos de la plataforma.

En la figura 6.6, se observa un diagrama de secuencias que ilustra el proceso de carga de videos a la plataforma integrando *Bunny.net*. El flujo comienza cuando un usuario selecciona la opción de subir video dentro del `/studio` y arrastra o selecciona un archivo de video. Antes de iniciar la carga, el cliente obtiene algunos metadatos del archivo como la duración y sus dimensiones. Esto se hace para evitar que se suban videos por encima de 10 minutos y así, reducir los costos de infraestructura. Además, se detectan el ancho y el alto para determinar si un video está en formato vertical. Esto se debe a que los usuarios tienen la opción de filtrar los videos verticales en su perfil de recomendaciones.

Capítulo 6. Diseño y Arquitectura

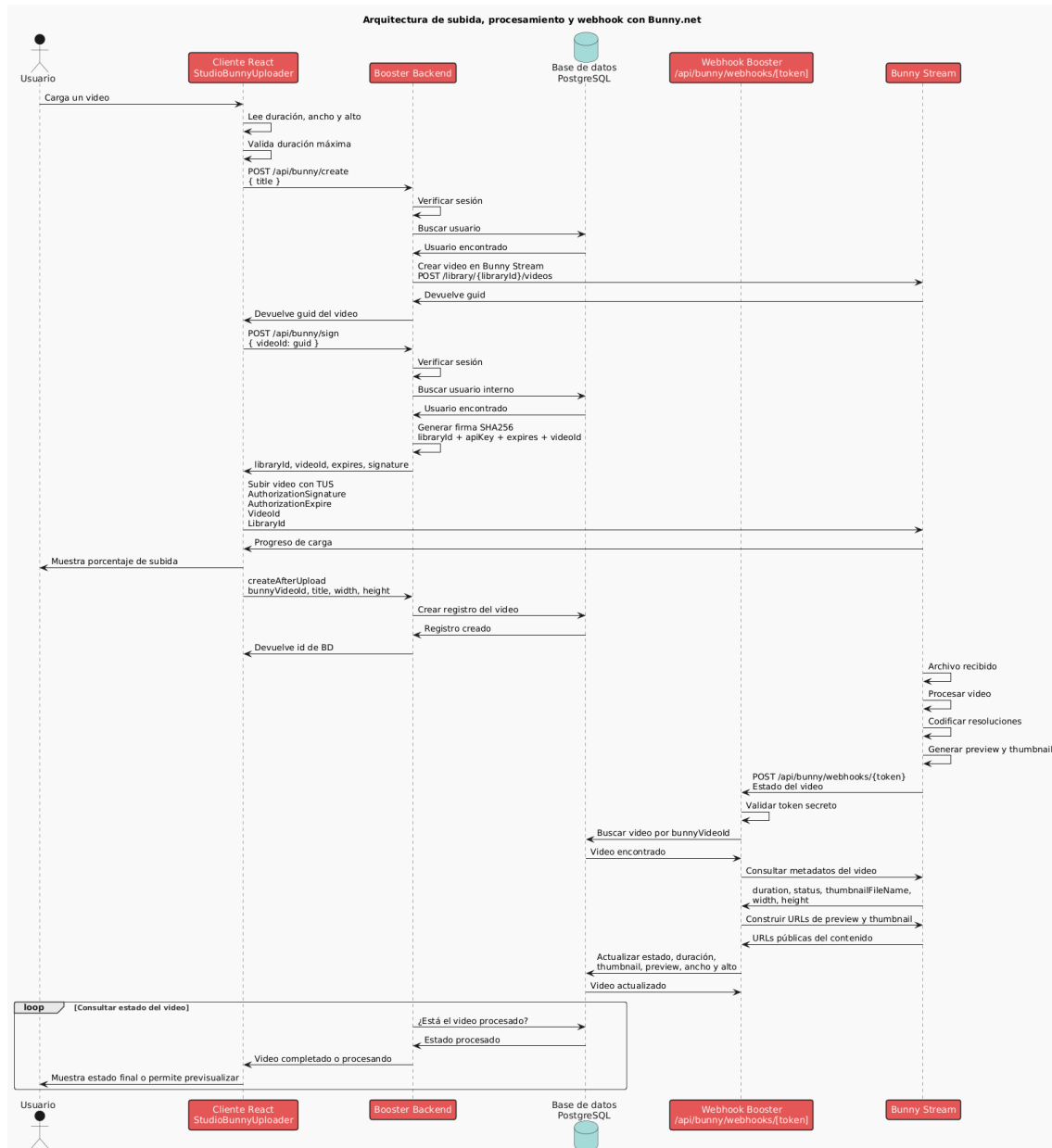


Figura 6.6: Arquitectura de Procesamiento con Bunny.net

Luego, el cliente llama al endpoint `/api/bunny/create` dentro del backend the *Booster* enviándole el título del video. Aquí, se verifica en primer lugar que el usuario tenga una sesión válida para cargar un archivo de video. Si no lo está, se rechaza la operación. Si todo es correcto, el backend realiza una petición a *Bunny Stream* para craer un nuevo objeto de video, según se especifica en su documentación [33]. Para esto, se debe realizar una petición *POST* autenticada de la siguiente forma, pasándole *Headers* como *Acces-Key* y *content-type*:

```

1 curl --request POST
2   --url https://video.bunnycdn.com/library/:libraryId/videos
3   --header 'AccessKey: <Your Stream Library API key>'
4   --header 'accept: application/json'
5   --header 'content-type: application/json'

```


6.4. Arquitecturas de procesamiento de videos

```
6 --data '{"title":"test"}'
```

Como se puede observar, esto requiere conocer el identificador de la librería de *Bunny* y además, la clave de API del mismo. Esta es información delicada que se debe recuperar del archivo `.env.local` destinado a almacenar este tipo de datos.

Este paso es importante porque *Bunny* necesita crear primero una entidad de video antes de recibir el archivo. Al crearlo, *Bunny* devuelve un identificador de video, denominado `guid`, que es una referencia al video dentro del proveedor del servicio. En *Booster*, ese valor se almacena en la base de datos para asociar el video gestionado por *Bunny* con su correspondiente en la plataforma.

Posteriormente, según la documentación, se podría enviar una petición *PUT* a la referencia externa del video recién creado con su *guid* de la siguiente forma:

```
1 curl --request PUT
2 --url <https://video.bunnycdn.com/library/><Your Library ID>/videos/<GUID>
3 --header 'AccessKey: <Your Stream Library API key>'
4 --header 'accept: application/json'
5 --data-binary '@path/to/your/video.mp4'
```

Con esto, el servidor devolvería el estado de la operación al finalizar y el archivo quedaría cargado y disponible en el servicio externo. Sin embargo, en *Booster* se utiliza el *TUS Resumable Uploads* de *Bunny Stream* [34]. La ventaja principal de *TUS* es que la subida puede reanudarse si se interrumpe. El cliente puede consultar subidas previas y continuar desde donde se quedó. Esto lo convierte en una solución más robusta en comparación con una subida HTTP normal, donde una interrupción obliga a reiniciar la carga. Esto lo hace ideal para evitar errores y malas experiencias de usuario con archivos pesados.

Esto se hace mediante el protocolo TUS contra `https://video.bunnycdn.com/tusupload`. No obstante, este proceso requiere introducir una capa de seguridad adicional, enviando una solicitud de firma temporal al endpoint `/api/bunny/sign`, para que *Bunny* acepte la solicitud de carga de video mediante TUS.

Mientras se realiza la carga del video a los servidores de *Bunny*, se registra en la base de datos de la plataforma el nuevo video junto con información como duración, dimensiones, identificador de *Bunny*, título. Esto es importante porque *Bunny* gestiona el archivo multimedia, pero *Booster* sigue siendo dueño de los datos funcionales de la plataforma como su visibilidad, sus comentarios, su calificación promedio, entre otros.

Una vez completada la subida, *Bunny* realiza su propio procesamiento de video y *Booster*, espera a que se complete esta operación. Para eso, se utiliza un endpoint dentro del backend `/api/bunny/webhooks/[token]`, el cual recibe notificaciones de *Bunny* cuando cambia el estado de un video. En la ruta, se incluye un token conocido solo por el backend de la plataforma y *Bunny*, para evitar peticiones maliciosas y ejecuciones no deseadas de código.

Cuando llega un *webhook* válido a la ruta anterior, el backend extrae el identificador del video enviado por *Bunny* y obtiene su referencia en la base de datos. Con esta información, *Booster* consigue actualizar datos del video como su estado, duración y miniaturas. Cabe destacar, que para reproducir el contenido una vez procesado, se utiliza el identificador de librería y el identificador de video de *Bunny* el cual utiliza una CDN para distribuir los recursos.

6.5. Diseño del Algoritmo de Recomendación

En *Booster* se implementa un algoritmo de recomendación propio que decide qué videos enseñar según su popularidad dentro de la página, la calidad, la relación del usuario y el creador, el historial de visualización del usuario y su contexto de navegación. Las recomendaciones dentro de la sección de vista de videos aplican más filtros para sugerir contenido similar al que se está reproduciendo.

Booster implementa un algoritmo que evalúa matemáticamente múltiples factores para mostrar el mejor contenido. Estos factores incluyen el canal de boost, la cantidad de seguidores del canal, la cantidad de visualizaciones, la calificación promedio, el número de comentarios entre otros. Estas métricas, se introducen en una fórmula que determina una puntuación a cada video, denominado $score(x)$, siendo x un video cualquiera en la plataforma. De esta forma, los videos se enseñan en orden descendente de $score$. La fórmula empleada es la siguiente:

$$\begin{aligned}
 score(x) = & \ln \left(\left(\frac{\sqrt{1000 \cdot B_x}}{1000} + 1 \right)^2 + V_x \right. \\
 & + \tanh(\bar{R}_x - 3,5) \cdot \ln(\text{máx}(R_x, 1)) \\
 & + \ln(\text{máx}(R_x, 1)) \\
 & + \ln(\text{máx}(C_x, 1)) + \frac{\sqrt{1000 \cdot B_x}}{1000} \Bigg) \\
 & + \frac{100}{\ln(H_x + 2)} \\
 & - 100 \cdot \mathbb{I}_{\text{watched}}(x) \\
 & + 50 \cdot \mathbb{I}_{\text{following}}(x) \\
 & + 20 \cdot \ln(U_{\text{cat}}(x) + 1) \\
 & + 50 \cdot \mathbb{I}_{\text{sameCategoryAs_y}}(x) \cdot \mathbb{I}_{\text{videosPage}}(x)
 \end{aligned} \tag{6.1}$$

Donde B_x representa los puntos de boost del creador del video, V_x el número de visualizaciones, $\bar{R}_x \in [1, 5]$ la calificación promedio, R_x el número de calificaciones, C_x el número de comentarios, H_x las horas transcurridas desde la publicación, $U_{\text{cat}}(x)$ el número de visualizaciones previas del usuario en la categoría del video, y $\mathbb{I}$ representa una función indicadora que toma valor 1 si la condición se cumple y 0 en caso contrario. Es decir,

$$\mathbb{I}_{\text{condicion}}(x) = \begin{cases} 1 & \text{si condicion se cumple en x} \\ 0 & \text{en caso contrario} \end{cases} \tag{6.2}$$

Como se puede observar, el nivel del canal determinado por esta fórmula

$$\frac{\sqrt{1000 \cdot B_x}}{1000} \tag{6.3}$$

es el factor que más influencia el valor de $score(x)$. Un espectador puede cambiar la puntuación de un video donando puntos de experiencia al canal.

Además, se observa que se consideran otras métricas de popularidad como la cantidad de visitas, comentarios y calificaciones. Para evitar que los videos muy populares dominen la recomendación, se aplica logaritmos a varias variables, suavizando el efecto de los valores grandes. La diferencia entre un video con 100.000 y 110.000 visualizaciones no debería alterar de forma desproporcionada el orden de recomendación. El logaritmo permite controlar ese crecimiento.

El algoritmo también toma en cuenta la calidad del video mediante la calificación promedio R_x . Se compara la valoración promedio con un umbral de 3.5 estrellas mediante la función $\tanh(x - 3,5)$. Si el video está por encima de este valor, obtiene un efecto positivo y aumenta el nivel del *score*. Por otro lado, si está por debajo, se obtiene un valor negativo y se penaliza. Este efecto se multiplica por la cantidad de calificaciones recibidas suavizadas con $\ln(x)$.

Además, el algoritmo incluye un factor de actualidad. Los videos más recientes reciben una bonificación adicional la cual va disminuyendo con el tiempo. De esta forma, el contenido nuevo es mejor valorado por el algoritmo de recomendación.

Asimismo, se incluye una penalización fuerte para los videos que han sido vistos recientemente por el usuario. Con esta estrategia, se evita recomendar repetidamente el mismo contenido y permite a los usuarios descubrir nuevos videos.

El algoritmo también se adapta al usuario en sesión. Como se puede observar en la fórmula, los videos pertenecientes a un creador que el usuario esté siguiendo reciben una bonificación mayor. Además, el algoritmo mantiene una cuenta de las veces que un usuario ha visto una misma categoría, y determina qué tan importante es para el usuario esa temática de videos. Es decir, si un usuario ha visualizado varios videos de gaming, el algoritmo le asignará más valor a videos con esa misma categoría.

Finalmente, el último factor solo se aplican cuando el usuario está visualizando un video. Aquí, para recomendar contenido similar, cuando el usuario está reproduciendo un video y , el algoritmo da prioridad a otros videos de la misma categoría que y . Esto solo aplica dentro de la página de videos y su propósito es dar continuidad temática.

Esta información se encuentra explicada en la página web en la sección de [/about](#).³

6.6. Diseño de la búsqueda semántica de videos

En *Booster*, además de implementar una búsqueda de videos según las coincidencias de términos, se ha explorado una alternativa mediante *embeddings*. Esta búsqueda semántica de videos permite encontrar contenido por cercanía conceptual de las palabras que el usuario escribe. A diferencia de la búsqueda tradicional donde se filtran los resultados que tengan literalmente algunas de las palabras introducidas, esta nueva búsqueda permite capturar el significado de la consulta.

En *Booster*, esto se ha implementado teniendo en cuenta el título y la descripción para crear *embeddings*. Estos son representaciones vectoriales de texto diseñados para capturar el significado semántico de las palabras.

Más concretamente, si un video tiene un título t_i y una descripción d_i , se combinan para aplicar una función de *embeddings* que transforma esa palabra en un vector que intenta representar su significado. Particularmente, se hace

$$S_i = t_i + d_i \tag{6.4}$$

³<https://www.boostervideos.net/about>

Capítulo 6. Diseño y Arquitectura

y luego, se aplica la función obteniendo

$$\vec{e}_i = f(S_i) \quad (6.5)$$

$\vec{e}_i$, es un vector de muchas dimensiones d donde cada una representa un significado semántico diferente. Por lo tanto,

$$\vec{e}_i \in \mathbb{R}^d \quad (6.6)$$

De esta forma, videos con títulos y descripciones similares quedan posicionados cerca. Por ejemplo, un video titulado "Cómo crear una página web" puede estar cercano a otro con un título como "Fundamentos de Javascript", aunque no tengan palabras en común.

Cuando un usuario introduce una consulta q se le aplica la función $f(q) = \vec{q}$, obteniendo un nuevo vector dentro del mismo espacio vectorial que los *embeddings* almacenados en la base de datos. A continuación, se calcula la similitud entre $\vec{q}$ y cada *embedding* de video, seleccionando aquellos con mayor parecido y por tanto, con mayor relación semántica. Una forma de medir el parecido es mediante el producto escalar de vectores, en este caso

$$\|\vec{e}_i\| \cdot \|\vec{q}\| \cdot \cos(\theta) = \vec{e}_i \cdot \vec{q} \quad (6.7)$$

A partir de esta fórmula, se obtiene el índice de similitud como

$$s(\vec{q}, \vec{e}_i) = \cos(\theta) = \frac{\vec{e}_i \cdot \vec{q}}{\|\vec{e}_i\| \cdot \|\vec{q}\|} \in [-1, 1] \quad (6.8)$$

Si ambos vectores apuntan en una dirección parecida, el valor del producto escalar tenderá a 1 y por tanto se dice que tienen significados similares. Por otro lado, si el valor se aproxima a -1 los vectores no están relacionados.

La búsqueda semántica, calcula el índice de similitud de la consulta con cada uno de los vectores almacenados en la base de datos para determinar los videos con un título y una descripción que se asemejan más a la búsqueda del usuario. Es decir,

$$\text{top}(x, \vec{q}) = \arg \max_{\vec{x}_i \in V} s(\vec{q}, \vec{x}_i) \quad (6.9)$$

Donde V es el conjunto de *embeddings* de los videos almacenados en la base de datos y $V \subset \mathbb{R}^d$

En *Booster*, los *embeddings* se almacenan como un atributo adicional en la tabla videos. Estos son de tipo vector con una dimensión $d = 1536$, debido al modelo de *embeddings* utilizado ⁴. Como este atributo depende del título y la descripción del video, cada vez que se actualiza alguno de estos datos, se debe reconstruir y actualizar el vector representado.

La base de datos NeonDB (3.1.2), permite realizar operaciones de similitud vectorial directamente sobre estos *embeddings*. Con esto, la consulta se envía a la base de datos, y esta calcula los índices de similitud y devuelve los más cercanos.

⁴OpenAI text-embedding-ada-002

Capítulo 7

Desarrollo

Este capítulo está destinado a explicar la implementación de algunos de los componentes más importantes de la aplicación. En concreto, se analizará el código utilizado para la precarga de datos con tRPC, la implementación del procesamiento de videos propio (incluyendo código Python y Docker) y la integración con el proveedor de servicios de video, estudiando los endpoints y el TUS upload. Además, se analizará la lógica detrás de los comentarios y cómo se recuperan datos de los usuarios y los videos desde el cliente.

7.1. Precarga de Datos con tRPC

En 6.1.11 y 3.1.4, se analizó cómo funciona el sistema de precarga de datos mediante tRPC. Más específicamente, se mencionó que en `/src/app` se encuentran los componentes servidores. Por ejemplo, el siguiente código es un fragmento del componente servidor de un canal de usuario:

```
1 //Booster/src/app/(users)/users/[userId]/page.tsx
2
3 import { UsersView } from "@modules/users/views/users-view";
4 import { HydrateClient, trpc } from "@trpc/server";
5
6 export const dynamic = 'force-dynamic'; // IMPORTANTE: ESTO FUERZA LA
7                                         // PRECARGA DE DATOS
8
9 interface PageProps {
10   params: Promise<{ userId: string }>;
11 }
12
13
14 const Page = async ({ params }: PageProps) => { // se recibe el parámetro
15                                                  // variable userId que se
16                                                  // introduce en la ruta
17   '''
18   Para obtener el id del usuario según Next.js,
19   en la ruta, el directorio se debe llamar [userId],
20   igual que la variable.
21   '''
22
23   const { userId } = await params;
24
25
```

Capítulo 7. Desarrollo

```
26
27  '''
28  Precarga de datos devueltos por este procedimiento.
29  se ejecuta el procedimiento y los datos se guardan
30  en cache
31  '''
32
33  void trpc.users.getByUserId.prefetch({ userId});
34  void trpc.users.getVideosByUserId.prefetch({ userId});
35
36  //...
37
38  '''
39  <HydrateClient> Pasa los datos precargados por el servidor
40  hacia la cache del cliente.
41  '''
42
43
44  return (
45    /* Se carga el componente UsersView (frontend)
46       y se le pasa el id del usuario */
47    <HydrateClient>
48      <UsersView userId={userId} />
49    </HydrateClient>
50  );
51 }
52
53 export default Page;
```

En este código, se observa cómo se realiza una precarga de datos mediante

```
1 void trpc.users.getByUserId.prefetch({ userId});
```

Esto invoca al procedimiento para realizar una precarga de datos y posteriormente con `<HydrateClient>`, se carga la información en la caché del cliente. Por tanto, cuando el componente cliente `UsersView` necesite recuperar los datos, consultará directamente a la caché. Dentro del código del cliente se realiza la siguiente acción:

```
1 const [user] = trpc.users.getByUserId.useSuspenseQuery({ userId });
2 const [userVideos] = trpc.users.getVideosByUserId.useSuspenseQuery({ userId, });
```

Estos métodos recuperan la información depositada por el servidor en la caché. Como se puede observar, el servidor realiza una precarga de datos de los procedimientos de tRPC `getByUserId` y `getVideosByUserId`, por tanto, el cliente recupera la información mediante `useSuspenseQuery()` y los datos se almacenan como un objeto JavaScript dentro de la variable `user` y `userVideos`.

Como las operaciones de recuperación de datos desde un servidor pueden tardar un tiempo en procesarse, los componentes de la página web se pueden cargar antes de disponer de la información. Por lo tanto, es común emplear métodos visuales que indican estados de carga. Por ejemplo, en la sección de videos se realiza lo siguiente.

```
1 export const VideoSection = ({ videoId }: VideoSectionProps) => {
2   return (
3     <Suspense fallback={<VideoSectionSkeleton />}>
4       <ErrorBoundary fallback={<VideoErrorFallback />}>
5         <VideoSectionSuspense videoId={videoId} />
6       </ErrorBoundary>
7     </Suspense>
```

7.2. Implementación del procesamiento de videos propio

```
8     )  
9 }
```

Aquí, mientras la información correspondiente a un video no está disponible, se muestra un `<Skeleton>`. Cuando se esté esperando a los datos, se enseña al usuario una página web esqueleto que indica visualmente los componentes que se rellenarán y además, indica claramente el estado de carga.

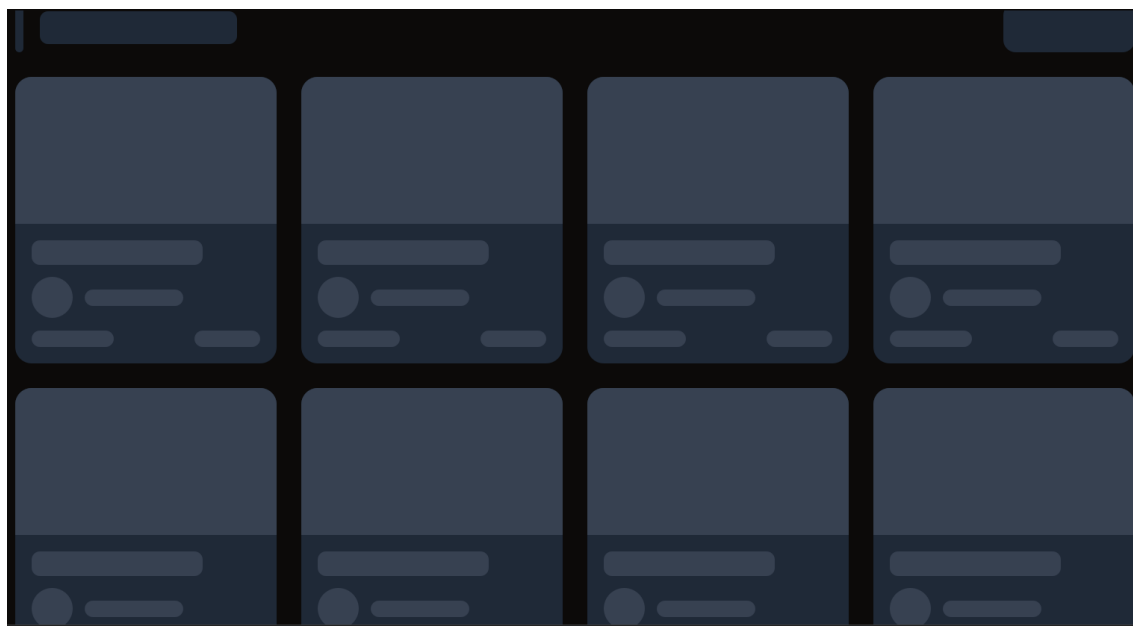


Figura 7.1: Esqueleto en la página principal

7.2. Implementación del procesamiento de videos propio

En 6.4.1 se analizó el funcionamiento de la arquitectura del procesamiento de videos con una solución personalizada utilizando los servicios de AWS. Aquí, se muestra el código en Python y el Dockerfile de los contenedores que ejecutan las tareas de consumidores de la cola de procesamiento y el procesamiento de videos.

7.2.1. Consumidor

El consumidor, es el componente encargado de leer trabajos pendientes en la cola de procesamiento de Amazon SQS y generar trabajos de procesamiento de videos. Este, está constantemente observando la cola y esperando nuevas peticiones para crear procesos de transcodificación y segmentación de videos utilizando ECS, ECR y *Fargate* de Amazon. Se implementa mediante un microservicio con Docker el cual también se despliega en los servicios de Amazon.

El siguiente código es el *Dockerfile* correspondiente al consumidor que se ejecuta en Amazon. Este prepara el contenedor para ejecutar la tarea instalando los paquetes de Python almacenados en `requirements.txt`: *boto3*, *pydantic_settings* y *python-dotenv*.

```
1 FROM python:3.11-slim  
2 WORKDIR /app
```

Capítulo 7. Desarrollo

```
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 CMD ["python", "main.py"]
```

El siguiente código, es el programa en Python que ejecuta el consumidor.

```
1 # main.py
2 import urllib
3 import boto3
4 from secret_keys import SecretKeys
5 import json
6
7 secret_keys = SecretKeys()
8 sqs_client = boto3.client("sqs", region_name=secret_keys.REGION_NAME)
9 ecs_client = boto3.client("ecs", region_name=secret_keys.REGION_NAME)
10
11
12 def poll_sqs():
13     while True:
14         response = sqs_client.receive_message( # Determinar mensajes en la cola
15             QueueUrl=secret_keys.AWS_SQS_VIDEO_PROCESSING,
16             MaxNumberOfMessages=10,
17             WaitTimeSeconds=10,
18         )
19
20         for message in response.get("Messages", []):
21             message_body = json.loads(message.get("Body"))
22
23             if (
24                 "Service" in message_body
25                 and "Event" in message_body
26                 and message_body.get("Event") == "s3:TestEvent"
27             ):
28                 sqs_client.delete_message(
29                     QueueUrl=secret_keys.AWS_SQS_VIDEO_PROCESSING,
30                     ReceiptHandle=message["ReceiptHandle"]
31                 )
32
33                 continue
34
35             if "Records" in message_body:
36                 s3_record = message_body['Records'][0]['s3']
37                 bucket_name=s3_record['bucket']['name']
38                 s3_key = urllib.parse.unquote_plus(s3_record['object']['key'])
39
40                 # spin up docker container -- we need ecs (ecs: like kubernetes)
41                 # ECR To upload docker containers
42                 # ECS uses ECR to spin up the docker container
43                 # ECR + ECS run on fargate
44
45                 response = ecs_client.run_task(
46                     cluster="arn:aws:ecs:eu-west-3:744389573944:cluster/"
```


7.2. Implementación del procesamiento de videos propio

```
48         boostervideos-transcoder-cluster",
49         launchType="FARGATE",
50         taskDefinition="arn:aws:ecs:eu-west-3:744389573944:
51             task-definition/boostervideo-transcoder:3",
52         overrides={
53             "containerOverrides": [
54                 {
55                     "name": "boostervideo-transcoder",
56                     "environment": [
57                         {"name": "S3_BUCKET", "value": bucket_name},
58                         {"name": "S3_KEY", "value": s3_key},
59                     ]
60                 }
61             ]
62         },
63         networkConfiguration={
64             "awsvpcConfiguration": {
65                 "subnets": [
66                     "subnet-027c641f401bf92a9",
67                     "subnet-04f29995b26c5b201",
68                     "subnet-097ecb0316d1b4dd6"
69                 ],
70                 "assignPublicIp": "ENABLED",
71                 "securityGroups": [
72                     "sg-082b2f92620250501"
73                 ]
74             }
75         }
76     )
77
78     print(response)
79     if(response.get('tasks')) continue
80     sqs_client.delete_message(
81         QueueUrl=secret_keys.AWS_SQS_VIDEO_PROCESSING,
82         ReceiptHandle=message['ReceiptHandle']
83     )
84     print(response)
85
86 if __name__ == "__main__":
87     poll_sqs()
```

Como se puede observar, el consumidor está dentro de un bucle infinito y espera a recibir mensajes en la cola. Se descartan los mensajes de prueba y, cuando llega un trabajo a la cola tras cargar un video en el *Raw Videos Bucket* de S3 de Amazon, se lanza un contenedor Docker para procesar el video. Esto se realiza mediante un cliente ECS (Elastic Container Service), un orquestador de contenedores que permite ejecutar tareas en AWS Fargate. Estas herramientas permiten enfocarse únicamente en diseñar y ejecutar los programas de los contenedores, eliminando la necesidad de gestionar o escalar estos servicios.

El código anterior se apoya en un archivo `.env` y en `secret_keys.py` para gestionar las claves de API y otra información sensible.

```
1 # secret_keys.py
2
```

Capítulo 7. Desarrollo

```
3 from pydantic_settings import BaseSettings
4 from dotenv import load_dotenv
5
6 load_dotenv()
7
8 class SecretKeys(BaseSettings):
9     REGION_NAME: str = ""
10    AWS_SQS_VIDEO_PROCESSING: str = ""
```

7.2.2. Procesador de video

Cuando se despliega una tarea de procesamiento, con Amazon Fargate se crea una instancia de un contenedor que se encarga de transcodificar y segmentar el contenido mediante el uso de la herramienta FFmpeg. Este, también presenta un *Dockerfile*:

```
1 FROM python:3.11-slim
2 RUN apt-get update && apt-get install -y ffmpeg && apt-get clean && rm -rf var/lib/ap
3 WORKDIR /app
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir -r requirements.txt
6 COPY . .
7
8 CMD ["python", "main.py"]
```

Como se puede observar, cuando se crea este contenedor, se instala en primer lugar FFmpeg, la herramienta que se utiliza para transcodificar y segmentar el video 2.1.1. A continuación, de la misma forma que con el consumidor, se instalan los paquetes de Python necesarios. Estos son los mismos que en el caso anterior, pero añadiendo la librería *requests*.

El procesamiento de videos ejecuta el siguiente código Python

```
1 # main.py
2
3 from pathlib import Path
4 from secret_keys import SecretKeys
5 import boto3
6 import subprocess
7 import os
8 import requests
9 from boto3.s3.transfer import TransferConfig
10
11
12
13 secret_keys = SecretKeys()
14 class VideoTranscoder:
15     '''
16     Inicializa conexión con mi servicio AWS
17     '''
18     def __init__(self):
19         self.s3_client = boto3.client(
20             "s3",
21             region_name=secret_keys.REGION_NAME,
22             aws_access_key_id=secret_keys.AWS_ACCESS_KEY_ID,
23             aws_secret_access_key=secret_keys.AWS_SECRET_ACCESS_KEY,
```

7.2. Implementación del procesamiento de videos propio

```
24         )
25
26
27     '''
28     Determina la duración de un video
29     '''
30     def ffprobe_duration(self, input_path: str) -> float:
31         """Return duration in seconds using ffprobe (float)."""
32         cmd = [
33             "ffprobe",
34             "-v", "error",
35             "-select_streams", "v:0",
36             "-show_entries", "format=duration",
37             "-of", "default=noprint_wrappers=1:nokey=1",
38             input_path,
39         ]
40         out = subprocess.check_output(cmd, text=True).strip()
41         return float(out) if out else 0.0
42
43
44     '''
45     Extrae un fotograma representativo para
46     generar una miniatura del video
47     '''
48
49     def extract_thumbnail(self, input_path: str, output_path: str, ts_seconds: float):
50         """
51         Save a single JPEG frame at timestamp ts_seconds.
52         """
53         # ensure non-negative, round to 3 decimals
54         ts = max(0.0, ts_seconds)
55         cmd = [
56             "ffmpeg",
57             "-hide_banner", "-loglevel", "error",
58             "-i", input_path,
59             "-ss", f"{ts:.3f}",
60             "-frames:v", "1",
61             "-q:v", "2",                # quality (2 is high)
62             "-y",                       # overwrite
63             output_path,
64         ]
65         process = subprocess.run(cmd, check=True)
66         if process.returncode != 0:
67             print(process.stderr)
68             raise Exception("Error in generating thumbnail")
69
70     def download_video(self, local_path: str):
71         self.s3_client.download_file(
72             secret_keys.S3_BUCKET,
73             secret_keys.S3_KEY,
74             local_path
75         )
76
```

Capítulo 7. Desarrollo

```
77     # ...
78
79     '''
80     Transcodificación y Segmentación de video
81     utilizando el protocolo MPEG-DASH
82     '''
83
84     def transcode_video(self, input_path: str, output_dir: str):
85         # DASH
86         cmd = [
87             "ffmpeg",
88             "-i", input_path,
89
90             # create 3 video ladders
91             "-filter_complex",
92             "[0:v]split=3[v1][v2][v3];"
93             "[v1]scale=640:360:flags=fast_bilinear[360p];"
94             "[v2]scale=1280:720:flags=fast_bilinear[720p];"
95             "[v3]scale=1920:1080:flags=fast_bilinear[1080p]",
96
97             # 360p
98             "-map", "[360p]", "-c:v:0", "libx264", "-b:v:0", "1000k",
99             "-preset", "veryfast", "-profile:v", "high", "-level:v", "4.1",
100
101             # 720p
102             "-map", "[720p]", "-c:v:1", "libx264", "-b:v:1", "4000k",
103             "-preset", "veryfast", "-profile:v", "high", "-level:v", "4.1",
104
105             # 1080p
106             "-map", "[1080p]", "-c:v:2", "libx264", "-b:v:2", "8000k",
107             "-preset", "veryfast", "-profile:v", "high", "-level:v", "4.1",
108
109             # audio (optional)
110             "-map", "0:a?", "-c:a", "aac", "-b:a", "128k",
111
112             # keyframe/segment alignment (choose seg = 6s; align GOP to it)
113             # If source fps=30, g=180; if 24fps, g=144. We'll force keyframes every
114             "-g", "180", # Assuming video is 30FPS. If 60 FPS use 360.
115             "-keyint_min", "180",
116             "-sc_threshold", "0",
117             "-force_key_frames", "expr:gte(t,n_forced*6)",
118
119             # DASH muxer (static VOD)
120             "-f", "dash",
121             "-use_timeline", "1",
122             "-use_template", "1",
123             "-seg_duration", "6", # 6 seconds for segment duration
124
125             # DO NOT set window_size for VOD; removing it avoids sliding window
126             # "-window_size", "0", # (optional) explicitly zero
127             "-adaptation_sets", "id=0,streams=v id=1,streams=a",
128             "-init_seg_name", "init-$RepresentationID$.m4s",
129             "-media_seg_name", "chunk-$RepresentationID$-$Number%05d$.m4s",
```

7.2. Implementación del procesamiento de videos propio

```
130
131         f"{output_dir}/manifest.mpd",
132     ]
133
134     process = subprocess.run(cmd)
135
136     if process.returncode != 0:
137         print(process.stderr)
138         raise Exception("Error in transcoding process")
139
140
141     '''
142     Cargar al bucket de Videos Procesados los
143     archivos y fragmentos recientemente generados
144     '''
145
146     def upload_files(self, prefix:str, local_dir:str):
147
148         cfg = TransferConfig(
149             multipart_threshold=5*1024*1024,
150             multipart_chunksize=5*1024*1024,
151             max_concurrency=16,
152             use_threads=True,
153         )
154
155         for root, _, files in os.walk(local_dir):
156             for file in files:
157                 local_path = os.path.join(root, file)
158                 s3_key = f"{prefix}/{os.path.relpath(local_path, local_dir)}"
159                 self.s3_client.upload_file(
160                     local_path,
161                     secret_keys.S3_PROCESSED_VIDEOS_BUCKET,
162                     s3_key,
163                     ExtraArgs={
164                         "ACL": "public-read",
165                         'ContentType': self._get_content_type(local_path),
166                     },
167                     Config=cfg
168                 )
169
170     '''
171     Método principal
172     '''
173     def process_video(self):
174         work_dir = Path("/tmp/workspace")
175         work_dir.mkdir(exist_ok=True)
176
177         input_path = work_dir / "input.mp4"
178         output_path = work_dir / "output"
179         output_path.mkdir(exist_ok=True) # if the folder already exists,
180                                         # do nothing, otw create it
181
182         try:
```

Capítulo 7. Desarrollo

```
183         self.download_video(input_path)
184         dur = self.ffprobe_duration(input_path)
185
186
187
188         if(dur > 600):    # 10 minute limit
189             print("Video too long")
190             exit(1)
191
192
193         print("DURACION:", dur)
194
195         try:
196             # Enviar solicitud a Booster para que actualice la duracion del video
197             res = requests.put(f"{secret_keys.BACKEND_URL}?
198                               id={secret_keys.S3_KEY}&duration={dur}")
199         except Exception as e:
200             print(e)
201
202         self.transcode_video(str(input_path), str(output_path))
203
204         try:
205             ts = dur * 0.10 if dur > 0 else 0.0
206             poster_path = output_path / "poster.jpg"
207             self.extract_thumbnail(input_path, poster_path, ts)
208         except Exception as e:
209             print("Thumbnail generation failed:", e)
210
211
212         # Cargar todos los archivos generados en output_path
213         # al bucket de videos procesado
214         self.upload_files(secret_keys.S3_KEY, str(output_path))
215         self.update_video()
216     finally:
217         if input_path.exists():
218             input_path.unlink() #delete the file
219         if output_path.exists():
220             import shutil
221             shutil.rmtree(str(output_path))
222
223
224     def update_video(self):
225         print("ENVIANDO A ", secret_keys.BACKEND_URL)
226         try:
227             print("envia")
228             response = requests.put(f"{secret_keys.BACKEND_URL}?id={secret_keys.S3_KEY}
229                                     &duration={dur}")
230             print(response.json())
231             return response.json()
232         except Exception as e:
233             print("errored")
234             print(e)
235
236     def process_video():
```

7.2. Implementación del procesamiento de videos propio

Este programa se compone de varios métodos con finalidades distintas. La primera función ejecutada es `process_video()`, la cual descarga el contenido desde el *Raw Videos Bucket* de AWS. Luego, se analiza la duración y si es inferior a 10 minutos, comienza el procesamiento del video.

El método `transcode_video()` es el encargado de segmentar y transcodificar el video. Se utiliza la herramienta FFmpeg para generar distintas calidades de video con tasas de bits diferentes. Además, se incluye el audio en el resultado final y se segmenta el video en fragmentos de 6 segundos. En la implementación elegida, se emplea MPEG-DASH (2.1.3) como protocolo de streaming de video y por consiguiente se debe particionar el video en fragmentos con la extensión `.m4s` y se debe generar un archivo `manifest.mpd` llamado MPD. Este se encarga de informar al reproductor de videos en el cliente sobre el contenido, como las calidades disponibles, las localizaciones de los segmentos `.m4s` en el servidor, cuándo se debe reproducir cada segmento y qué códec utilizar. Además, se genera un archivo de inicialización (`init-... .m4s`) para reproducir los fragmentos del video `.m4s`.

Finalmente, los archivos de inicialización, los fragmentos y el manifiesto (MPD) se cargan al *bucket* que almacena los videos procesados para ser entregados al cliente.

7.3. Implementación del procesamiento de videos con Bunny

En 6.4.2, se analizó cómo integrar el proveedor que se encarga de la gestión y procesamiento de videos, *Bunny.net*, más concretamente, su servicio *BunnyStream*.

En esa sección, se describieron los pasos para cargar un archivo de video a un proveedor externo. Esto comienza enviando una solicitud POST a un endpoint de bunny, pero en la implementación elegida, el cliente llama a una ruta dentro del backend de *Booster* que se encarga de verificar la sesión del usuario y comunicarse directamente con *Bunny*. Este endpoint está en `/api/bunny/create` y realiza lo siguiente:

```
1 // app/api/bunny/create/route.ts
2 import { NextResponse } from "next/server";
3 import { auth } from "@clerk/nextjs/server";
4 import { uploadRateLimit } from "@lib/ratelimit";
5 import { db } from "@db";
6 import { users, videos } from "@db/schema";
7 import { inArray } from "drizzle-orm";
8
9 export async function POST(req: Request) {
10   '''
11   La petición que llega tiene información de
12   sesión de Clerk.
13   Clerk indica que las peticiones desde un
14   Client Component hacia un Route Handler leen
15   la sesión desde cookies y no necesitan enviar
16   un Bearer token manualmente.
17
18   auth(), lee la información de sesión de
19   la petición (request).
20   '''
21
22   const { userId: clerkUserId } = await auth();
23   if (!clerkUserId) {
24     return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
25   }
26
27
28
29   let userId;
30
31   const [user] = await db
32     .select()
33     .from(users)
34     .where(inArray(users.clerkId, clerkUserId ? [clerkUserId] : [])); //trick
35
36   if (user) {
37     userId = user.id;
38   }
39
40   if(!userId) {
41     return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
42   }
43
44   const { success } = await uploadRateLimit.limit(userId);
45   if (!success) {
46     return NextResponse.json(
```


7.3. Implementación del procesamiento de videos con Bunny

```
47     { error: "Daily upload limit reached (5 uploads per day)" },
48     { status: 429 }
49   );
50 }
51
52 const { title } = await req.json();
53 const lib = process.env.NEXT_PUBLIC_BUNNY_STREAM_LIBRARY_ID!;
54 const r = await fetch(`https://video.bunnycdn.com/library/${lib}/videos`, {
55   method: "POST",
56   headers: {
57     AccessKey: process.env.BUNNY_STREAM_API_KEY!,
58     accept: "application/json",
59     "content-type": "application/json",
60   },
61   body: JSON.stringify({ title }),
62 });
63 if (!r.ok)
64   return NextResponse.json({ error: await r.text() }, { status: r.status });
65 const json = await r.json(); //video id is on guid
66
67
68
69 return NextResponse.json(json);
70 }
```

Este fragmento de código corresponde a la lógica ejecutada una vez llega la petición por parte del cliente al endpoint `/create`. En primer lugar, se verifica que la petición corresponda a un usuario con una sesión válida. Gracias a Clerk y Next.js, esta información está disponible en las *cookies* y el método `auth()` puede determinarlo directamente. Si no hay una sesión de usuario activa, se descarta la petición y la operación. En caso contrario, a partir del `clerkId` identificado de la sesión se obtiene una referencia en la base de datos al usuario correspondiente. Posteriormente, se aplican límites de uso y, si todo funciona correctamente, se envía una petición POST al endpoint de Bunny para crear un objeto de video. Es importante destacar que, en este caso, se utilizan las claves de API de *Bunny* y los identificadores de biblioteca para enviar la solicitud. Por motivos de seguridad, esto se almacena en `.env.local` y se hace a través del servidor. Cuando el video es creado, se obtiene en la respuesta el identificador del objeto de video en el servicio externo.

Como se analizó anteriormente en 6.4.2, el siguiente paso para cargar un archivo a *Bunny* a través del protocolo TUS, es crear una firma digital. Esto se hace mediante el endpoint `/api/bunny/sign` que realiza los pasos recomendados por la documentación del servicio de procesamiento de videos:

```
1 // /app/api/bunny/sign/route.ts
2 import { db } from "@/db";
3 import { users, videos } from "@/db/schema";
4 import { auth } from "@clerk/nextjs/server";
5 import crypto from "crypto";
6 import { and, eq, inArray } from "drizzle-orm";
7 import { NextResponse } from "next/server";
8
9 export async function POST(req: Request) {
10   const { videoId } = (await req.json()) as { videoId: string };
11   if (!videoId) return new Response("Missing videoId", { status: 400 });
12
13   const { userId: clerkUserId } = await auth();
14   if (!clerkUserId) return new Response("Unauthorized", { status: 401 });
```

Capítulo 7. Desarrollo

```
15
16 let userId;
17
18 const [user] = await db
19   .select()
20   .from(users)
21   .where(inArray(users.clerkId, clerkUserId ? [clerkUserId] : [])); //trick
22
23 if (user) {
24   userId = user.id;
25 }
26
27 if (!userId) {
28   return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
29 }
30
31
32 const libraryId = process.env.NEXT_PUBLIC_BUNNY_STREAM_LIBRARY_ID!;
33 const apiKey = process.env.BUNNY_STREAM_API_KEY!;
34
35 const expires = Math.floor(Date.now() / 1000) + 10 * 60; // 10 minutos (arbitrario)
36 const payload = `${libraryId}${apiKey}${expires}${videoId}`;
37 const signature = crypto.createHash("sha256").update(payload).digest("hex");
38
39 return Response.json({
40   libraryId,
41   videoId,
42   expires,
43   signature,
44 });
45 }
```

Aquí, se realizan las mismas comprobaciones de sesión anteriores y a continuación se crea una firma tomando el identificador de librería de *Bunny*, la clave de API, entre otros. Finalmente, si todo se realiza correctamente, se devuelve la firma al cliente quien continúa con el proceso del *TUS Upload* y comienza a cargar el archivo de video.

En particular, el cliente ejecuta el siguiente código después de seleccionar un archivo para cargar al servidor mediante TUS. Se omiten partes relativas a la interfaz de usuario, sólo se analiza la implementación del *TUS Upload*

```
1
2 const utils = trpc.useUtils();
3 const router = useRouter();
4 const videoIdRef = useRef<string | null>(null);
5
6 const createAfterUpload = trpc.videos.createAfterUpload.useMutation({
7   onSuccess: (data) => {
8     utils.studio.getMany.invalidate({ limit: DEFAULT_LIMIT })
9     videoIdRef.current = data.id;
10    onUploadStarted?.(data.id);
11  }
12 });
13
14 const tusUploader = async (file: File) => {
15   try {
16     const { duration, width, height } = await getVideoMetadata(file);
17     if (duration > 600) {
18       toast.error("Video is longer than 10 minutes");
19     }
20   }
21 }
```

7.3. Implementación del procesamiento de videos con Bunny

```
19     return;
20 }
21
22 setState({ file, progress: 0, uploading: true });
23 onProgressChange?.(true);
24 const createRes = await fetch("/api/bunny/create", {
25     method: "POST",
26     headers: { "content-type": "application/json" },
27     body: JSON.stringify({ title: file.name }),
28 });
29
30 if (!createRes.ok) {
31     const errorData = await createRes.json();
32     throw new Error(errorData.error || "Failed to create video");
33 }
34 const { guid } = await createRes.json() as { guid: string };
35
36 const signRes = await fetch("/api/bunny/sign", {
37     method: "POST",
38     headers: { "content-type": "application/json" },
39     body: JSON.stringify({ videoId: guid }),
40 });
41
42 if (!signRes.ok) throw new Error(await signRes.text());
43 const { libraryId, videoId, expires, signature } = await signRes.json();
44
45
46 const upload = new tus.Upload(file, {
47     // Endpoint is the upload creation URL from your tus server
48     endpoint: 'https://video.bunnycdn.com/tusupload',
49     headers: {
50         AuthorizationSignature: signature,
51         AuthorizationExpire: expires,
52         VideoId: videoId,
53         LibraryId: libraryId,
54     },
55
56     // Retry delays will enable tus-js-client to automatically retry on errors
57     retryDelays: [0, 3000, 5000, 10000, 20000],
58     // Attach additional meta data about the file for the server
59     metadata: {
60         filename: file.name,
61         filetype: file.type,
62     },
63     // Callback for errors which cannot be fixed using retries
64     onError: (err) => {
65         setState((s) => ({ ...s, uploading: false }));
66         onProgressChange?.(false);
67         toast.error(`Upload failed: ${err.message}`);
68     },
69     onProgress: (bytesUploaded, bytesTotal) => {
70         const pct = Math.min(99, Math.round((bytesUploaded / bytesTotal) * 100));
71         setState((s) => ({ ...s, progress: pct, }));
72     },
73     onSuccess: async () => {
74         setState((s) => ({ ...s, progress: 100, uploading: false }));
75         onProgressChange?.(false);
76         toast.success("Uploaded the video! Processing started.");
```

Capítulo 7. Desarrollo

```
77         if (onSuccess && videoIdRef.current) {
78             onSuccess(videoIdRef.current);
79         } else if (!onSuccess && videoIdRef.current) {
80             router.push(`/studio/videos/${videoIdRef.current}`)
81         }
82     },
83 })
84 upload.findPreviousUploads().then(async function (previousUploads) {
85
86
87     // Start the upload
88     upload.start()
89
90     await createAfterUpload.mutateAsync({
91         bunnyVideoId: videoId,
92         title: file.name,
93         width,
94         height,
95     });
96 })
97 } catch(error:any) {
98     setState((s) => ({ ...s, uploading: false }));
99     onProgressChange?.(false);
100    toast.error("Upload failed" + error.message)
101 }
102 }
103
104 const onPick = (e: ChangeEvent<HTMLInputElement>) => {
105     const f = e.target.files?.[0]; if (f) void tusUploader(f);
106 };
107 const onDrop = (e: DragEvent<HTMLDivElement>) => {
108     e.preventDefault();
109     const f = e.dataTransfer.files?.[0]; if (f) void tusUploader(f);
110 };
```

Como se puede observar, se invoca a los endpoints `/create` y `/sign` para finalmente, comenzar la carga de contenido hacia el servidor de *Bunny.net*. Mientras se sube el video a los servidores externos, se ejecuta el procedimiento `createAfterUpload` el cual crea un nuevo registro de video en la base de datos.

```
1
2 createAfterUpload: protectedProcedure
3
4     // primero se comprueba que los datos de entrada
5     // cumplan con las restricciones y sean válidos.
6     .input(
7         z.object({
8             bunnyVideoId: z.string().uuid(), // Bunny GUID just uploaded to
9             title: z.string().min(1).max(200),
10            description: z.string().max(5000).optional(),
11            categoryId: z.string().uuid().optional(),
12            width: z.number().optional(),
13            height: z.number().optional(),
14        })
15    )
16    .mutation(async ({ ctx, input }) => {
17        const { id: userId, accountType } = ctx.user;
18        const [row] = await db
19            .insert(videos)
```

7.4. Implementación de los comentarios

```
20     .values({
21       userId,
22       title: input.title,
23       description: input.description,
24       categoryId: input.categoryId,
25       bunnyVideoId: input.bunnyVideoId,
26       bunnyStatus: "uploaded", // webhook will flip to "processed"
27       isAi: false,
28       width: input.width,
29       height: input.height,
30     })
31     .returning();
32     return row;
33   }},
```

7.4. Implementación de los comentarios

En 4.2.3, se explicaron las mejoras presentes en los comentarios de un video. Allí, se describió que los comentarios están estructurados en formato de árbol de respuestas. Dado este requerimiento, fue necesario modelar los comentarios como nodos dentro de un árbol. Además, esto supuso añadir un atributo en formato de clave foránea en la tabla de comentarios que indicara el padre de un nodo. Más específicamente, se creó del lado del cliente un componente `<Comment>` que recibe como propiedades el identificador del comentario a renderizar.

```
1   export const Comment = ({ comment, videoId, viewer, depth, maxDepth }: CommentProps) => {
2     .....
3   }
```

Con este diseño, para obtener las respuestas de un comentario, se busca en la base de datos todos aquellos registros que referencien a ese comentario como padre. Posteriormente, con esta lista de respuestas y con el uso de React, se renderizan de forma recursiva los mensajes que responden a otros. Es decir, un comentario incluye a otros dentro de él, correspondientes a sus respuestas.

```
1   <div>
2     <div className="mt-4 pl-6 border-l-2 border-amber-300
3       dark:border-amber-600 rounded-b-2xl space-y-4">
4       {replies.map((child) => (
5         <Comment key={child.commentId} isPending={isReplying} comment={child}
6           videoId={videoId} viewer={viewer} depth={depth + 1}
7             maxDepth={maxDepth} />
8       ))}
9     </div>
10    ...
11  </div>
```

7.5. Acceder y modificar datos de usuario

En esta sección se enseñan algunos de los fragmentos de código más importantes al momento de acceder y modificar los datos de un usuario. Dentro del archivo `/src/modules/users/server/procedures.ts`, se encuentran los procedimientos relativos a la gestión de usuarios. Aquí, se realizan acciones cuyo objetivo es modificar o

Capítulo 7. Desarrollo

recuperar propiedades de los usuarios en la plataforma. Por ejemplo, este procedimiento encuentra a un usuario a partir de su identificador dentro de Clerk, verificando primero que se introducen datos válidos en la consulta.

```
1  getByClerkId: protectedProcedure
2  .input(z.object({ clerkId: z.string().nullish() }))
3  .query(async ({ input }) => {
4    const { clerkId } = input;
5    const [user] = await db
6      .select()
7      .from(users)
8      .where(inArray(users.clerkId, clerkId ? [clerkId] : []));
9
10
11    if (!user) {
12      throw new TRPCError({
13        code: "NOT_FOUND",
14        message: `User with clerkId ${input.clerkId} not found`
15      });
16    }
17
18    return user;
19  }),
```

Los componentes servidores invocan estos procedimientos para realizar una precarga y los clientes recuperan la información de la caché. Estos son los datos que el cliente recupera cuando necesita mostrar un perfil de usuario.

```
1  const { userId: clerkUserId } = useAuth();
2  const { user: clerkUser } = useUser();
3  const { openMessages } = useNotificationDropdown();
4  const [user] = trpc.users.getByUserId.useSuspenseQuery({ userId });
5  const [followers] = trpc.follows.getFollowersByUserId.useSuspenseQuery({ userId, });
6  const [userVideos] = trpc.users.getVideosByUserId.useSuspenseQuery({ userId, });
7  const [boostPoints] = trpc.xp.getBoostByUserId.useSuspenseQuery({ userId });
8  const [creatorViews] = trpc.videoViews.getAllViewsByUserId.useSuspenseQuery({ userId, });
```

7.6. Acceder y modificar datos de video

De la misma forma que con un usuario, en *Booster* se implementan procedimientos para realizar acciones que modifiquen o recuperen propiedades relacionadas con los videos dentro de la plataforma. Por ejemplo, este fragmento actualiza los datos de un video:

```
1  update: protectedProcedure
2  .input(
3    videoUpdateSchema.extend({
4      title: z.string().min(1).max(200).optional(),
5      description: z.string().max(5000).optional().nullable(),
6    })
7  )
8  .mutation(async ({ ctx, input }) => {
9    const { id: userId } = ctx.user;
10    if (!input.id) {
11      throw new TRPCError({ code: "NOT_FOUND" });
12    }
13
14    if (input.communityId) {
```

7.6. Acceder y modificar datos de video

```
15     const [community] = await db
16       .select({ allowUserPosts: communities.allowUserPosts })
17       .from(communities)
18       .where(eq(communities.communityId, input.communityId));
19
20     if (!community) {
21       throw new TRPCError({
22         code: "NOT_FOUND",
23         message: "Community not found",
24       });
25     }
26
27     if (!community.allowUserPosts) {
28       const [isModerator] = await db
29         .select()
30         .from(communityModerators)
31         .where(
32           and(
33             eq(communityModerators.communityId, input.communityId),
34             eq(communityModerators.userId, userId)
35           )
36         );
37
38       if (!isModerator) {
39         throw new TRPCError({
40           code: "FORBIDDEN",
41           message: "Only moderators can post in this community",
42         });
43       }
44     }
45   }
46
47   const textToEmbed = `${input.title}\n${input.description}`;
48   const embedding = await embedText(textToEmbed);
49
50   const [updatedVideo] = await db
51     .update(videos)
52     .set({
53       title: input.title,
54       description: input.description,
55       categoryId: input.categoryId,
56       communityId: input.communityId,
57       visibility:
58         input.visibility === "private" || input.visibility === "public"
59           ? input.visibility
60           : undefined,
61       updatedAt: new Date(),
62       isAi: input.isAi,
63       embedding: embedding,
64       tags: input.tags,
65     })
66     .where(and(eq(videos.id, input.id), eq(videos.userId, userId)))
67     .returning();
68
69   if (!updatedVideo) {
70     throw new TRPCError({ code: "NOT_FOUND" });
71   }
72
```

Capítulo 7. Desarrollo

```
73     return updatedVideo;
74   }),
```

Los componentes servidores realizan la precarga de datos y luego los clientes los consumen. El servidor ejecuta este código:

```
1
2 export default async function Page({ params }: PageProps) {
3   const { videoId } = await params;
4
5   void trpc.videos.getOne.prefetch({ id: videoId });
6
7   return (
8     <HydrateClient>
9     <VideoView videoId={videoId} />
10    </HydrateClient>
11  );
12 }
```

Posteriormente, el cliente obtiene esos datos.

```
1 const VideoSectionSuspense = ({ videoId }: VideoSectionProps) => {
2   const [video] = trpc.videos.getOne.useSuspenseQuery({ id: videoId });
3
4   ...
5
6 }
```


Capítulo 8

Despliegue y Riesgos de Seguridad

En este capítulo se analiza cómo se ha conseguido desplegar la aplicación en internet mediante el uso de *Vercel* junto con sus ventajas y capacidades. Además, se destina una sección para estudiar las opiniones de usuarios sobre la plataforma. A continuación, se examinan las valoraciones de usuarios reales, quienes han aportado mejoras, detectado fallos e incluso encontrado vulnerabilidades. Finalmente, se analizan los riesgos de seguridad detectados, mejores prácticas empleadas, vulnerabilidades sufridas y las medidas tomadas para mitigarlas.

8.1. Despliegue

Para realizar el despliegue en internet de la plataforma y que esta sea accesible desde cualquier parte del mundo, se han utilizado dos proveedores de servicios de *hosting*. Por un lado, *Vercel* permite desplegar aplicaciones web y se integra bien con *Next.js* 3.1.5. Por otro, se han aprovechado las capacidades de *Hostinger* para configurar los registros DNS y SMTP, logrando así dar de alta un dominio para la plataforma y habilitar un servicio de correo electrónico.

8.1.1. Vercel

Vercel es una plataforma centrada en despliegue y hosting de aplicaciones web, especialmente aquellas construidas con *Next.js* 3.1.5. Además, *Vercel* permite conectar un repositorio de *GitHub* y generar despliegues automáticos cuando se detectan cambios, facilitando el proceso de actualización e integración, comúnmente referido como CI/CD (integración continua y despliegue continuo).

Ventajas

La principal ventaja de *Vercel* es su facilidad de despliegue. Como se mencionó anteriormente, se puede integrar con Git para monitorear los cambios en una rama específica y activar de forma automática un proceso de actualización. Más concretamente, en el caso de *Booster*, se ejecuta el comando `npm run build` para generar los archivos estáticos de las páginas, optimizar el rendimiento y asegurar que el código no tiene errores.

Otra ventaja importante de *Vercel* para *Booster* es su integración natural con el framework de *Next.js*. No es necesario implementar procesos de *build* complejos ya que

Capítulo 8. Despliegue y Riesgos de Seguridad

lo detecta automáticamente *Vercel* y aplica las configuraciones requeridas. Además, se encarga de optimizar el rendimiento de las distintas páginas dentro de la aplicación, mejorando los tiempos de carga.

Asimismo, *Vercel* cuenta con herramientas integradas que permiten visualizar el rendimiento que experimentan los usuarios con la página, analíticas de visitas, un *log* de eventos, entre otros.

Desventajas

La principal desventaja de *Vercel* es su costo de uso, que puede generar precios elevados. Aunque dispone de una versión gratuita, esta tiene límites que son rápidamente alcanzados. *Vercel* factura por tiempo de cómputo, cantidad de datos transferidos y peticiones, de modo que una arquitectura mal optimizada puede encarecerse significativamente con altos volúmenes de usuarios y tráfico.

Otra desventaja es la dependencia del proveedor. Aunque el despliegue es rápido y sencillo, muchas funcionalidades están fuertemente vinculadas a *Vercel* y requieren instalar sus librerías. Al integrar estas herramientas, el código queda ligado a su infraestructura, lo que dificultaría cambiar de proveedor en un futuro.

Uso en Booster

Para *Booster*, *Vercel* es una opción ideal ya que permite desplegar, actualizar cambios y optimizar el rendimiento de las páginas de forma rápida. Además, su integración con *Next.js* es frecuente y recomendada, convirtiéndola en la opción más favorable para adoptar como infraestructura de *hosting* cuando se trabaja con este framework. Por estas razones, se ha empleado *Vercel* para desplegar la aplicación en internet utilizando los dominios adquiridos en *Hostinger*.



Figura 8.1: Logo de Vercel

8.2. Feedback de Usuarios

A continuación, se exponen algunas opiniones de usuarios sobre la plataforma, primeras impresiones, sugerencias, cuestiones y quejas.

8.2.1. Opiniones positivas y sugerencias

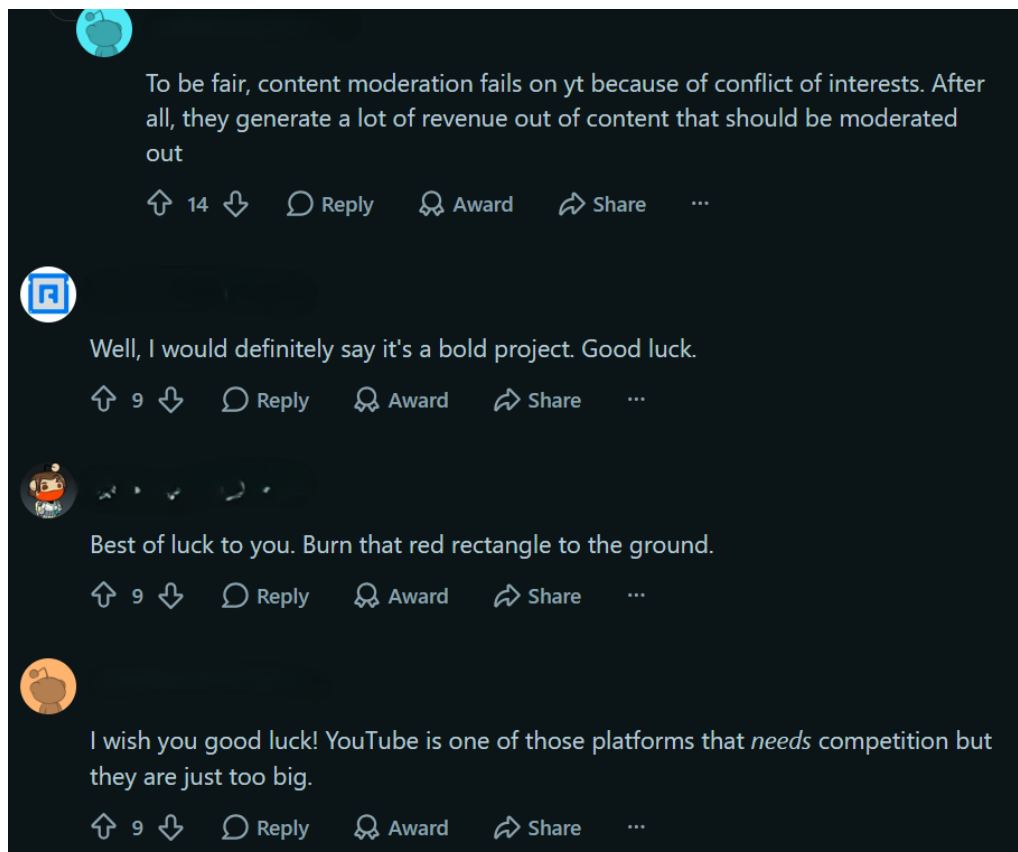


Figura 8.2: Opiniones positivas I

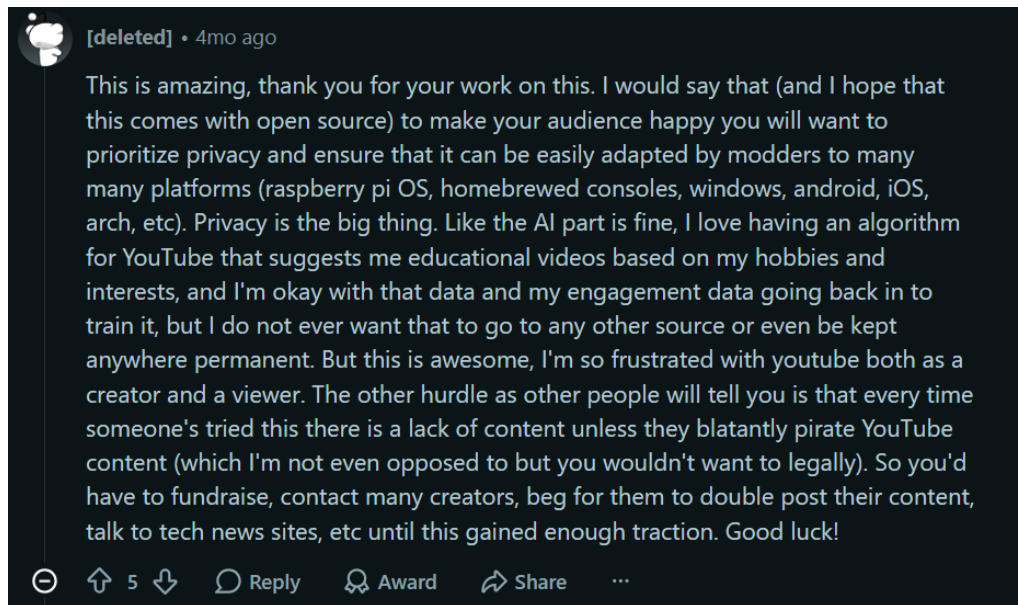


Figura 8.3: Opiniones positivas II

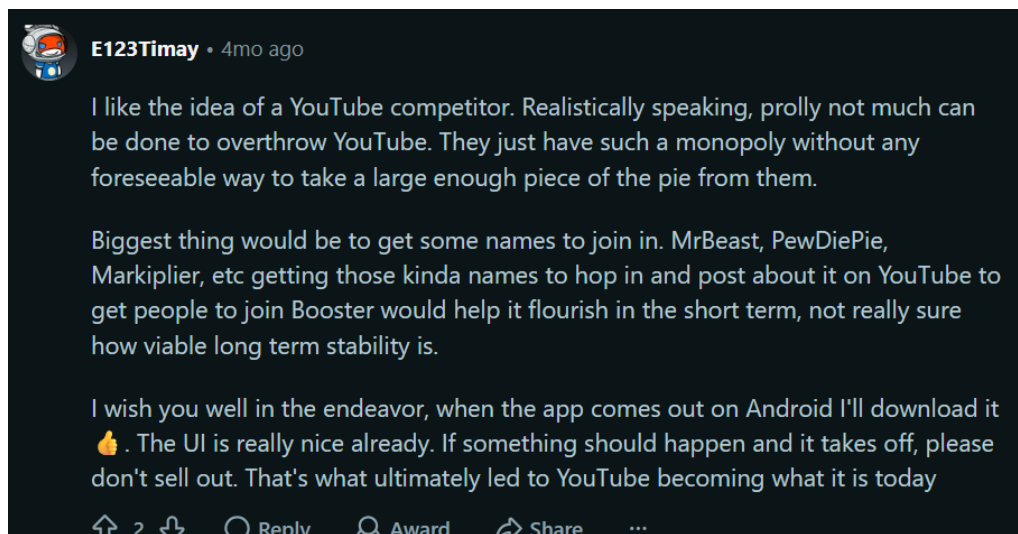


Figura 8.4: Opiniones positivas III



Figura 8.5: Filtrar contenido por IA

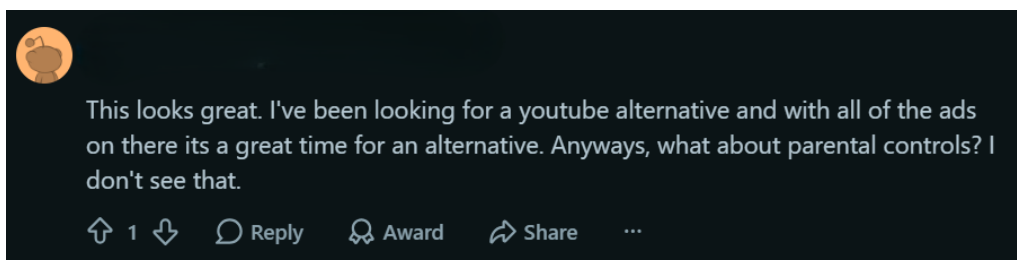


Figura 8.6: Control parental

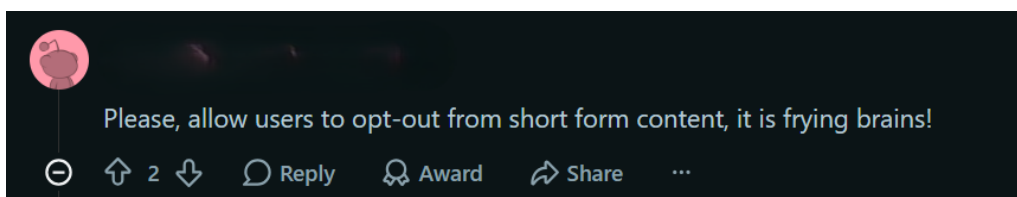


Figura 8.7: Filtrar contenido corto *shorts*

Capítulo 8. Despliegue y Riesgos de Seguridad

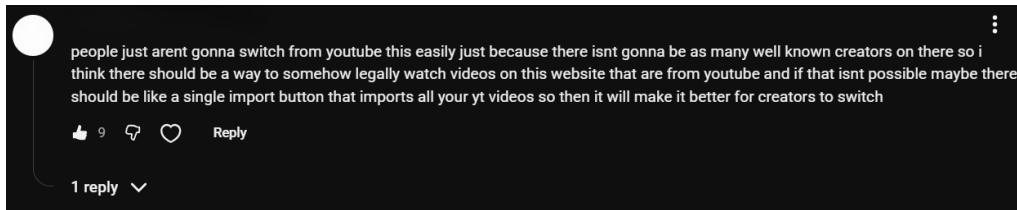


Figura 8.8: Falta de contenido en la plataforma

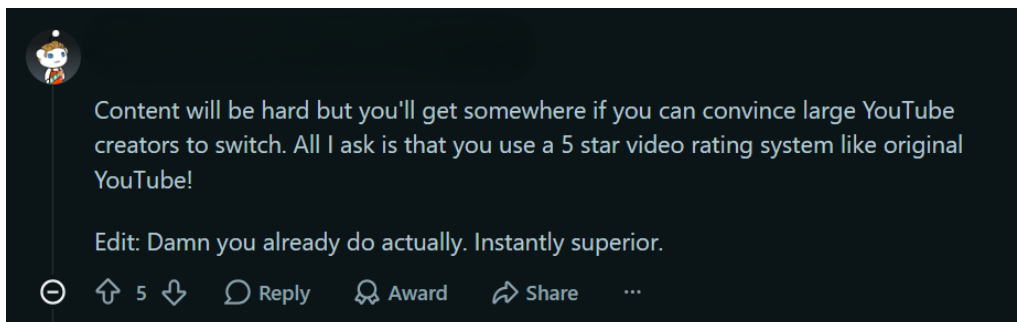


Figura 8.9: Contenido en la plataforma

8.2.2. Opiniones negativas y preocupaciones

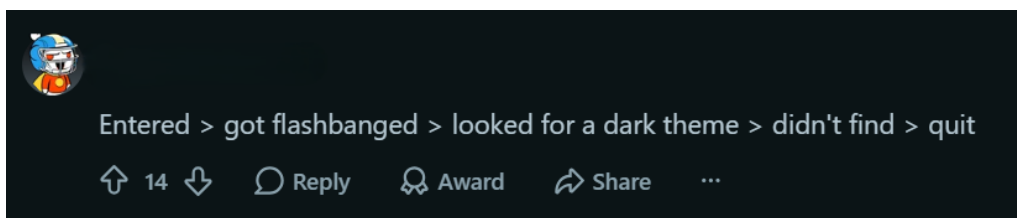


Figura 8.10: Sugerencia tema oscuro I

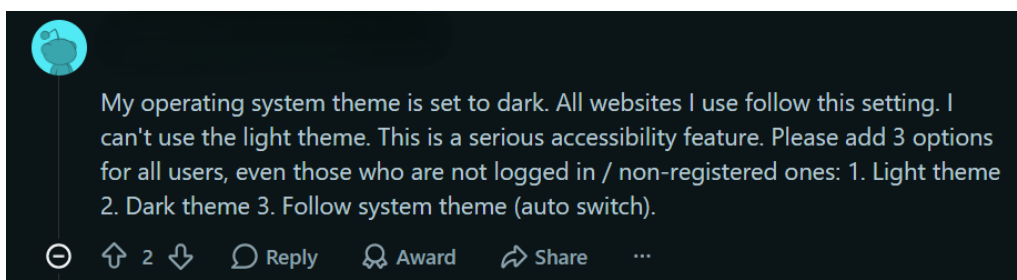


Figura 8.11: Sugerencia tema oscuro II

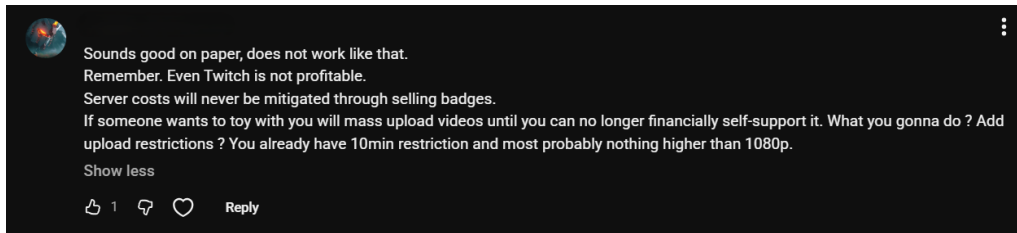


Figura 8.12: Preocupación por los costos y abusos del sistema

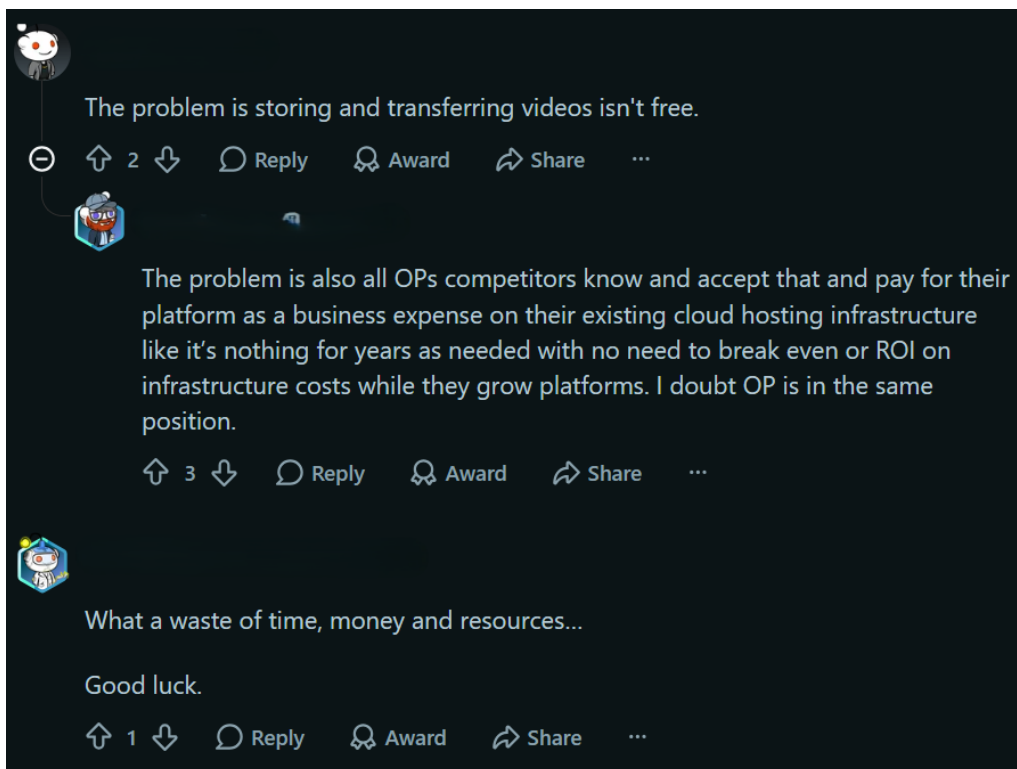


Figura 8.13: Opiniones negativas I

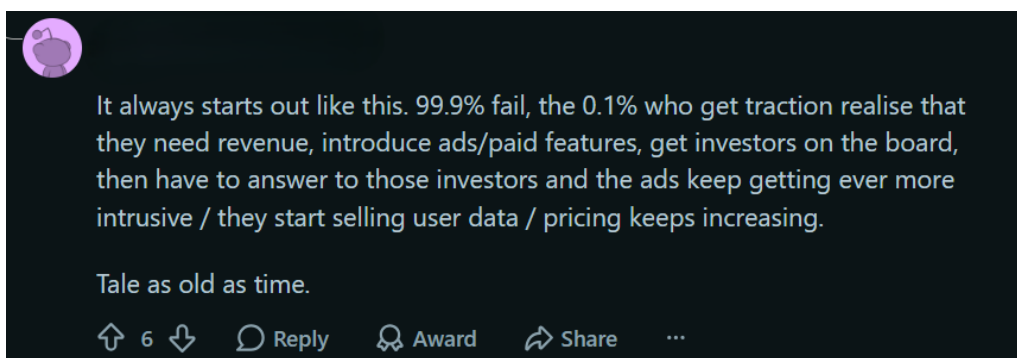


Figura 8.14: Opiniones negativas II

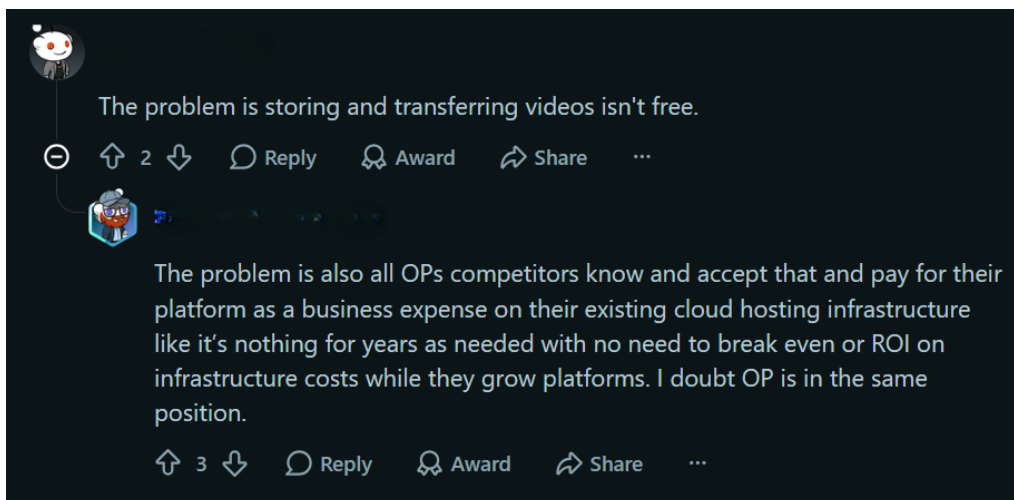


Figura 8.15: Opiniones negativas III

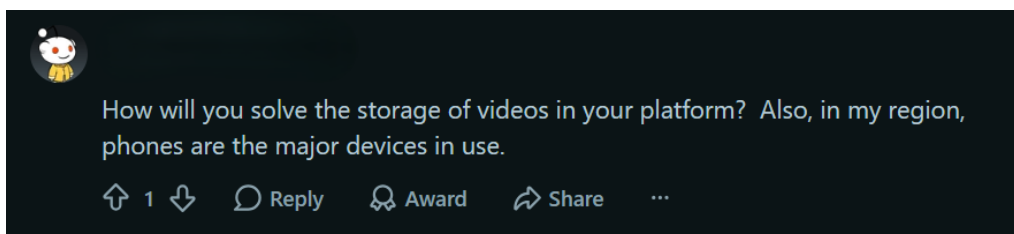


Figura 8.16: Preocupación por el almacenamiento

8.3. Vulnerabilidades detectadas y acciones tomadas

En esta sección se analizan algunas de las vulnerabilidades detectadas durante el proceso de desarrollo y también las sufridas en producción junto con las medidas tomadas para mitigarlas. Estas acciones han hecho más segura la plataforma tanto desde el punto de vista del usuario como de las operaciones.

8.3.1. Protección de accesos y modificación de videos

Durante el proceso de pruebas de la aplicación se detectaron que algunos procedimientos contenían fallos lógicos. Esto se detectó en los procedimientos correspondientes al `/studio`. Más específicamente, en `getOne` y `getMany`. En ellos, obtienen videos almacenados en la base de datos pero estos solo comprobaban el identificador del video. `getOne` recibía un identificador de video y recuperaba aquel registro con ese identificador. Además, este procedimiento actualizaba el estado del video y su miniatura. Realizar solo esta comprobación resultaba peligroso ya que cualquier usuario tenía la capacidad de invocar esta función, recuperar los datos de un video que no es suyo y cambiar la miniatura del video. Esto corresponde a una ruptura de control de acceso, permitiendo a los usuarios actuar fuera de sus permisos.

Para solucionar estos fallos, se incluyeron las siguientes comprobaciones en los procedimientos que recuperan videos de un usuario en `/studio`. Anteriormente se tenía incorrectamente:

8.3. Vulnerabilidades detectadas y acciones tomadas

```
1  getOne: baseProcedure
2    .input(z.object({ id: z.string().uuid() }))
3    .query(async ({ input }) => {
4
5      const [videos] = await db
6        .select({ ...getTableColumns(videos) })
7        .from(videos)
8        .where(eq(videos.userId, userId));
9
10     //...
11
12   })
```

Esto se actualizó a

```
1  getOne: protectedProcedure
2    .input(z.object({ id: z.string().uuid() }))
3    .query(async ({ input, ctx }) => {
4
5
6      const userId = ctx.user.id;
7
8      const [video] = await db
9        .select({ ...getTableColumns(videos) })
10       .from(videos)
11       .where(and(eq(videos.id, input.id), eq(videos.userId, userId)));
12
13     //...
14   })
```

Como se puede observar, se incluyó una nueva comprobación al recuperar el video: `eq(videos.userId, userId)`. Además, se convirtió el procedimiento en protegido para obtener la sesión del usuario que lo invoca. Con esto, se evita que usuarios no creadores de un video sean capaces de recuperar los datos dentro del contexto de `/studio` y actualizar su miniatura.

Estas mismas medidas fueron tomadas para otros procedimientos que sufrían este problema.

Algo similar sucedía en la lógica de negocio de los *endpoints* de Bunny 7.3, donde se omitía la existencia de una sesión y un usuario válido, provocando operaciones no deseadas en la aplicación. Esto se resolvió añadiendo una comprobación adicional antes de ejecutar cada acción, en la que se verifica la presencia de una sesión activa y un usuario legítimo.

8.3.2. Webhook de Bunny

Cuando *Bunny* termina de procesar un video, este envía una notificación a un *webhook* definido en el backend de la aplicación. Sin embargo, se debe comprobar que la petición recibida viene solamente de *Bunny.net*. En caso contrario, cualquier atacante podría enviar una solicitud de actualización falsa y actualizaría datos de un video erróneamente. Este ha resultado ser el error de seguridad más grave, ya que el *webhook* actualiza un gran número de atributos del video, incluyendo su enlace de reproducción. Por consiguiente, sin verificar quién envía la petición, un atacante conseguía cambiar el video por completo y arruinar seriamente el servicio.

Esto se solucionó incluyendo un parámetro variable en el *webhook* conocido únicamente por *Bunny* y *Booster*. Cuando *Bunny* envía una solicitud al *webhook* incluye un token

Capítulo 8. Despliegue y Riesgos de Seguridad

en la URL (secreto de webhook). Es decir, invoca a `/api/bunny/webhooks/[token]`. Como la petición viaja por HTTPS, un atacante no puede interceptar el token ya que está cifrado. Una vez el mensaje llega al servidor, este se encarga de verificar que el token es el mismo que él tiene. Como solo lo saben *Bunny* y el backend de *Booster*, se entiende que la petición es legítima y se continúa con el proceso. En caso contrario, se aborta la operación.

Esto se implementó de la siguiente forma:

```
1 // app/api/bunny/webhooks/[token]/route.ts
2
3 export async function POST( req: Request, { params }: { params: { token: string } }) {
4
5   //Verificar que es Bunny quien envía la petición
6   if (params.token !== process.env.BUNNY_WEBHOOK_SECRET) {
7     return new Response("Not found", { status: 404 });
8   }
9   //...
10 }
```

8.3.3. Inyección SQL detectada

Durante el proceso de pruebas, se detectó una inyección SQL en el procedimiento `getMore` dentro de `/videos`. La inyección SQL se encontraba en esta parte del procedimiento:

```
1 sql.raw(` ${currentTags.join(',')} `)
```

El problema es que se está construyendo manualmente un literal SQL de array con strings que vienen de un usuario. Debido al `join()`, cualquier valor dentro de `currentTags` entraba en el SQL final sin escapar de forma segura. `sql.raw()` no parametriza y no protege los fragmentos que se introducen. Si `currentTags` tiene caracteres especiales se puede romper el literal SQL e interpretar como instrucciones.

Por ejemplo

```
1 currentTags = ["math", "finance' } OR true --"]
```

resultaba en

```
1 videos.tags && ' {math,finance' } OR true --'
```

Para solucionar este problema, se hizo lo siguiente

```
1 const tagsOverlapExpr =
2   currentTags && currentTags.length > 0
3   ? arrayOverlaps(videos.tags, currentTags)
4   : sql`false`
```

Aquí, `arrayOverlaps` permite que Drizzle construya la expresión de forma segura.

8.3.4. Limitaciones de uso (Rate Limits)

Para evitar abusos del servicio se ha adoptado un restrictor de uso (Rate Limit) mediante Redis. Con esto, se evita que se realicen un gran número de operaciones en poco tiempo y se alivia la carga del servidor y la base de datos. Todos estos límites de uso están definidos en `/src/lib/ratelimit.ts` y establecen el número máximos de operaciones durante un intervalo de tiempo que puede realizar un usuario.

Por ejemplo, en la aplicación solo se permiten subir 5 videos diarios a un único usuario. Esta limitación se define en `ratelimit.ts` de la siguiente forma:

```
1 export const uploadRateLimit = new Ratelimit({
2   redis: redis,
3   limiter: Ratelimit.slidingWindow(5, "1d"),
4   prefix: "@upstash/ratelimit/upload",
5 });
```

Posteriormente, en el endpoint del *backend* donde se intenta crear un nuevo registro de video se comprueba si se ha superado este límite en el tiempo establecido antes de continuar con la operación. En caso afirmativo, se devuelve un código de error 429 y se informa al usuario de ello. Por otro lado, se permite crear un nuevo video y Redis registra una petición de carga del usuario en la ventana deslizante de 1 día.

```
1 const { success } = await uploadRateLimit.limit(userId);
2 if (!success) {
3   return NextResponse.json(
4     { error: "Daily upload limit reached" },
5     { status: 429 }
6   );
7 }
```

Se implementan más límites de uso para evitar abusos hacia la plataforma y mejorar la seguridad de la misma. Por defecto, un usuario solo puede invocar 20 operaciones cada 10 segundos pero además, se establecen restricciones más específicas para otros casos como: cantidad de comentarios enviados (5 en 1 minuto), visitas a un video (1 cada 12 horas), puntos de experiencia ganados por video (20 puntos por cada video en un día), entre otros.

Además de estas restricciones, en todos los campos donde el usuario introduce texto, se ha incluido un límite de caracteres.

Capítulo 9

Conclusiones

A lo largo de este trabajo se ha abordado el proceso de análisis, diseño, desarrollo y puesta en marcha de una aplicación de software. El primer paso fue analizar los problemas existentes en las plataformas de video actuales con el objetivo de crear una alternativa que solucione estos inconvenientes y, en definitiva, brinde un mejor servicio a un menor costo. A partir del diagnóstico inicial, se diseñó una solución tecnológica orientada a resolver las necesidades de los usuarios, explorando distintos patrones arquitectónicos y herramientas.

Durante la ejecución del proyecto, se analizaron diferentes proveedores, herramientas, servicios y tecnologías disponibles para implementar una aplicación de streaming de video. Esto permitió comparar alternativas para procesos clave como el almacenamiento de video, la transcodificación y segmentación de videos, la distribución de contenido, la autenticación, el despliegue y la lógica de negocio. Asimismo, se evaluaron las ventajas y desventajas de cada opción teniendo en cuenta los costos, sus limitaciones y su grado de complejidad. También se presentaron casos de uso públicos en los que estas tecnologías son utilizadas por empresas o plataformas reales. Esto brindó una visión más clara de la utilidad de las herramientas y demuestra que el proyecto incorpora criterios de análisis propios de un entorno profesional de desarrollo de software. Esta etapa fue fundamental para comprender que el desarrollo de una aplicación moderna, además de escribir código, implica analizar servicios de terceros, evaluar alternativas y tomar decisiones de diseño coherentes con los objetivos del proyecto.

La puesta en marcha del servicio web de la plataforma también fue un aspecto importante del trabajo. Esto significó afrontar desafíos en un entorno de producción, resolver problemas de configuración, gestionar errores, integrar servicios de terceros, controlar la estabilidad del sistema y monitorear los recursos utilizados por la aplicación. Además, se realizaron actividades publicitarias creando un video anuncio y realizando publicaciones en redes sociales para atraer usuarios. Esto, a su vez, implicó una observación cuidadosa de los recursos consumidos por el servicio. Con un incremento en el volumen de usuarios, era necesario vigilar los límites para garantizar la continuidad del sistema y evitar caídas abruptas.

Otro aspecto fundamental del proyecto fue la identificación de vulnerabilidades. Durante el desarrollo se detectaron problemas relacionados con la seguridad que afectaban seriamente la calidad del servicio. Además, se introdujeron límites de uso para aliviar la carga sobre el sistema y evitar un uso indebido de los recursos. Estas vulnerabilidades fueron analizadas y mitigadas mediante mejoras en el código y cambios en la arquitectura.

Capítulo 9. Conclusiones

En definitiva, el proyecto ha permitido experimentar de forma completa el proceso de creación de una aplicación de software y de un proyecto empresarial. Se tomaron decisiones de negocio orientadas a la optimización de costos y se realizaron campañas publicitarias en redes sociales para captar usuarios. Además, se enfrentaron desafíos tecnológicos y problemas técnicos para desarrollar y mantener una plataforma en internet.

Como resultado, el trabajo demuestra el proceso de construcción de una solución tecnológica a partir de un problema, aplicando conceptos técnicos y estratégicos para crear una plataforma de streaming de video alternativa y mejorada.

Capítulo 10

Análisis de Impacto

Con el desarrollo del proyecto se han identificado distintos contextos donde tiene impacto. Desde el punto de vista personal, el trabajo permitió adquirir experiencia práctica en el análisis, diseño, desarrollo, despliegue y mantenimiento de una aplicación software completa. Además, se tomaron decisiones técnicas para alcanzar los objetivos de la forma más óptima posible. Desde el punto de vista empresarial, el proyecto demuestra la viabilidad de construir una plataforma de streaming de contenido en línea. Se analizaron proveedores, tecnologías y herramientas teniendo en cuenta los costos, y además se diseñaron campañas publicitarias para atraer a nuevos usuarios.

Desde un punto de vista social, una plataforma de este tipo facilita la difusión de contenido y la creación de comunidades. No obstante, también puede generar efectos sociales adversos, como la distribución de contenido dañino o inapropiado, lo cual supuso crear mecanismos de moderación.

En cuanto al contexto económico, el uso de servicios y proveedores externos en la nube permite reducir la inversión inicial. Sin embargo, esto introduce costos variables que aumentan con el volumen de usuarios en la plataforma. Por esta razón, se analizaron los servicios y proveedores considerando estos factores, así como su complejidad y escalabilidad.

El proyecto tiene un impacto medioambiental debido al consumo energético asociado al procesamiento, almacenamiento, distribución de videos y a las operaciones de la plataforma. La incorporación de límites de uso para los usuarios reduce la utilización de recursos de la aplicación y por tanto, su consumo energético.

Finalmente, según los Objetivos de Desarrollo Sostenible, el proyecto se alinea con ODS 4 (Educación de calidad) por su potencial para difundir contenido educativo; ODS 8 (Trabajo decente y crecimiento) porque crea oportunidades laborales para los creadores de contenido, quienes son remunerados por publicar en la plataforma; y ODS 9 (Industria, innovación e infraestructura) al desarrollar una solución tecnológica con herramientas y servicios modernos.

Bibliografía

- [1] A. Inc. «Alphabet Announces Fourth Quarter and Fiscal Year 2025 Results», visitado 26 de mar. de 2026. dirección: https://s206.q4cdn.com/479360582/files/doc_financials/2025/q4/2025q4-alphabet-earnings-release.pdf
- [2] DataReportal. «Digital 2025: Global Overview Report», visitado 12 de abr. de 2026. dirección: <https://datareportal.com/reports/digital-2025-global-overview-report>
- [3] TikTok. «TikTok's community in Europe Tops 200 Million», visitado 26 de mar. de 2026. dirección: https://newsroom.tiktok.com/tiktok-community-in-europe-tops-200-million?from_seo_redirect=1&lang=en-150
- [4] I. Netflix. «FINAL Q4 25 Shareholder Letter», visitado 26 de mar. de 2026. dirección: https://s22.q4cdn.com/959853165/files/doc_financials/2025/q4/FINAL-Q4-25-Shareholder-Letter.pdf
- [5] I. Meta Platforms. «Meta Reports Fourth Quarter and Full Year 2025 Results», visitado 26 de mar. de 2026. dirección: <https://investor.atmeta.com/investor-news/press-release-details/2026/Meta-Reports-Fourth-Quarter-and-Full-Year-2025-Results/default.aspx>
- [6] International Energy Agency. «Energy and AI», visitado 26 de mar. de 2026. dirección: <https://www.iea.org/reports/energy-and-ai/energy-demand-from-ai>
- [7] Ofcom. «Adults' Media Use and Attitudes 2025», visitado 26 de mar. de 2026. dirección: <https://www.ofcom.org.uk/siteassets/resources/documents/research-and-data/media-literacy-research/adults/adults-media-use-and-attitudes-2025/adults-media-use-and-attitudes-report-2025.pdf?v=396240>
- [8] Ofcom. «Online Nations Report 2025», visitado 26 de mar. de 2026. dirección: <https://www.ofcom.org.uk/siteassets/resources/documents/research-and-data/online-research/online-nation/2025/online-nations-report-2025.pdf?v=409837>
- [9] X. Lu, X. Hou, N. Yuan y J. Wang. «Allow or not? Strategic choices of competing platforms in response to ad blockers», visitado 26 de mar. de 2026. dirección: <https://www.sciencedirect.com/science/article/abs/pii/S037872062500120X>

- [10] P. Ranganathan, D. Stodolsky, J. Calow et al. «Warehouse-Scale Video Acceleration: Co-design and Deployment in the Wild», visitado 28 de mar. de 2026. dirección: <https://gwern.net/doc/cs/hardware/2021-ranganathan.pdf>
- [11] I. Stephanie Susnjara. «¿Qué es Docker?», visitado 28 de abr. de 2026. dirección: <https://www.ibm.com/es-es/think/topics/docker>
- [12] MPEG. «MPEG-DASH», visitado 26 de mar. de 2026. dirección: <https://www.mpeg.org/standards/MPEG-DASH/>
- [13] Apple Inc. «HTTP Live Streaming», visitado 26 de mar. de 2026. dirección: <https://developer.apple.com/streaming/>
- [14] Mux Video. «Play your Videos | Mux», visitado 26 de mar. de 2026. dirección: <https://www.mux.com/docs/guides/play-your-videos>
- [15] H. P. G. T. F. Casper Haems Jeroen van der Hooft. «Hybrid Unicast–Broadcast Video Delivery for Scalable Low-Latency Live Streaming», visitado 26 de mar. de 2026. dirección: <https://dl.acm.org/doi/10.1145/3742868>
- [16] Google Cloud. «Google Cloud Storage», visitado 26 de mar. de 2026. dirección: <https://cloud.google.com/learn/what-is-object-storage>
- [17] Cloudflare. «What is a CDN?», visitado 26 de mar. de 2026. dirección: <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
- [18] Mux Video. «Mux Video Pricing», visitado 28 de mar. de 2026. dirección: <https://www.mux.com/pricing>
- [19] Bunny.net. «BunnyStream Pricing», visitado 28 de mar. de 2026. dirección: <https://bunny.net/pricing/stream/>
- [20] W3C. «ActivityPub», visitado 28 de mar. de 2026. dirección: <https://activitypub.rocks/>
- [21] Neon. «The pgvector extension», visitado 31 de mar. de 2026. dirección: <https://neon.com/docs/extensions/pgvector>
- [22] Drizzle. «Drizzle ORM - Why Drizzle?», visitado 31 de mar. de 2026. dirección: <https://orm.drizzle.team/docs/overview>
- [23] tRPC. «Concepts | tRPC», visitado 31 de mar. de 2026. dirección: <https://trpc.io/docs/concepts>
- [24] Next.js. «Next.js Docs: App Router», visitado 31 de mar. de 2026. dirección: <https://nextjs.org/docs/app>
- [25] Clerk. «Welcome to Clerk Docs», visitado 1 de abr. de 2026. dirección: <https://clerk.com/docs>
- [26] shadcn | Vercel. «The Foundation for your Design System», visitado 1 de abr. de 2026. dirección: <https://ui.shadcn.com/>
- [27] European Commission. «Commission sends requests for information to YouTube, Snapchat, and TikTok on recommender systems under the Digital Services Act», visitado 1 de abr. de 2026. dirección: <https://digital-strategy.ec.europa.eu/en/news/commission-sends-requests-information-youtube-snapchat-and-tiktok-recommender-systems-under-digital>

- [28] Raman Media Network. «YouTube Faces User Backlash Over Ad Overload, Threatening Platform's Future», visitado 3 de abr. de 2026. dirección: <https://www.ramanmedianetwork.com/youtube-faces-user-backlash-over-ad-overload-threatening-platforms-future/>
- [29] A. Shrestha, S. Shrestha, B. Paneru y B. Paneru, *An Exploration of Effects of Dark Mode on University Students: A Human Computer Interface Analysis*, 2024. arXiv: 2409.10895 [cs.HC]. dirección: <https://arxiv.org/abs/2409.10895>
- [30] Google. «Implement Dark Theme», visitado 18 de abr. de 2026. dirección: <https://developer.android.com/develop/ui/views/theming/darktheme>
- [31] Vercel. «File-system conventions: Route Groups | NextJS», visitado 27 de abr. de 2026. dirección: <https://nextjs.org/docs/app/api-reference/file-conventions/route-groups>
- [32] Clerk. «Next.js Quickstart (App Router) - Getting started», visitado 28 de abr. de 2026. dirección: <https://clerk.com/docs/nextjs/getting-started/quickstart>
- [33] Bunny.net. «HTTP - bunny.net Documentation», visitado 28 de abr. de 2026. dirección: <https://docs.bunny.net/stream/http-api#how-to-create-a-video>
- [34] Bunny.net. «TUS Resumable Uploads - bunny.net Documentation», visitado 28 de abr. de 2026. dirección: <https://docs.bunny.net/stream/tus-resumable-uploads>